

From Neural Network to Hardware Accelerator

Understanding the ONNX to RISC-V + Gemmini Compilation Pipeline

A Practical Guide for Systems Programmers

Technical Documentation

July 10, 2026

Chapter 0: Concepts You Need Before the Pipeline

How Chapter 0 is organized

The rest of the document is long because it follows the complete path from an ONNX model to a RISC-V executable. To avoid confusion, read the system as **three different flows** instead of one overloaded graph:

1. **Compiler flow:** transforms `resnet18.onnx` into `resnet18.o`.
2. **Link flow:** combines `resnet18.o`, the runner, and runtime libraries into `resnet18_rv64`.
3. **Execution flow:** runs `resnet18_rv64` under Spike or on real Gemmini hardware.

Each diagram below uses one flow only. This prevents mixing compiler concepts such as *lowering* with runtime concepts such as *OMTensor* or hardware concepts such as *RoCC*.

0.0 Visual Roadmap: Three Separate Flows

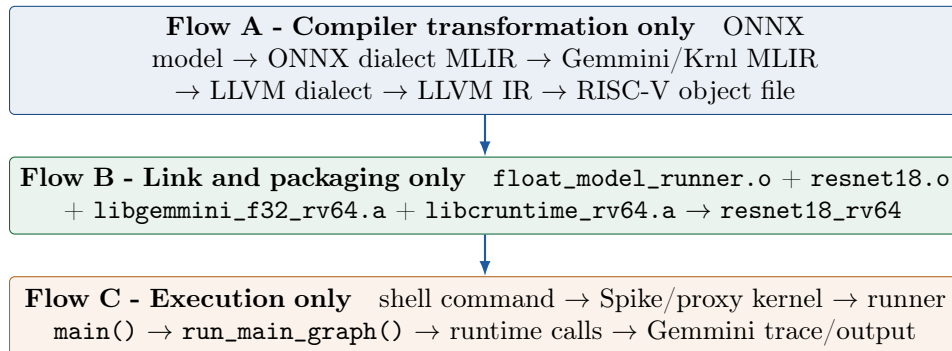


Figure 1: The document uses three separate mental flows. Do not read linking artifacts as compiler passes, and do not read runtime logs as MLIR transformations.

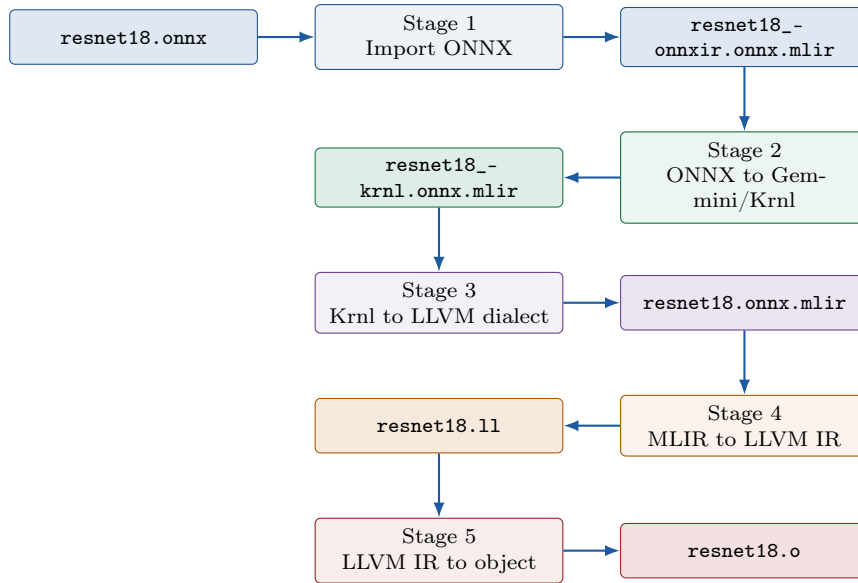


Figure 2: Compiler flow only: this graph stops at `resnet18.o`. It does not include runtime compilation or Spike execution.

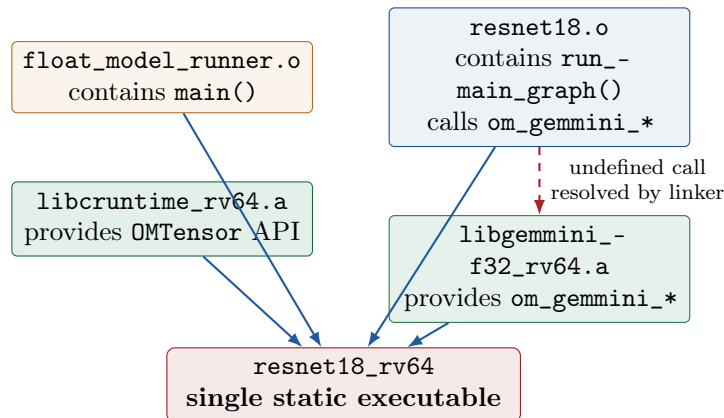


Figure 3: Link flow only: this graph explains how incomplete objects and runtime archives become one executable.

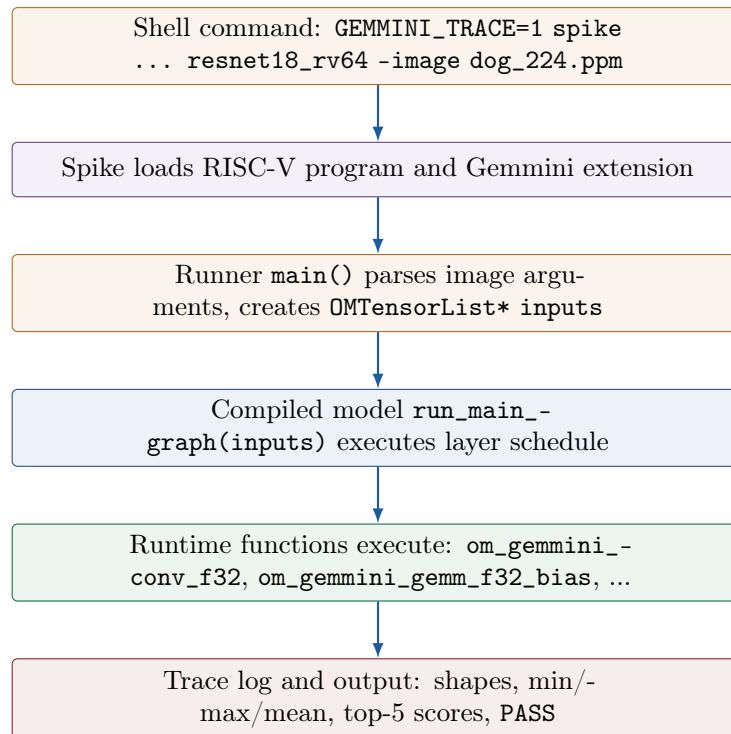


Figure 4: Execution flow only: this graph starts after linking. It explains what happens when the executable is actually run.

0.0.1 Complete Definition Map

Definition group 1 - Model representation

Term	Definition	Project example
ONNX	Portable open format for AI models. It represents the model as a graph with operators, tensors, data types, and initializers.	<code>resnet18.onnx</code> is the input model file.
Graph	Directed computation structure: nodes consume values and produce values.	ResNet-18 graph contains Conv, BatchNorm, Relu, Add, Pool, Gemm, Softmax.
Node / operator	One operation in the graph.	Conv computes feature maps; Add implements residual skip connections.
Tensor	Multi-dimensional typed data object.	Input image tensor: <code>[1,3,224,224]</code> float32.
Initializer	Constant tensor stored in the model file.	Convolution weights <code>[64,3,7,7]</code> .
Shape	Dimension list of a tensor.	<code>[N,C,H,W] = [1,3,224,224]</code> .
Rank	Number of dimensions.	<code>[1,3,224,224]</code> has rank 4.
Data type	Type of each element.	f32, i8, i64.
NCHW	Image layout: batch, channels, height, width.	ResNet input is NCHW.
OIHW	Conv weight layout: output channels, input channels, kernel height, kernel width.	First conv weight: <code>[64,3,7,7]</code> .

Definition group 2 - Compiler and IR

Term	Definition	Why it is used here
Compiler	Program that transforms one representation into another executable-oriented representation.	ONNX-MLIR transforms ONNX into MLIR/LLVM/RISC-V objects.
IR	Intermediate Representation: internal language used by compilers.	Easier to analyze and transform than raw ONNX protobuf or machine code.
MLIR	Multi-Level Intermediate Representation framework.	Allows high-level ONNX ops, loop-like Krnl ops, accelerator ops, and LLVM-like ops to coexist during lowering.
Dialect	A family of MLIR operations and types at one abstraction level.	ONNX dialect, Krnl dialect, Gemmini dialect, LLVM dialect.
Pass	Compiler transformation run over the IR.	Example: ONNX-to-Gemmini lowering pass.
Lowering	Replacing high-level operations with lower-level operations.	<code>onnx.Conv</code> becomes <code>krnl.call @om_gemmini_conv_f32</code> .
SSA value	A compiler temporary assigned once.	<code>%0</code> , <code>%1</code> , <code>%y</code> .
Attribute	Static property attached to an operation.	<code>strides=[2,2]</code> , <code>pads=[3,3,3,3]</code> , <code>group=1</code> .
memref	MLIR memory reference type; closer to a buffer than an abstract tensor.	Used after tensor-level lowering when code must access memory.
krnl.global	Krnl global constant or buffer symbol.	Holds model weights before final read-only ELF data.
krnl.call	Runtime call emitted in Krnl-level IR.	Calls <code>om_gemmini_conv_f32</code> .
LLVM dialect	MLIR dialect whose operations map to LLVM IR semantics.	Last MLIR form before exporting <code>resnet18.ll</code> .

Definition group 3 - Runtime, linking, and execution

Term	Definition	Project example
Runtime	Library code used by the generated program while it runs.	ONNX-MLIR C runtime and Gemmini runtime.
Header file	Declares functions/types to the compiler.	OMRuntimeGemmini.hpp declares runtime entry points.
Source file	Implements functions.	OMRuntimeGemmini.cpp implements om_gemmini_conv_f32.
Object file	Compiled machine code, maybe with unresolved symbols.	resnet18.o, OMRuntimeGemmini.o.
Static archive	Library file containing object files.	libgemmini_f32_rv64.a.
Linker	Tool that merges object files and resolves symbols.	Produces resnet18_rv64.
Symbol	Name of a function/global in an object file.	om_gemmini_conv_f32.
Undefined symbol	Symbol called but not implemented in that object.	U om_gemmini_conv_f32 in resnet18.o.
Defined symbol	Symbol implemented in a file/library.	T om_gemmini_conv_f32 after linking.
OMTensor	ONNX-MLIR runtime tensor object containing data pointer, shape, rank, strides, data type, and ownership metadata.	Runner creates one input OMTensor for the image.
OMTensorList	Container holding one or more OMTensor* values.	run_main_graph() receives inputs and returns outputs as tensor lists.
run_main_graph()	Generated entry function for the compiled model.	Runner calls it to execute ResNet-18.

Definition group 4 - RISC-V, Gemmini, and simulation

Term	Definition	Project example
RISC-V	Open instruction set architecture.	Target architecture for <code>resnet18_rv64</code> .
Target triple	String describing target CPU/OS/ABI family.	<code>riscv64-unknown-linux-gnu</code> .
ABI	Calling convention and binary interface rules.	<code>lp64d</code> controls integer/pointer sizes and floating-point ABI.
RoCC	Rocket Custom Coprocessor interface for custom accelerators.	Gemmini is controlled via RoCC/-custom instructions.
Gemmini	Systolic-array matrix accelerator for RISC-V systems.	Used for conv/matmul/GEMM-like work.
Systolic array	Grid of processing elements that passes data rhythmically through multiply-accumulate units.	Trace shows <code>dim = 16</code> .
Scratchpad	Explicit accelerator-local memory.	Runtime packs tiles/im2col data before Gemmini compute.
Spike	RISC-V ISA simulator.	Command uses <code>spike -extension=gemmini</code> .
Proxy kernel (pk)	Lightweight runtime used to launch RISC-V ELF programs in Spike.	<code>/usr/local/riscv/.../pk</code> .
GEMMINI_-TRACE	Environment flag enabling runtime trace prints.	Produces <code>[gemmini-trace] conv f32:</code>

0.0.2 Source-backed Reference Notes

Official references used for definitions

The definitions in this chapter follow the official ONNX, ONNX-MLIR, MLIR, LLVM dialect, OMTensor, and Clang/LLVM documentation. The project-specific Gemmini behavior follows the uploaded `OMRuntimeGemmini.cpp` and trace logs used in the later runtime chapters.

- ONNX: <https://github.com/onnx/onnx>
- ONNX-MLIR: <https://onnx.ai/onnx-mlir/>
- MLIR overview: <https://mlir.llvm.org/>
- MLIR LLVM dialect: <https://mlir.llvm.org/docs/Dialects/LLVM/>
- OMTensor API: https://onnx.ai/onnx-mlir/doxygen_html/OMTensor_h/_o_m_tensor_8h.html

- Clang compilation stages: <https://clang.llvm.org/docs/CommandGuide/clang.html>

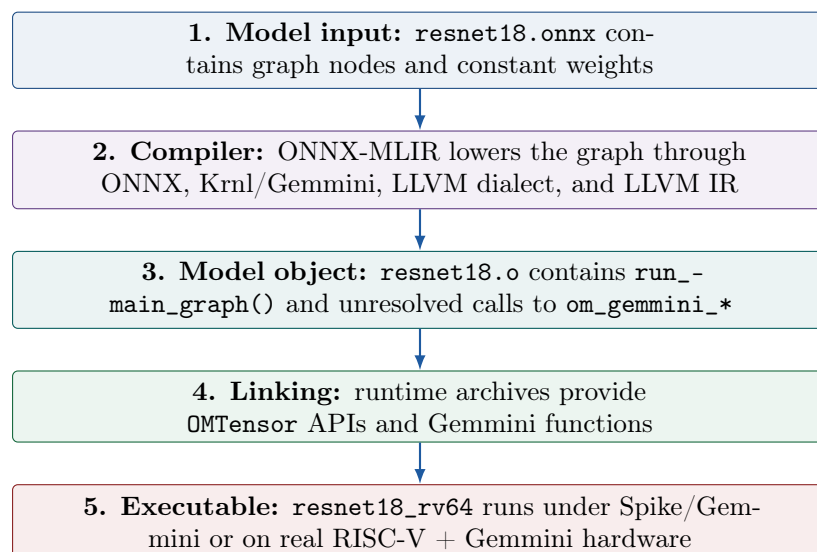
Purpose of this Chapter

This chapter explains the basic vocabulary before the real build stages begin. It answers these questions:

- What is **ONNX-MLIR**?
- What is a **runtime library**, and why is it needed?
- What is **linking**, and why do we need object files and static libraries?
- What are **OMTensor**, **OMTensorList**, and **run_main_graph()**?
- How does one ONNX operator, such as **Conv**, move from a model graph to a runtime function and finally to Gemmini?

The goal is to remove ambiguity before reading the command-by-command pipeline. After this chapter, every later stage should have a clear role: **parse**, **lower**, **generate code**, **compile runtime**, **link**, and **run**.

0.1 The Whole Story in One Sentence



Plain meaning: ONNX-MLIR converts the neural-network graph into RISC-V code. The generated code does not implement every tensor operation alone. Instead, it calls runtime functions such as `om_gemmini_conv_f32`. The linker then connects those calls to the compiled runtime implementation inside `libgemmini_f32_rv64.a`.

0.2 Core Definitions

Term	Simple Definition	In This Project
ONNX	Open Neural Network Exchange. A portable file format for neural-network graphs.	<code>resnet18.onnx</code> contains the ResNet-18 computation graph and trained weights.
ONNX node	One operation in the model graph.	Examples: <code>Conv</code> , <code>Relu</code> , <code>Add</code> , <code>Gemm</code> , <code>Softmax</code> .
Initializer	Constant tensor stored inside the ONNX file.	Weights and biases become constants such as <code>krnl.global</code> or read-only data in the final executable.
ONNX-MLIR	A compiler that imports ONNX models into MLIR and lowers them to executable code.	It transforms <code>onnx.Conv</code> into lower-level code and, for Gemmini-supported cases, into calls to Gemmini runtime functions.
MLIR	Multi-Level Intermediate Representation. A compiler framework with many internal languages called dialects.	The model moves through ONNX dialect, Krnl dialect, Gemmini-related dialects, LLVM dialect, and finally LLVM IR.
Dialect	A group of operations used at one compiler abstraction level.	ONNX dialect is high-level; Krnl is loop/runtime-oriented; LLVM dialect is close to machine code.
Runtime	Library code needed by generated model code at execution time.	<code>OMRuntimeGemmini.cpp</code> implements functions like <code>om_gemmini_conv_f32</code> ; ONNX-MLIR runtime implements <code>OMTensor</code> .
Linking	Final step that connects unresolved function calls to their implementations.	<code>resnet18.o</code> calls <code>om_gemmini_conv_f32</code> ; <code>libgemmini_f32_rv64.a</code> provides it.
Executable	Final ELF program that can run on the target system.	<code>resnet18_rv64</code> contains the model, weights, runtime code, and runner entry point.

Table 1: Chapter 0 vocabulary used throughout the document.

0.3 What ONNX-MLIR Actually Does

ONNX-MLIR = model compiler, not just a converter

ONNX-MLIR does more than read an ONNX file and print another file. It performs a sequence of compiler transformations:

1. **Import:** read ONNX protobuf and create MLIR ONNX operations.
2. **Shape/type reasoning:** understand tensor ranks, dimensions, and data types.

3. **Lowering:** replace high-level ONNX operations with lower-level operations.
4. **Accelerator selection:** when `-maccel=Gemmini` is enabled, route eligible operators to Gemmini runtime calls.
5. **LLVM lowering:** convert MLIR objects into LLVM-compatible operations.
6. **Object generation:** produce RISC-V object code that the linker can combine with runtime libraries.

CPU compiler analogy

For a C program, the compiler transforms `for` loops and function calls into machine instructions. For an ONNX model, ONNX-MLIR transforms `Conv`, `Relu`, `Add`, and `Gemm` nodes into executable code and runtime calls. The model graph becomes a normal program with functions, memory, constants, and external symbols.

0.4 Why We Need a Runtime

The compiler-generated model code cannot reasonably contain every low-level tensor implementation inline. Instead, ONNX-MLIR emits calls to runtime functions. There are two runtime families in this project:

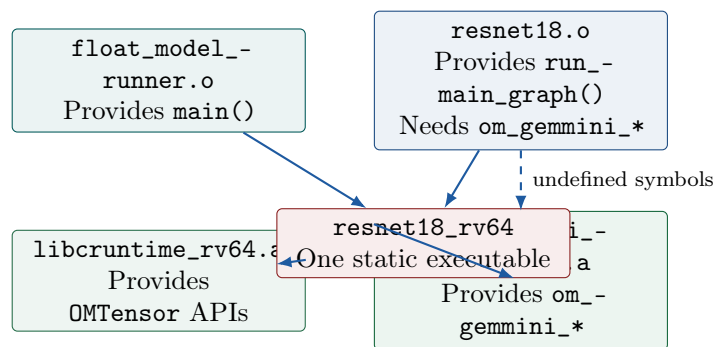
Runtime Part	Responsible Files	Purpose
ONNX-MLIR C runtime	<code>src/Runtime/OMTensor.c</code> , <code>OMTensorList.c</code> , <code>OnnxDataType.c</code>	Creates and manages tensor containers used by generated code and the runner.
Gemmini runtime	<code>src/Accelerators/Gemmini/Runtime/OMRuntimeGemmini.cpp</code> , <code>OMRuntimeGemmini.hpp</code>	Runtime/OMRuntimeGemmini.cpp, functions such as <code>om_gemmini_conv_f32</code> , <code>om_gemmini_gemm_f32_bias</code> , pooling, softmax, and scalar helper ops.
Gemmini hardware ABI	<code>gemmini-hardware-abi/include</code> and RoCC headers	Provides the low-level Gemmini interface used by the runtime to issue or emulate accelerator commands.

Table 2: The two runtime layers and why they are separate.

Important separation

The compiled model decides what to call. The runtime implements what happens when that call is executed. For example, `resnet18.o` can contain a call to `om_gemmini_conv_f32`, but the real implementation is inside `libgemmini-f32_rv64.a`.

0.5 Linking: Why the Executable Needs Object Files and Libraries



Artifact	Role in Linker	Meaning
.o file	Object file	Compiled machine code, but not necessarily complete. It may still have unresolved external symbols.
.a file	Static archive	A library made of object files. The linker extracts only the members needed to satisfy unresolved symbols.
U in nm	Undefined symbol	The object calls this function, but the definition is not in that object file. The linker must find it in another object or library.
T in nm	Text symbol	The function is defined in executable code. After linking, <code>om_gemmini_conv_f32</code> should appear as a defined text symbol.
-static	Static link option	Copies needed library code into the executable so the target does not need shared libraries at runtime.

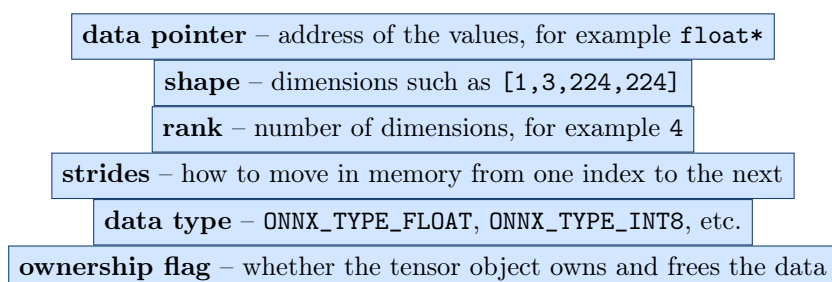
Table 3: Linking vocabulary.

0.6 OMTensor and OMTensorList

Definition: OMTensor

OMTensor is the ONNX-MLIR runtime object that represents one tensor at execution time. It is not just a raw pointer. It carries the tensor data plus metadata required by generated model code and runtime functions.

Conceptual OMTensor layout



```
1 int64_t inputShape[4] = {1, 3, 224, 224};
2 float *inputData = malloc(sizeof(float) * 1 * 3 * 224 * 224);
3 OMTensor *input = omTensorCreateWithOwnership(
4     inputData, inputShape, 4, ONNX_TYPE_FLOAT, 1);
5 OMTensorList *inputs = omTensorListCreate(&input, 1);
6 OMTensorList *outputs = run_main_graph(inputs);
```

Listing 1: Conceptual use of OMTensor in the runner

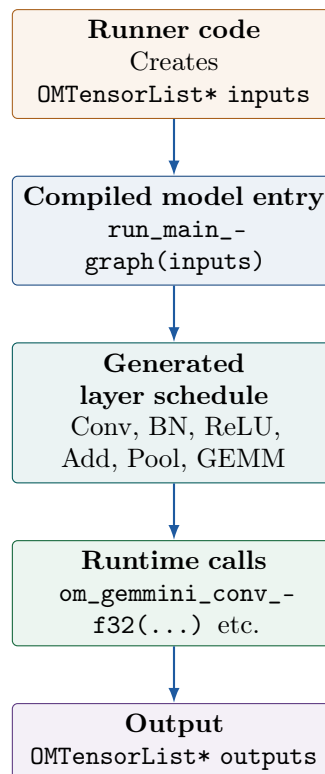
Line-by-line meaning:

- `inputShape`: tells the compiled model the tensor is NCHW: batch 1, channels 3, height 224, width 224.
- `inputData`: raw memory buffer containing the actual floating-point image values.
- `omTensorCreateWithOwnership`: wraps raw memory into an ONNX-MLIR tensor object.
- `omTensorListCreate`: wraps one or more input tensors into the list expected by `run_main_graph()`.
- `run_main_graph`: executes the compiled ONNX model and returns an output tensor list.

0.7 What is `run_main_graph()`?

Definition: `run_main_graph()`

`run_main_graph()` is the public entry point generated by ONNX-MLIR for the compiled model. The runner program calls it exactly like a normal C function. Internally, it executes the model graph in the correct order.



Do not confuse these two functions

`main()` belongs to the test runner. It prepares input data and prints results. `run_main_graph()` belongs to the compiled model. It executes the neural-network graph.

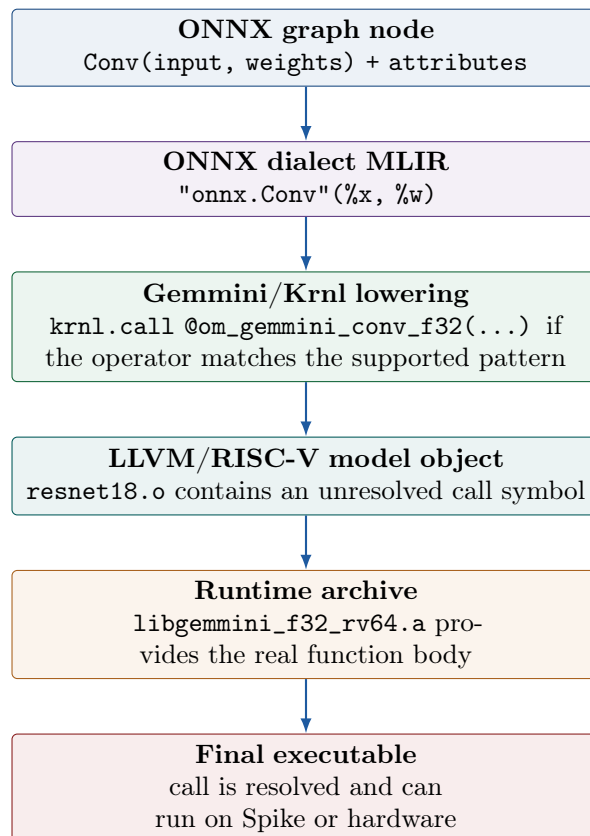
0.8 Syntax You Will See in the Compiler Outputs

Syntax	Where It Appears	Meaning
<code>%0, %1</code>	MLIR / LLVM IR	SSA values. Think of them as temporary variables created by compiler operations.
<code>"onnx.Conv"</code>	ONNX dialect MLIR	A high-level ONNX convolution operation imported from the model graph.
<code>(%x, %w, %b)</code>	MLIR operands	Inputs to an operation, similar to function arguments.
<code>strides = [2,2]</code>	ONNX op attribute	Static configuration attached to the operator. It is not a runtime tensor.
<code>tensor<1x3x224x224x...></code>	ONNX dialect type	Compile-time tensor type: rank 4, shape NCHW, float32 values.
<code>memref<...></code>	Lowered MLIR	Memory reference. This is closer to an allocated array/buffer than an abstract tensor.

Syntax	Where It Appears	Meaning
<code>krnl.global</code>	Krnl MLIR	A global constant, usually model weights or bias values. Later it becomes read-only data in the executable.
<code>krnl.call</code>	Krnl MLIR	A call from generated model code to a runtime function. Example: <code>krnl.call @om_gemmini_conv_f32</code> .
<code>llvm.call</code>	LLVM dialect / LLVM IR	A lower-level function call using LLVM-compatible types such as pointers and integers.
<code>ptr</code>	LLVM IR	Generic pointer type. At this level, rich tensor objects have been lowered to pointers.
<code>declare void @f(...)</code>	LLVM IR	Function is used by this file but implemented somewhere else. The linker must resolve it.
<code>extern "C"</code>	C++ runtime header	Prevents C++ name mangling so generated C/LLVM calls can link to C++ implementations by exact symbol name.
<code>U om_gemmini_conv_f32</code>	<code>nm resnet18.o</code>	Undefined symbol in the model object. This proves the model object calls the runtime but does not implement it.
<code>T om_gemmini_conv_f32</code>	<code>nm resnet18_rv64</code>	Defined text symbol in the final executable. This proves linking found the implementation.

Table 4: Syntax dictionary for reading ONNX-MLIR outputs.

0.9 Tracking One Convolution Before the Pipeline Starts



0.10 Example: Reading an ONNX Conv in MLIR

```

1 %y = "onnx.Conv"(%x, %w) {
2   group = 1,
3   dilations = [1, 1],
4   kernel_shape = [7, 7],
5   pads = [3, 3, 3, 3],
6   strides = [2, 2]
7 } : (tensor<1x3x224x224xf32>,
8     tensor<64x3x7x7xf32>) -> tensor<1x64x112x112xf32>

```

Listing 2: Typical ONNX dialect Conv syntax

Part	Meaning	Why It Matters
<code>%y</code>	Output SSA value.	Later operations consume this feature map.
<code>"onnx.Conv"</code>	ONNX convolution operation.	This is the high-level operator that lowering passes inspect.
<code>%x</code>	Input activation tensor.	For ResNet-18 first layer, it has shape <code>[1,3,224,224]</code> .
<code>%w</code>	Weight tensor.	For the first layer, it has shape <code>[64,3,7,7]</code> .
<code>group = 1</code>	Standard convolution, not grouped convolution.	A simple supported case for accelerator lowering.
<code>dilations = [1,1]</code>	Kernel samples adjacent pixels.	Avoids the more complex dilated-convolution indexing pattern.
<code>pads = [3,3,3,3]</code>	Symmetric zero padding.	The runtime can compute output coordinates with one pad value.
<code>strides = [2,2]</code>	Moves the kernel by 2 pixels in both H and W.	Reduces spatial output from 224x224 to 112x112.
<code>tensor<...xf32></code>	Compile-time tensor type.	Tells the compiler the rank, dimensions, and element type.

Table 5: How to read one ONNX Conv operation.

0.11 Why Multi-Step Lowering is Useful

Benefit	Explanation	Concrete Result
Separation of concerns	Each compiler stage solves one problem.	ONNX import does not need to know final RISC-V instruction encodings.
Better debugging	You can stop after each stage and inspect the output.	Files like <code>resnet18_onnxir.onnx.mlir</code> , <code>resnet18_krnl.onnx.mlir</code> , and <code>resnet18.ll</code> show what changed.
Retargeting	The same high-level model can be lowered differently for different targets.	Gemmini uses accelerator runtime calls; another target could use a different runtime.
Runtime reuse	Common functions are compiled once and linked into many models.	<code>OMRuntimeGemmini.cpp</code> can support ResNet, BERT-style MatMul, and other models.
Hardware abstraction	The model object does not need to know exact RoCC instruction details.	The runtime and <code>gemmini.h</code> isolate hardware-specific logic.

Table 6: Why ONNX-MLIR uses several intermediate representations instead of one big conversion step.

0.12 Minimal Mental Checklist Before Reading the Build Commands

Checklist

When you see a build command later in this document, ask:

1. **What is the input file?** Example: `resnet18.onnx`, `resnet18.ll`, `OMRuntimeGemmini.cpp`.
2. **What abstraction level is being produced?** ONNX dialect, Krnl/Gemmini MLIR, LLVM IR, object file, archive, or executable.
3. **Are we compiling model code or runtime code?**
4. **Are we creating new code or linking existing code?**
5. **Which symbol proves the connection?** For convolution, follow `om_gemmini_conv_f32`.

Plain English Summary

What This Pipeline Actually Does

This document describes how we take a neural network model (specifically ResNet-18) and turn it into a program that runs on a RISC-V processor with a Gemmini hardware accelerator. In simple terms:

- **Input:** A ResNet-18 model saved as an ONNX file (about 46 MB of network structure and weights)
- **Output:** A static executable file (about 50 MB) that runs on RISC-V Linux
- **What happens:** The compiler translates the neural network operations into RISC-V CPU code and accelerator library calls
- **How it works:** The model weights become constant data in the executable; the model operations become function calls to a runtime library that talks to the accelerator

Why We Need This

- Neural networks are computationally intensive (millions of multiply-add operations)
- CPUs alone are too slow for practical inference
- Accelerators like Gemmini can perform matrix operations 10-100x faster
- We need a way to compile the model to use the accelerator automatically
- This pipeline does that: it converts high-level operations into accelerator instructions

CPU Programmer's Mental Model

If you understand CPU programming, you already know most of what's happening here. Here's a quick mapping:

Compiler/Accelerator Term	CPU Programming Analogy
ONNX Model	A graph of function calls and data dependencies
Tensor	A multi-dimensional array with metadata (shape, rank, strides)
Weights	Global <code>const</code> arrays initialized with pre-trained values
Lowering	Translating high-level code to lower-level code (like Python $\rightarrow$ C $\rightarrow$ asm)
MLIR Dialect	Different intermediate languages within the compiler toolchain
Pass	A transformation that modifies the program representation
Memref	A C struct: <code>{void* data; int64_t* shape; int rank;}</code>
OMTensor	A standardized C struct for passing tensor data between functions
Krnl.call	A function call to a runtime library
Runtime Library	Like <code>libc</code> , but for tensor operations
ABI	Calling convention and binary interface rules
Static Linking	Copying library code directly into your executable (<code>clang++ -static</code>)
RoCC	A coprocessor interface, similar to SIMD instructions
Gemmini	A systolic array accelerator (SIMD on steroids)
Scratchpad	On-chip memory managed explicitly (like L2 cache you control)

Table 7: Quick translation guide for CPU programmers.

From CPU to Accelerator

Gemmini is like... a 16x16 SIMD unit that can do 256 multiplications per cycle, with its own local memory. It's connected to the CPU through custom instructions (RoCC).

RoCC (Rocket Custom Coprocessor) is like... a coprocessor interface similar to how x86 has SIMD instructions, but more flexible. The CPU issues a special instruction, and the accelerator executes it.

Systolic array is like... a 2D pipeline where data flows in a wave, like a conveyor belt of calculations. Each processing element does multiply-add and passes results to its neighbors.

Weight-stationary dataflow is like... keeping the filter weights in the accelerator's local memory and streaming the input data through. Similar to keeping a loop invariant in a register.

Scratchpad memory is like... L2 cache that you manage explicitly. You decide what goes there and when to load/store data.

im2col is like... converting a convolution into a matrix multiply. It's like unrolling a nested loop to use optimized matrix multiply routines.

The Big Picture in One Diagram

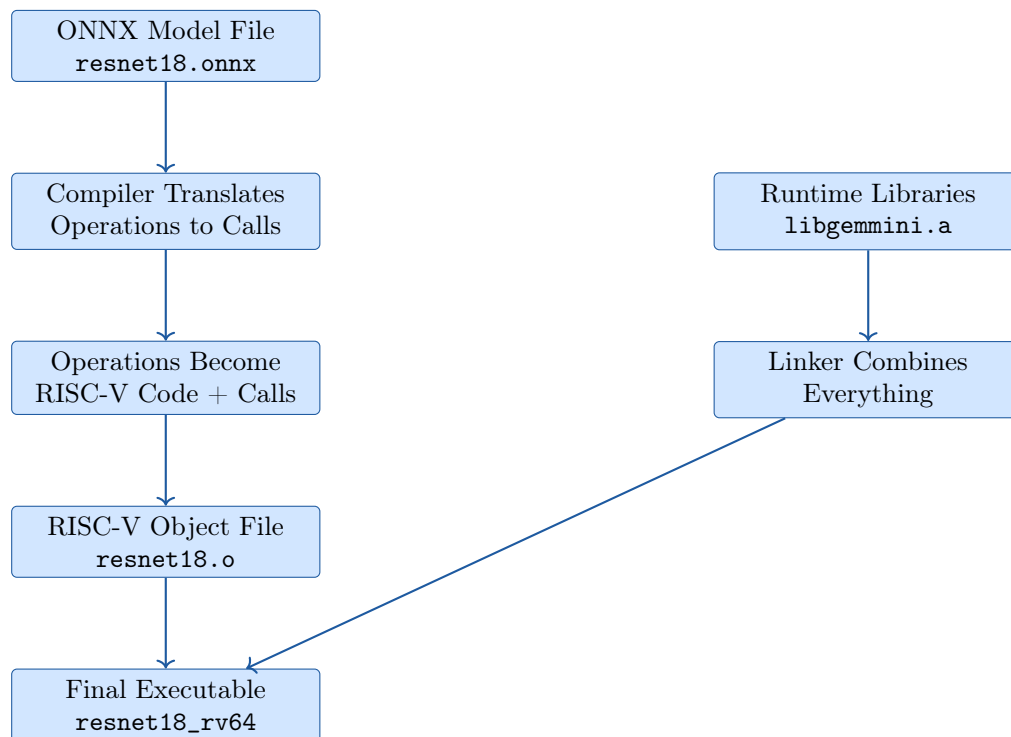


Figure 5: High-level pipeline flow. The compiler transforms the model into RISC-V code, which is then linked with runtime libraries to produce the final executable.

Key Terms

ONNX-MLIR: The compiler that reads ONNX models and generates MLIR **MLIR:** Multi-Level Intermediate Representation - a compiler framework with multiple languages **LLVM IR:** Low-level virtual machine intermediate representation (like assembly with unlimited registers) **RISC-V:** Open instruction set architecture for CPUs **Gemmini:** Hardware accelerator for matrix operations **RoCC:** Rocket Custom Coprocessor - interface for accelerators **ResNet-18:** A neural network architecture with 18 layers **Static Linking:** Combining all code into one executable (no shared libraries at runtime) **OMTensor:** Runtime data structure for passing tensors between compiler-generated code and runtime libraries

Contents

Chapter 0: Concepts You Need Before the Pipeline	1
Plain English Summary	18
Key Terms	20
1 Introduction: What We're Building	21
1.1 What Problem Does This Solve?	21

1.2	Why ONNX-MLIR plus LLVM/Clang?	22
1.3	What's in This Document	23
2	The Runtime Directory: Understanding the Bridge	23
2.1	What Each Runtime File Does	24
2.2	The Runtime Flow in Practice	25
2.3	Understanding the Gemmini ABI Headers	25
3	Pipeline Overview: The Nine Stages	26
3.1	Project Files and Folders Used by Each Stage	26
4	Stage-by-Stage Breakdown	28
4.1	Stage 1: ONNX to ONNX Dialect	28
4.1.1	What Problem Does This Solve?	29
4.1.2	What It Does	29
4.1.3	What It Looks Like	29
4.2	Stage 2: ONNX to Krnl/Gemmini MLIR	30
4.2.1	What Problem Does This Solve?	30
4.2.2	What It Does	30
4.2.3	Support Check Example	30
4.2.4	What It Looks Like	31
4.2.5	What Operations Get Accelerated?	32
4.3	Stage 3: MLIR to LLVM Dialect	32
4.3.1	What Problem Does This Solve?	32
4.3.2	What It Does	32
4.3.3	What It Looks Like	32
4.4	Stage 4: LLVM Dialect to LLVM IR	33
4.4.1	What Problem Does This Solve?	33
4.4.2	What It Does	33
4.4.3	What It Looks Like	33
4.5	Stage 5: LLVM IR to RISC-V Object File	34
4.5.1	What Problem Does This Solve?	34
4.5.2	What It Does	34
4.5.3	What It Looks Like	34
4.6	Stage 6: ONNX-MLIR C Runtime Library	35
4.6.1	What Problem Does This Solve?	35
4.6.2	What It Does	35
4.6.3	What the Runtime Provides	36
4.7	Stage 7: Gemmini Runtime Library	36
4.7.1	What Problem Does This Solve?	36
4.7.2	What It Does	36
4.7.3	What's Inside Each Runtime Function	36
4.7.4	Implementation Example (Conceptual)	37
4.8	Stage 8: Runner Program	37
4.8.1	What Problem Does This Solve?	37
4.8.2	What It Does	37
4.8.3	What It Looks Like	38
4.9	Stage 9: Static Linking	38
4.9.1	What Problem Does This Solve?	38

4.9.2	What It Does	38
4.9.3	Link Order Matters!	39
4.9.4	What the Final Executable Contains	39
5	Deep Trace: One Convolution from ONNX to Hardware	39
5.1	Stage 1: ONNX File to ONNX Dialect	40
5.2	Stage 2: ONNX Dialect to Gemmini/Krnl MLIR	40
5.3	Stage 3: Krnl to LLVM Dialect	40
5.4	Stage 4: LLVM Dialect to LLVM IR	41
5.5	Stage 5: LLVM IR to RISC-V Object	41
5.6	Stages 6-7: Runtime Libraries	41
5.7	Stage 8: Runner	42
5.8	Stage 9: Static Linking	42
5.9	The Journey Complete	43
6	Verification and Debugging	43
6.1	Checking the Final Executable	43
6.2	Common Issues and Solutions	44
6.3	Common Misunderstandings	44
7	Complete Build Script	44
8	Detailed Build Command Explanation	46
8.1	Stage 1: ONNX File to ONNXIR	47
8.2	Stage 2: ONNXIR to Gemmini/Krnl MLIR	48
8.2.1	What Changes in the IR	49
8.2.2	What <code>krnl.call</code> , <code>krnl.global</code> , and <code>memref</code> Mean	49
8.2.3	Gemmini Compatibility Decision	51
8.2.4	Observed Runtime Calls for ResNet-18	53
8.3	Stage 3: Gemmini/Krnl MLIR to LLVM Dialect MLIR	54
8.3.1	Why This Stage Matters	55
8.4	Stage 4: LLVM Dialect MLIR to Plain LLVM IR	57
8.5	Stage 5: LLVM IR to RISC-V Object File	58
8.5.1	What Is Inside <code>resnet18.o</code> ?	59
8.6	Stage 6: Build the ONNX-MLIR C Runtime Library	60
8.7	Stage 7: Build the Gemmini Runtime Library	62
8.7.1	Inside a Gemmini Runtime Function	64
8.7.2	Runtime Compile Smoke Test and Warning Pass	64
8.8	Stage 8: Compile the Runner Program	67
8.8.1	How the ResNet-18 Runner Connects to the Compiled Model	67
8.9	Stage 9: Static Linking into the Final RISC-V Executable	70
8.10	End-to-End Artifact Map	73
8.11	Most Important Conceptual Flow	74
9	Running the Final Executable on Spike with Gemmini	74
9.1	Preparing the Simulator Library Path	74
9.2	Basic Spike Run Command	75
9.3	Line-Buffered Spike Run Command	76
9.4	Observed Spike Output	77

9.5	How to Interpret the Output	78
9.6	Spike Execution Flow Diagram	79
10	Six-Level Responsibility Stack: Who Really Talks to Gemmini?	79
10.1	Colored Overview Graph	80
10.2	Detailed Level-by-Level Explanation	80
10.3	Separated Responsibility Table	83
10.4	Do Not Confuse These Layers	84
10.5	Minimal Verification Commands	84
11	Real Image Execution with Gemmini Trace	84
11.1	Command Used for the Real Image Run	85
11.2	Argument-by-Argument Explanation	86
11.3	Execution Flow Design	87
11.4	What the Trace Lines Mean	87
11.5	Shape Ladder Observed in the Trace	88
11.6	Evidence Table: What Was Proven by the Real-Image Run	90
11.7	Important Note About Comparing Scores	90
11.8	Compact Extract of the Observed Log	91
12	Line-by-Line Explanation of the Spike Trace Run	91
12.1	Who Receives Each Part of the Command?	92
12.2	Command Lines Explained	93
12.3	Output Lines Explained	93
12.4	Visual Reading of the Trace	96
13	High-Quality ResNet-18 ONNX Graph and Complete Trace Run	98
13.1	Readable Reconstruction of the ResNet-18 Execution Graph	98
13.2	Designed Vector Graph Instead of the Uploaded Screenshot	100
13.2.1	Clean Stage-Level Design	100
13.2.2	Residual Block Design: Normal Block vs Downsample Block	102
13.2.3	Detailed Designed Graph by ResNet-18 Stage	102
13.2.4	Layer Sequence Table with Clear Separators	104
13.3	Exact Spike Command Used for the Full Trace	104
13.4	Complete Observed Trace Log	105
14	Source Code Reference: GitHub Gemmini Accelerator Folder	109
14.1	Designed Directory Map	109
14.2	How the Source Folder Connects to the Pipeline	110
14.3	Folder-by-Folder Explanation	110
14.4	File-Level Meaning for the Most Important Components	111
14.5	How to Know Which Source File Was Responsible	112
14.6	Minimal Commands to Inspect the GitHub-Mapped Source Locally	113
15	Quick Reference: What Each Command Does	114
16	Summary: What We've Built	114

17 Complete Technical Audit: From ONNX Model to RISC-V Executable 116

17.1 The Main Idea in One Sentence	116
17.2 Stage-by-Stage Transformation Table	118
17.3 Tracked Example: The First ResNet-18 Convolution	119
17.4 Why the Gemmini Converter Checks Group, Dilation, Pads, and Strides	120
17.5 How to Know That Gemmini Was Actually Used	121
17.6 Conv Function Tracking Across Files	122
17.7 The Runtime Conv Implementation in Plain English	124
17.8 Syntax Dictionary: Tensor, MemRef, OMTensor, Pointer, Symbol	125
17.9 What the Headers and C++ Files Do	125
17.10 Why ONNX-MLIR Uses This Multi-Step Design	126
17.11 Minimal Debugging Checklist	127

18 Deep Runtime Folder Audit: Every File, Function, and Variable Family 128**Deep Runtime Folder Audit: Every File, Function, and Variable Family 128**

18.1 Runtime Folder Responsibility Graph	128
18.2 Every Runtime Folder and File	128
18.3 Runtime Call Chain: From Compiled Model to Hardware Helper	130
18.4 File-by-File Deep Explanation	130
18.4.1 <code>OMRuntimeGemmini.hpp</code> : public symbol contract	130
18.4.2 <code>OMRuntimeGemmini.cpp</code> : implementation and execution logic	131
18.4.3 <code>CMakeLists.txt</code> : build integration	132
18.4.4 <code>gemmini-hardware-abi/</code> : hardware ABI layer	132
18.5 Function Inventory: Every Function in <code>OMRuntimeGemmini.cpp</code>	133
18.6 Variable Dictionary by Runtime Area	138
18.6.1 Common OMTensor variables	138
18.6.2 Convolution variables	139
18.6.3 GEMM and MatMul variables	140
18.6.4 Quantization variables	140
18.7 How to Trace a Runtime Function Back to Source	141
18.8 One Function Fully Explained: <code>om_gemmini_conv_f32</code>	142
18.9 Hardware-backed vs CPU-scalar Runtime Functions	143
18.10 Why This Runtime Design Is Useful	143

1 Introduction: What We're Building

This document describes the complete process of taking a ResNet-18 neural network model (saved as an ONNX file) and turning it into a RISC-V executable that uses the Gemmini hardware accelerator. The final output is a self-contained static executable that can run on RISC-V hardware with Gemmini support.

What's in the Final Executable

The executable `resnet18_rv64` (about 50 MB) contains:

- The compiled ResNet-18 model code (which layer to run, in what order)
- All model weights (converted to constant data in `.rodata`)
- The Gemmini runtime library (code that talks to the accelerator)
- The ONNX-MLIR runtime library (tensor management)
- System libraries (math functions, memory management)
- A test harness (the `main()` function)

1.1 What Problem Does This Solve?

- **Neural networks are slow on CPUs:** Each inference involves millions of multiply-add operations. A typical CNN layer might need 100 million operations for a single image.
- **Hardware accelerators exist but need special code:** Gemmini is a systolic array that can do many multiplications in parallel, but you need to tell it what to do through custom instructions.
- **We need automation:** Manually rewriting neural networks to use accelerators is impossible for large models. We need a compiler to do it automatically.
- **This pipeline provides the automation:** It takes the high-level model description and generates executable code that uses the accelerator efficiently.

Think of it like compiling C

When you compile C code:

1. You write high-level code (`for` loops, function calls)
2. The compiler translates it to assembly instructions
3. The assembler creates object files
4. The linker combines them into an executable

This pipeline does the same thing, but for neural networks:

1. The model is described as operations (convolution, matrix multiply, activation)

2. The compiler translates these to function calls (to the accelerator runtime)
3. The runtime library implements those functions using hardware instructions
4. Everything is linked into one executable

1.2 Why ONNX-MLIR plus LLVM/Clang?

You might wonder: why can't we just use one command for everything? The answer is division of labor: ONNX-MLIR lowers the neural-network graph, while the LLVM/Clang toolchain compiles the C/C++ runtime support and performs the final link.

Tool	What It's Good At	What It Produces
ONNX-MLIR (LLVM-based)	Understanding neural network structure, tensor shapes, and operations	RISC-V object code for the model itself
LLVM/Clang and LLD	Compiling C/C++ runtime libraries, archiving objects, and static linking	Runtime libraries (OMTensor, Gemmini runtime) and final ELF executable

The tools meet in the middle through the ABI (Application Binary Interface):

- ONNX-MLIR generates code that calls functions like `om_gemmini_conv_f32()`.
- Clang/Clang++ compiles the implementation of those functions from C/C++ source.
- LLD, through the Clang++ driver, connects the calls to the implementations during final linking.

Think of it like this

Imagine you're writing a program that uses the `sin()` function:

1. You write code that calls `sin(x)`
2. Your compiler generates a call to an external symbol
3. The linker connects that call to the math library (`-lm`)
4. The math library is pre-compiled and contains the actual implementation

The same thing happens here:

1. The compiled model calls `om_gemmini_conv_f32(...)`
2. The linker connects that call to the Gemmini runtime library
3. The Gemmini runtime library contains the actual accelerator implementation

1.3 What's in This Document

This document walks through the entire pipeline step by step. For each stage, you'll learn:

- **What problem it solves** - why we can't skip this stage
- **What it does in plain English** - the core transformation
- **CPU programming analogy** - how this is like something you already know
- **Before and after examples** - what the code looks like at each stage
- **What goes wrong if we skip it** - the consequences of omission

2 The Runtime Directory: Understanding the Bridge

Before diving into the pipeline, let's look at the runtime directory. This is where the bridge between the compiled model and the accelerator lives.

What's in the Runtime Directory

The runtime directory (`src/Accelerators/Gemmini/Runtime/`) contains:

- **OMRuntimeGemmini.hpp/cpp**: The main implementation of accelerator operations
- **gemmini-hardware-abi/**: Bundled Gemmini hardware headers
- **CMakeLists.txt**: Build configuration
- **FILES_EXPLAINED.md**: Documentation

This directory is the translator between compiler-generated calls and actual accelerator instructions.

Understanding the Runtime Files

Let's map these to CPU concepts:

- `OMRuntimeGemmini.hpp` is like a header file that declares functions you'll call
- `OMRuntimeGemmini.cpp` is like the implementation of those functions
- `gemmini-hardware-abi/` is like the CPU's instruction set manual (but for the accelerator)
- The compiled `libgemmini_f32_rv64.a` is like a static library you link with

2.1 What Each Runtime File Does

File	CPU Programmer's Understanding	What It Actually Does
<code>OMRuntimeGemmini.hpp</code>	Header file declaring accelerator functions	Declares <code>om_gemmini_conv_f32</code> , <code>om_gemmini_matmul</code> , etc. with <code>extern "C"</code>
<code>OMRuntimeGemmini.cpp</code>	Implementation of those functions	Implements convolution, matrix multiply, pooling, activation functions
<code>gemmini-hardware-abi/</code>	CPU instruction set documentation	Defines the low-level interface to Gemmini (RoCC instructions, register layout)
<code>CMakeLists.txt</code>	Makefile alternative	Controls compilation, finds headers, sets flags

Think of it like hardware drivers

The Gemmini runtime is like a device driver for a GPU or a custom accelerator:

- `OMRuntimeGemmini.cpp` is the driver implementation
- It translates high-level operations (like "do a convolution") into low-level hardware commands
- `gemmini-hardware-abi/` is the hardware specification
- The driver knows how to configure the hardware, move data, and execute operations

When your compiled model calls `om_gemmini_conv_f32()`, it's like calling `write()` on a device file. The runtime driver translates it into the actual hardware commands.

2.2 The Runtime Flow in Practice

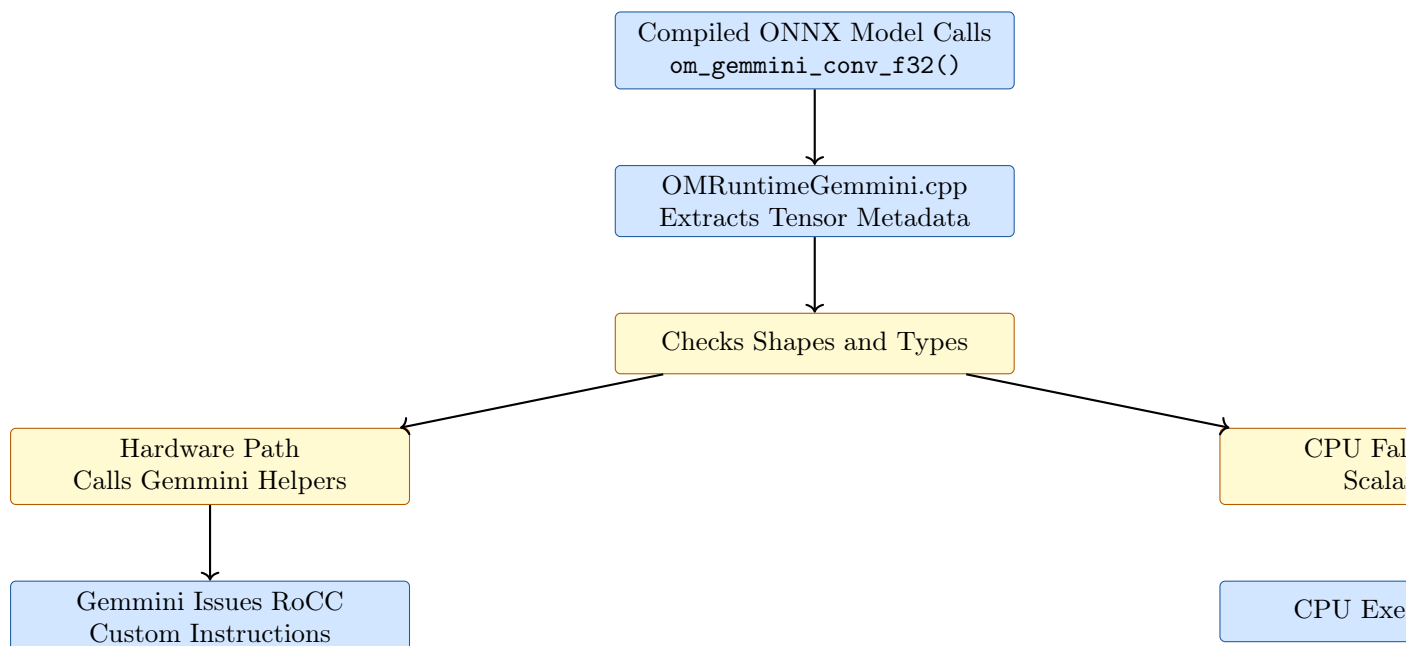


Figure 6: Runtime execution flow. The runtime decides whether to use the accelerator or fall back to CPU.

2.3 Understanding the Gemmini ABI Headers

The `gemmini-hardware-abi/` directory bundles the headers needed to talk to Gemmini. Here's what each file does:

Header File	CPU Programmer's Understanding	What It Defines
<code>gemmini.h</code>	Main API header	<code>tilled_matmul_auto</code> , <code>tilled_conv_auto</code> , configuration functions
<code>gemmini_params.h</code>	CPU feature flags	Array size (16), data types (<code>int8_t</code> , <code>int32_t</code>), scratch-pad sizes
<code>gemmini_nn.h</code>	Neural network helper functions	High-level conv/pooling helpers, parameter structs
<code>gemmini_counter.h</code>	Performance counters	Cycle counters, DMA counters, etc.
<code>xcustom.h</code>	Custom instruction macros	Macros that emit actual RISC-V custom instructions

3 Pipeline Overview: The Nine Stages

Here's the complete pipeline at a glance. The diagram below shows what each stage does and what files it produces.

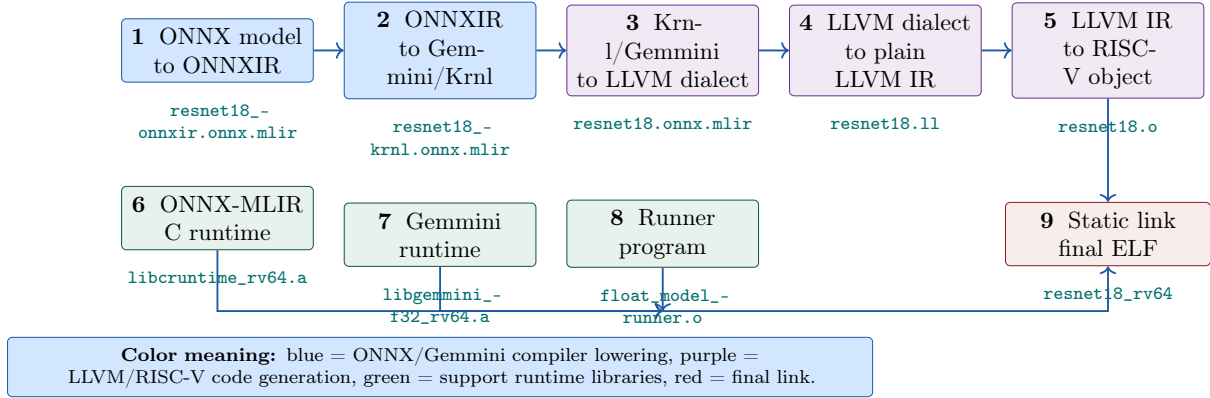


Figure 7: Complete nine-stage pipeline with all outputs. Stages 4, 5, and 6 are shown explicitly: Stage 4 creates plain LLVM IR, Stage 5 creates the RISC-V model object, and Stage 6 builds the ONNX-MLIR C runtime archive.

Stage	Input	Output	What Happens
1	ONNX model	ONNX dialect MLIR	Parse the model into compiler's internal representation
2	ONNX dialect	Krnl/Gemmini MLIR	Replace operations with runtime calls where possible
3	Krnl/Gemmini	LLVM dialect MLIR	Convert to LLVM's internal representation
4	LLVM dialect	LLVM IR text	Export as standard LLVM assembly
5	LLVM IR	RISC-V object file	Generate RISC-V machine code
6	C runtime sources	Static archive	Build ONNX-MLIR tensor management library
7	C++ runtime sources	Static archive	Build Gemmini accelerator library
8	Runner C source	Object file	Build test harness with <code>main()</code>
9	Objects + archives	Static executable	Link everything into one ELF file

3.1 Project Files and Folders Used by Each Stage

The command examples in this guide assume the working output directory is:

```
1 ~/toolchains/onnx-mlir-xcompile/onnx-mlir/gemmini/tests/15-resnet18-float
```

In that directory, the important generated files are the following:

Stage	Observed Output in Test Folder	Meaning
1	<code>resnet18_-onnxir.onnx.mlir</code> , <code>resnet18_onnxir.tmp</code>	ONNX model imported into ONNX dialect MLIR.
2	<code>resnet18_krnl.onnx.mlir</code> , <code>resnet18_krnl.tmp</code>	Gemmini/Krnl lowered MLIR. This is where <code>om_gemmini_*</code> calls can be checked.
3	<code>resnet18.onnx.mlir</code>	LLVM-dialect MLIR produced by ONNX-MLIR.
4	<code>resnet18.ll</code>	Plain LLVM IR text produced by <code>mlir-translate</code> .
5	<code>resnet18.o</code>	RISC-V relocatable model object.
6	<code>rt_obj/</code> , <code>libcruntime_rv64.a</code>	ONNX-MLIR tensor/runtime object files and archive.
7	<code>gem_rt_obj/</code> , <code>libgemmini_f32_rv64.a</code>	Gemmini runtime object and static archive.
8	<code>float_model_runner.o</code> , <code>run_test.sh</code>	Test harness object and run helper.
9	<code>resnet18_rv64</code> , <code>resnet18_rv64.disasm</code>	Final static RISC-V ELF executable and disassembly.
Debug	<code>*.csv</code> , <code>*.txt</code> , <code>*.disasm</code> , <code>DISASSEMBLY_NOTES.md</code> , <code>PIPELINE_FILE_OUTPUTS.md</code>	Extra files used to inspect constants, GEMM weights, bias values, and final generated code.

Table 8: Observed files in `gemmini/tests/15-resnet18-float`.

The Gemmini source tree under `src/Accelerators/Gemmini/` is responsible for accelerator-specific compiler behavior and runtime behavior:

Stage	Responsible Folder/File	Role
Setup / options	<code>Compiler/GemminiCompilerOptions.hpp</code>	Defines Gemmini compiler options such as accelerator selection.
Setup / options	<code>Compiler/GemminiCompilerUtils.*</code>	Helper utilities used by Gemmini compiler support.
Target registration	<code>GemminiAccelerator.cpp</code>	Registers/connects Gemmini inside the accelerator framework.
Target registration	<code>GemminiAccelerator.hpp</code>	Declares the Gemmini accelerator interface.
Target registration	<code>Support/GemminiTargetInfo.*</code>	Stores target information used by the compiler.
2	<code>Conversion/ONNXToGemmini/ONNXToGemmini.cpp</code>	Converts eligible ONNX operations to Gemmini-oriented operations or calls.
2	<code>Dialect/Gemmini/</code>	Defines higher-level Gemmini MLIR operations and types.
2	<code>Transform/Gemmini/GemminiTiling.cpp</code>	Applies tiling/preparation so operations fit Gemmini-style execution constraints.
2-3	<code>Conversion/GemminiToGemminiLow/</code>	Lowers high-level Gemmini operations into the lower-level Gemmini representation.
2-3	<code>Dialect/GemminiLow/</code>	Defines lower-level Gemmini MLIR operations and types.
2-3	<code>Transform / GemminiLow / StaticScratchpadAllocation.cpp</code>	Assigns static scratchpad regions for lowered accelerator operations when that path is used.
3-4	<code>Conversion / GemminiLowToLLVM / GemminiLowToLLVM.cpp</code>	Converts low-level Gemmini dialect operations toward LLVM-compatible operations.
7	<code>Runtime/OMRuntimeGemmini.cpp</code>	Implements <code>om_gemmini_*</code> runtime functions called by generated model code.
7	<code>Runtime/OMRuntimeGemmini.hpp</code>	Declares the Gemmini runtime functions.
7	<code>Runtime/gemmini-hardware-abi/include</code>	Provides Gemmini hardware ABI headers.
7	<code>Runtime / gemmini-hardware-abi / rocc-software</code>	Provides RoCC/custom-instruction support code.
Build	<code>CMakeLists.txt</code> files in each folder	Tell the build system how to compile and register passes, dialects, transforms, and runtime code.

Table 9: Main Gemmini source folders and their role in the pipeline.

Important Distinction

ACCELERATOR in the compiler output means that generated model code calls a Gemmini runtime symbol such as `om_gemmini_conv_f32`. It does **not** mean that a RoCC instruction is already visible in the MLIR file. The RoCC/custom instruction code is inside the compiled Gemmini runtime and hardware ABI path.

4 Stage-by-Stage Breakdown

4.1 Stage 1: ONNX to ONNX Dialect

Think of it like reading a header file

Stage 1 is like the compiler reading a `.h` file. It parses the ONNX file and builds an internal representation. The compiler doesn't generate code yet - it just understands what the model looks like.

4.1.1 What Problem Does This Solve?

The ONNX file is a binary protobuf format. We need to convert it into a format that the compiler can manipulate. This stage parses the binary and creates an MLIR representation.

4.1.2 What It Does

- Reads the ONNX protobuf file
- Parses the graph structure (which operations, in what order)
- Imports tensor initializers (weights) as constants
- Creates MLIR operations like `onnx.Conv`, `onnx.MatMul`
- Emits `resnet18_onnxir.onnx.mlir`

4.1.3 What It Looks Like

Before (ONNX file):

```
1 resnet18.onnx # binary protobuf file, about 46 MB
```

After (ONNX dialect MLIR):

```
1 %0 = "onnx.Conv"(%input, %weights, %bias) {
2   dilations = [1, 1],
3   group = 1,
4   kernel_shape = [7, 7],
5   pads = [3, 3, 3, 3],
6   strides = [2, 2]
7 } : (tensor<1x3x224x224xf32>,
8     tensor<64x3x7x7xf32>,
9     tensor<64xf32>) -> tensor<1x64x112x112xf32>
```

Listing 3: Stage 1 Output Example

Understanding the syntax

This is like a C function call:

- `%0` is the result variable (like `int result =`)
- `"onnx.Conv"` is the operation name (like a function name)
- `(%input, %weights, %bias)` are the arguments
- The attributes (`strides`, `pads`) are like function parameters
- The types are like `tensor<...>` describing the data shapes

Checkpoint: What to Verify

After Stage 1, verify:

- The file exists: `resnet18_onnxir.onnx.mlir`

- Search for "onnx.Conv" to find convolution operations
- Search for "onnx.MatMul" to find matrix multiplications
- Check that weights appear as constants

If any of these are missing, the ONNX file may be corrupted or unsupported.

4.2 Stage 2: ONNX to Krnl/Gemmini MLIR

Think of it like inlining functions

This stage is like the compiler deciding to replace some function calls with inline code or with calls to specialized libraries. For operations that the accelerator can handle, it replaces the high-level operation with a call to the accelerator runtime.

4.2.1 What Problem Does This Solve?

The ONNX dialect is high-level. We need to decide which operations will run on the accelerator and which will run on the CPU. This stage makes those decisions and rewrites the operations accordingly.

4.2.2 What It Does

- Examines each operation to see if Gemmini can accelerate it
- For supported operations (convolution, matmul, pooling, etc.):
 - Checks shapes, types, and attributes
 - Replaces the operation with a runtime function call
- For unsupported operations:
 - Lowers them to Krnl dialect (loops, memory operations)
- Weight constants become `krnl.global` symbols
- Emits `resnet18_krnl.onnx.mlir`

4.2.3 Support Check Example

For a convolution to be routed to the Gemmini runtime path, the lowering usually checks that the convolution shape and attributes match the implementation supported by [ONNX2Gem.cpp](#) and [Runtime.cpp](#). The most important checks are:

- **Rank-4 input tensor:** the input has the classic image layout `N x C x H x W`, meaning batch, input channels, height, and width.
- **Float32 data type:** the runtime symbol `om_gemmini_conv_f32` is the float32 path. The suffix `f32` is the clue.

- **Group = 1:** this is a normal dense convolution. Every output channel can use all input channels. A grouped convolution splits the channels into independent groups, which needs different indexing and sometimes multiple smaller convolutions.
- **Dilation = 1:** the kernel elements are adjacent. For example, a 3x3 kernel reads neighboring pixels. With dilation greater than 1, there are gaps between kernel samples, so the im2col/data-layout logic is different.
- **Supported stride and padding:** the runtime must be able to compute the output shape and build the correct data movement pattern.

How We Know Whether Gemmini Was Used

Do not guess from the ONNX file. Check the generated files:

```

1 # Accelerator-oriented path: runtime calls appear
2 grep -n "om_gemmini_conv_f32" resnet18_krnl.onnx.mlir
3 grep -n "om_gemmini_" resnet18.ll | head
4 $LLVM_NM -u resnet18.o | grep om_gemmini
5
6 # Final executable: symbols are resolved by libgemmini_f32_rv64.a
7 $LLVM_NM -S resnet18_rv64 | grep om_gemmini_conv_f32

```

If a layer is handled by Gemmini, you should see `om_gemmini_*` call names before linking and defined `om_gemmini_*` symbols after linking. If you only see generated loops, `krnl.iterate`, or ordinary LLVM operations for an operation, then that operation was handled by the generic compiler/runtime path instead of the Gemmini runtime path.

4.2.4 What It Looks Like

Before (ONNX dialect):

```

1 %result = "onnx.Conv"(%input, %weights, %bias) { ... }

```

After (Krnl/Gemmini MLIR):

```

1 krnl.call @om_gemmini_conv_f32(
2   %output,           # where to write the result
3   %input,            # input tensor
4   %weights,          # weight tensor
5   %stride,           # stride value (e.g., 2)
6   %padding           # padding value (e.g., 3)
7 )

```

Understanding the transformation

This is like replacing:

```

1 // High-level operation
2 result = convolution(input, weights, bias, stride=2, pad=3)
3
4 // With a function call to a specialized library
5 conv2d_optimized(output, input, weights, bias, 2, 3);

```

The compiler says: "I know how to do convolution, but the accelerator library can do

it faster, so I'll call that instead."

4.2.5 What Operations Get Accelerated?

Operation	Count in ResNet-18	What Happens
Conv2D	20	om_gemmini_conv_f32
BatchNorm	20	om_gemmini_batchnorm_f32
ReLU	17	om_gemmini_relu_f32
Add (residual)	8	om_gemmini_add_f32
MaxPool	1	om_gemmini_maxpool_f32
GlobalAvgPool	1	om_gemmini_globalavgpool_f32
GEMM (fully connected)	1	om_gemmini_gemm_f32_bias
Softmax	1	om_gemmini_softmax_f32

4.3 Stage 3: MLIR to LLVM Dialect

Think of it like generating assembly

This stage is like the compiler moving from a language with high-level constructs (loops, arrays) to something closer to assembly (registers, memory addresses, function calls).

4.3.1 What Problem Does This Solve?

MLIR has many dialects. We need to lower to a single, low-level dialect that LLVM can understand. LLVM dialect is the MLIR representation that maps directly to LLVM IR.

4.3.2 What It Does

- Converts Krnl calls to LLVM function calls
- Converts Krnl globals to LLVM global variables
- Converts memrefs to pointer-based operations
- Creates `OMTensor*` wrappers for crossing the runtime boundary
- Emits `resnet18.onnx.mlir` (LLVM dialect MLIR)

4.3.3 What It Looks Like

Before (Krnl/Gemmini MLIR):

```
1 krnl.call @om_gemmini_conv_f32(%out, %x, %w, %stride, %pad)
2 : (!krnl.OMTensorType, !krnl.OMTensorType, !krnl.OMTensorType, i64,
  i64) -> ()
```

After (LLVM dialect MLIR):

```
1 llvm.call @om_gemmini_conv_f32(
2   %out_omtensor, %x_omtensor, %w_omtensor, %stride_i64, %pad_i64
3 ) : (!llvm.ptr, !llvm.ptr, !llvm.ptr, i64, i64) -> ()
```

Understanding the ABI boundary

This is a crucial moment. Before this stage, the compiler used MLIR's internal data structures (memrefs, OMTensorType). After this stage, everything is a raw pointer or integer.

It's like the difference between:

```
1 // Internal compiler representation
2 struct Tensor { float* data; int* shape; int rank; };
3
4 // Stable ABI representation
5 void* tensor; // Opaque pointer
```

The runtime library knows how to interpret the opaque pointer. The compiler-generated code just passes it along.

4.4 Stage 4: LLVM Dialect to LLVM IR

Think of it like disassembling

This stage takes the compiler's internal representation and prints it as standard LLVM assembly. It's like taking the compiler's internal data structures and writing them as text.

4.4.1 What Problem Does This Solve?

MLIR's LLVM dialect is still MLIR syntax. Standard LLVM tools (`llc`, `opt`) expect standard LLVM IR syntax. This stage converts from MLIR syntax to LLVM IR syntax.

4.4.2 What It Does

- Reads `resnet18.onnx.mlir` (LLVM dialect)
- Translates to standard LLVM IR text
- Emits `resnet18.ll`

4.4.3 What It Looks Like

```
1 target triple = "riscv64-unknown-linux-gnu"
2
3 declare void @om_gemmini_conv_f32(ptr, ptr, ptr, i64, i64)
4
5 define ptr @run_main_graph(ptr %input) {
6 entry:
7   %0 = call ptr @main_graph_resnet18(ptr %input)
8   ret ptr %0
```

9 }

Listing 4: Stage 4 LLVM IR Output

Reading LLVM IR

This looks like assembly, but with some important differences:

- `define` and `declare` mark function definitions vs. declarations
- `ptr` is a pointer (like `void*`)
- `i64` is a 64-bit integer
- `call` is a function call
- `ret` is a return instruction

The `declare` line means: "This function exists somewhere, but not in this file." The linker will find it in the runtime libraries.

4.5 Stage 5: LLVM IR to RISC-V Object File

Think of it like assembling

This stage is exactly like running an assembler on a `.s` file. LLVM IR becomes machine code for the RISC-V CPU.

4.5.1 What Problem Does This Solve?

LLVM IR is still CPU-independent. We need actual RISC-V machine code that can run on the target hardware.

4.5.2 What It Does

- Instruction selection: chooses RISC-V instructions for each operation
- Register allocation: assigns CPU registers
- Generates RISC-V machine code
- Creates relocatable ELF object file
- Leaves external symbols unresolved (for the linker)
- Emits `resnet18.o`

4.5.3 What It Looks Like

```
1 $ $LLVM_NM -S resnet18.o | grep constant_30_resnet18
2 0000000000000000 00000000000009300 r constant_30_resnet18
3
4 $ $LLVM_NM -u resnet18.o | grep om_gemmini
```

```
5      U  om_gemmini_conv_f32
```

Listing 5: Checking the Object File

Understanding the Symbol Table

`nm` shows the symbols in the object file:

- 000... is the address (will be filled in by linker)
- 00000000000009300 is the size (37,632 bytes, the weight size)
- `r` means read-only data (goes to `.rodata`)
- `U` means undefined - this symbol must be provided by another file

The `U` is crucial. It means: "I call this function, but I don't know where it is yet. Linker, please find it for me."

4.6 Stage 6: ONNX-MLIR C Runtime Library

Think of it like compiling `libc`

This stage is like compiling the C standard library for your target architecture. The ONNX-MLIR runtime provides the tensor management functions that the compiled model expects.

4.6.1 What Problem Does This Solve?

The compiled model needs helpers for:

- Creating and destroying tensors
- Accessing tensor data, shape, and strides
- Managing tensor lists (for multiple inputs/outputs)
- Data type conversions

These helpers are provided by the ONNX-MLIR runtime library.

4.6.2 What It Does

- Compiles C source files to RISC-V object files
- Object files: `OMTensor.o`, `OMTensorList.o`, `OnnxDataType.o`, `SmallFPCConversion.o`, `OMInstrument.o`
- Archives them into `libcruntime_rv64.a`
- Uses the same ABI flags as the model (`-march=rv64gc -mabi=lp64d`)

4.6.3 What the Runtime Provides

Object File	Key Functions	What It Does
OMTensor.o	omTensorCreateUntyped, omTensorGetDataPtr, omTensorGetShape	Tensor creation, data/shape access, destruction
OMTensorList.o	omTensorListCreate, omTensorListGetSize	List of tensors for inputs/outputs
OnnxDataType.o	OM_DATA_TYPE_SIZE	Data type sizes and names
SmallFPConversion.o	om_f16_to_f32, om_bf16_ to_f32	Float16/bfloat16 conversions

4.7 Stage 7: Gemmini Runtime Library

Think of it like a device driver

This is the actual driver for the Gemmini accelerator. It knows how to configure the hardware, move data to scratchpad memory, and execute operations.

4.7.1 What Problem Does This Solve?

The compiled model calls functions like `om_gemmini_conv_f32`. These functions need to be implemented. The Gemmini runtime library implements all accelerator operations.

4.7.2 What It Does

- Compiles `OMRuntimeGemmini.cpp` to `OMRuntimeGemmini.o`
- Includes Gemmini ABI headers for hardware interface
- Archives into `libgemmini_f32_rv64.a`
- Uses C++ compiler with the same ABI flags

4.7.3 What's Inside Each Runtime Function

A typical Gemmini runtime function does:

1. **Extract metadata:** `omTensorGetShape(x)`, `omTensorGetDataPtr(x)`
2. **Validate shapes:** Check dimensions, types, compatibility
3. **Prepare data:** Convert layout (e.g., NCHW to im2col matrix)
4. **Configure accelerator:** Set up scratchpad, data movement
5. **Execute:** Call Gemmini helper functions (`tilled_matmul_auto`)
6. **Write back:** Copy results from scratchpad to output tensor

4.7.4 Implementation Example (Conceptual)

```

1 extern "C" void om_gemmini_conv_f32(
2     OMTensor *out, const OMTensor *x, const OMTensor *w,
3     int64_t stride, int64_t pad) {
4     // Extract metadata
5     float* x_data = (float*)omTensorGetDataPtr(x);
6     float* w_data = (float*)omTensorGetDataPtr(w);
7     float* out_data = (float*)omTensorGetDataPtr(out);
8
9     int64_t* x_shape = omTensorGetShape(x);
10    int64_t* w_shape = omTensorGetShape(w);
11
12    // NCHW: N, C, H, W
13    int64_t N = x_shape[0], C = x_shape[1], H = x_shape[2], W = x_shape[3];
14    int64_t OC = w_shape[0], KH = w_shape[2], KW = w_shape[3];
15
16    // Compute output dimensions
17    int64_t OH = (H + 2*pad - KH) / stride + 1;
18    int64_t OW = (W + 2*pad - KW) / stride + 1;
19
20    // im2col: convert convolution to matrix multiply
21    // ... build im2col matrix of shape (OH*OW) x (C*KH*KW)
22
23    // Use Gemmini helper for the actual compute
24    tiled_matmul_auto(im2col_matrix, weight_matrix, out_matrix,
25                      OH*OW, OC, C*KH*KW, WS);
26
27    // Write output back to OMTensor
28    // ... copy from out_matrix to out_data
29 }

```

Listing 6: Conceptual Runtime Implementation

4.8 Stage 8: Runner Program

4.8.1 What Problem Does This Solve?

We need a `main()` function to test the compiled model. The runner provides a test harness that loads inputs, calls the model, and prints outputs.

4.8.2 What It Does

- Compiles `float_model_runner.c` to `float_model_runner.o`
- Provides `main()` function
- Creates input tensors with dummy data
- Calls `run_main_graph()`
- Prints output scores

4.8.3 What It Looks Like

```

1 int main() {
2     // Create input tensor (1x3x224x224)
3     float* data = malloc(1*3*224*224*sizeof(float));
4     // Fill with dummy data...
5
6     OMTensor* input = omTensorCreateUntyped(
7         data, 4, shape, NULL, ONNX_TYPE_FLOAT, NULL);
8
9     OMTensorList* inputs = omTensorListCreate(&input, 1);
10
11     // Call the compiled model
12     OMTensorList* outputs = run_main_graph(inputs);
13
14     // Get result and print
15     OMTensor* result = omTensorListGetOmtByIndex(outputs, 0);
16     float* scores = (float*)omTensorGetDataPtr(result);
17     printf("Top score: %f\n", scores[0]);
18
19     return 0;
20 }

```

Listing 7: Runner Program Concept

4.9 Stage 9: Static Linking

Think of it like linking a C program

This is exactly like running `clang++ -static` on a C/C++ program. The linker combines all object files and libraries into a single, self-contained executable.

4.9.1 What Problem Does This Solve?

We have many object files:

- `resnet18.o` - the compiled model
- `float_model_runner.o` - the test harness
- `libgemmini_f32_rv64.a` - Gemmini runtime
- `libcruntime_rv64.a` - ONNX-MLIR runtime
- System libraries (`-lm, libc, libstdc++`)

We need to combine them into one executable that can run on RISC-V.

4.9.2 What It Does

- Reads all object files and archives
- Resolves all symbols (connects calls to definitions)
- Copies code from archives into the executable

- Assigns final addresses to code and data
- Emits the final ELF executable

4.9.3 Link Order Matters!

The linker processes inputs left to right:

```

1 g++ -static -o resnet18_rv64 \
2   float_model_runner.o \      # provides main()
3   resnet18.o \                # provides model, references runtime
4   libgemmini_f32_rv64.a \     # provides om_gemmini_* functions
5   libcruntime_rv64.a \       # provides OMTensor functions
6   -lm                        # math library (sqrtf, expf)

```

Listing 8: Correct Link Order

Why order matters

Think of it like a puzzle:

1. The puzzle (model object) has empty spaces labeled "need function X"
2. The first library (`libgemmini`) fills some spaces
3. The second library (`libcruntime`) fills the remaining spaces
4. If you give the libraries in the wrong order, some spaces remain empty

4.9.4 What the Final Executable Contains

ELF Section	Size (approx)	Contents
<code>.text</code>	1.55 MB	CPU code (model, runtime, system libraries)
<code>.rodata</code>	46.9 MB	Model weights, string constants
<code>.data.rel.ro</code>	0.05 MB	Read-only relocatable data
<code>.data</code>	0.01 MB	Initialized mutable globals
<code>.bss</code>	0.03 MB	Zero-initialized globals

5 Deep Trace: One Convolution from ONNX to Hardware

Now we'll follow one convolution operation through the entire pipeline. This shows how a high-level operation becomes a hardware instruction.

Following the Journey

We'll trace a single convolution from ResNet-18's first layer:

- Input shape: 1x3x224x224 (batch, channels, height, width)

- Weight shape: 64x3x7x7 (output channels, input channels, kernel height, width)
- Stride: 2, Padding: 3
- Output shape: 1x64x112x112

This convolution has $64 \times 3 \times 7 \times 7 = 9,408$ weights and does millions of multiply-adds.

5.1 Stage 1: ONNX File to ONNX Dialect

What happens: The frontend parser reads the ONNX model and creates an `onnx.Conv` operation.

What it looks like:

```

1 %y = "onnx.Conv"(%input, %weight, %bias) {
2   dilations = [1, 1],
3   group = 1,
4   kernel_shape = [7, 7],
5   pads = [3, 3, 3, 3],
6   strides = [2, 2]
7 } : (tensor<1x3x224x224xf32>,
8     tensor<64x3x7x7xf32>,
9     tensor<64xf32>) -> tensor<1x64x112x112xf32>

```

What changed: Nothing fundamental. The operation is now in the compiler's internal representation, but it's still a high-level convolution.

Why we need this: The compiler needs to work with the model in a structured format. The ONNX dialect is that format.

5.2 Stage 2: ONNX Dialect to Gemmini/Krnl MLIR

What happens: The compiler checks if this convolution can be accelerated. Since it meets all criteria (float32, group=1, dilation=1, equal stride, symmetric padding), it's replaced with a runtime call.

What it looks like:

```

1 krnl.call @om_gemmini_conv_f32(
2   %output, # memref<1x64x112x112xf32>
3   %input,  # memref<1x3x224x224xf32>
4   %weight, # memref<64x3x7x7xf32>
5   %stride, # 2
6   %padding # 3
7 )

```

What changed: The high-level `onnx.Conv` is gone. It's been replaced with a call to `om_gemmini_conv_f32`.

Why we need this: The compiler has made the decision: "This operation will run on the accelerator." The runtime function knows how to talk to Gemmini.

5.3 Stage 3: Krnl to LLVM Dialect

What happens: The Krnl call is converted to an LLVM call. The memrefs become OMTensor pointers.

What it looks like:

```

1 llvm.call @om_gemmini_conv_f32(
2   %out_omtensor, # !llvm.ptr
3   %x_omtensor,   # !llvm.ptr
4   %w_omtensor,   # !llvm.ptr
5   %stride_i64,   # i64
6   %pad_i64       # i64
7 )

```

What changed: The MLIR-specific types (`memref`, `i64`) are now LLVM types (`ptr`, `i64`). This is the ABI boundary.

Why we need this: The runtime library is written in C/C++, not MLIR. It expects standard C types, not compiler-internal types.

5.4 Stage 4: LLVM Dialect to LLVM IR

What happens: The LLVM dialect is exported as standard LLVM IR.

What it looks like:

```

1 declare void @om_gemmini_conv_f32(ptr, ptr, ptr, i64, i64)
2
3 call void @om_gemmini_conv_f32(
4   ptr %out_omtensor,
5   ptr %x_omtensor,
6   ptr %w_omtensor,
7   i64 2,
8   i64 3
9 )

```

What changed: The syntax is now standard LLVM IR. The `declare` line says "this function exists, but not in this file."

Why we need this: LLVM tools expect standard LLVM IR syntax. This is the last compiler-specific stage before CPU code generation.

5.5 Stage 5: LLVM IR to RISC-V Object

What happens: LLVM generates RISC-V code. The call to `om_gemmini_conv_f32` becomes a relocation.

What it looks like:

```

1 $ $LLVM_NM -u resnet18.o | grep om_gemmini_conv_f32
2           U om_gemmini_conv_f32

```

What changed: The function call is now an unresolved symbol (U) in the object file.

Why we need this: The object file can't know the final address of the runtime function. The linker will fill it in later.

5.6 Stages 6-7: Runtime Libraries

What happens: The runtime libraries are built as separate static archives.

What it looks like:

```

1 $ $LLVM_AR t libgemmini_f32_rv64.a
2 OMRuntimeGemmini.o
3
4 $ $LLVM_OBJDUMP -t OMRuntimeGemmini.o | grep om_gemmini_conv_f32
5 0000000000000000 g      F .text 00000000 om_gemmini_conv_f32

```

What changed: The runtime function now has an address (tentative) and is marked as global (g).

Why we need this: The linker needs to find the symbol. `OMRuntimeGemmini.o` provides the definition.

5.7 Stage 8: Runner

What happens: The test harness provides `main()` which calls the model.

What it looks like:

```

1 int main() {
2     OMTensorList* inputs = create_input_tensor();
3     OMTensorList* outputs = run_main_graph(inputs);
4     // ...
5 }

```

What changed: We now have a complete program with a starting point.

Why we need this: The final executable needs an entry point. `main()` is that entry point.

5.8 Stage 9: Static Linking

What happens: The linker resolves all symbols. The call to `om_gemmini_conv_f32` is now connected to its implementation.

What it looks like:

```

1 $ $LLVM_NM -S resnet18_rv64 | grep om_gemmini_conv_f32
2 000000000000a1a0 000000000000b78 T om_gemmini_conv_f32
3
4 $ $LLVM_OBJDUMP -d resnet18_rv64 | grep -A2 om_gemmini_conv_f32
5 105f8: jal 23a40 <om_gemmini_conv_f32>

```

What changed:

- The symbol is now defined (T means text section)
- It has a final address: `0x000000000000a1a0`
- The call site has the final address: `jal 23a40`

Why we need this: The executable is now complete. All function calls have addresses. The program can run.

5.9 The Journey Complete

Stage	Form of the Convolution	Who's Responsible	Key Characteristic
1	ONNX Conv operation	Frontend parser	High-level, still in compiler
2	<code>krnl.call "om_gemmini_conv_f32"</code>	Gemmini lowering	Replaced with function call
3	<code>llvm.call</code> with OMTensor pointers	Krnl-to-LLVM	ABI boundary crossed
4	LLVM IR <code>call</code>	LLVM translation	Standard IR, not MLIR
5	Relocation in object file	LLVM <code>llc</code>	U symbol, unresolved
6-7	<code>OMRuntimeGemmini.o</code>	LLVM/Clang compilation	Definition exists in archive
8	<code>main()</code> calls <code>run_main_graph</code>	Runner compilation	Entry point provided
9	Final resolved call	Linker	<code>jal</code> instruction with address

6 Verification and Debugging

6.1 Checking the Final Executable

```

1 # Check file type
2 file resnet18_rv64
3 # Output: ELF 64-bit LSB executable, UCB RISC-V, version 1 (SYSV),
4   statically linked
5
6 # Check sections
7 $LLVM_READELF -SW resnet18_rv64 | grep -E '\.text|\.rodata|\.data|\.bss'
8
9 # Check symbols
10 nm -S --size-sort resnet18_rv64 | grep constant_
11
12 # Find runtime calls
13 $LLVM_OBJDUMP -d resnet18_rv64 | grep om_gemmini_

```

Listing 9: Basic Verification Commands

6.2 Common Issues and Solutions

Error	Cause and Solution
undefined reference to 'om_gemmini_*'	libgemmini_f32_rv64.a is not linked or is in the wrong order. Add it after resnet18.o.
undefined reference to 'omTensor*'	libcruntime_rv64.a is missing. Link it after libgemmini_f32_rv64.a.
unknown command line argument '-accel'	Use -maccel=Gemmini (note the 'm').
error: cannot find 'onnx-mlir'	Use the Gemmini-enabled build, not the base ONNX-MLIR.
objdump: unrecognized architecture	Use \$LLVM_OBJDUMP, not host objdump.
.rodata section too large	Add -mmodel=medany to LLVM/Clang flags.

6.3 Common Misunderstandings

Frequently Asked Questions

Q: Why can't we just compile to RISC-V directly?

A: The ONNX model isn't code yet; it's a graph description. We need to translate the graph to code.

Q: Where are the weights?

A: They become constant data in the .rodata section of the executable. Like global arrays.

Q: Why use two compilers?

A: ONNX-MLIR understands the model structure; LLVM/Clang compiles C/C++ runtime libraries. They meet via the OMTensor ABI.

Q: Why static linking?

A: For embedded/simulator targets, it's easier to have one self-contained executable without shared library dependencies.

Q: Why is the executable so big (50 MB)?

A: Most of it is model weights (46.9 MB) stored as constant data. The code itself is only about 1.5 MB.

Q: How does the accelerator know what to do?

A: The runtime library issues custom RISC-V instructions (RoCC) that the accelerator recognizes. The CPU executes these instructions, and the accelerator handles them.

7 Complete Build Script

Here's the complete script that implements the entire pipeline:

```
1 #!/bin/bash
2 set -e
3
```



```

4 ROOT_DIR="/home/lahcen/toolchains/onnx-mlir-xcompile/onnx-mlir"
5 BIN_DIR="$ROOT_DIR/gemmini_toolchain_build/Release/bin"
6 LLVM_BIN="$ROOT_DIR/./llvm-install/bin"
7 OUT_DIR="$ROOT_DIR/gemmini/tests/15-resnet18-float"
8 MODEL_FILE="$ROOT_DIR/gemmini/examples/15-resnet18-float/resnet18.onnx"
9 B=resnet18
10 RISC_V_CC="$LLVM_BIN/clang"
11 RISC_V_CXX="$LLVM_BIN/clang++"
12 LLVM_AR="$LLVM_BIN/llvm-ar"
13 LLVM_NM="$LLVM_BIN/llvm-nm"
14 LLVM_OBJDUMP="$LLVM_BIN/llvm-objdump"
15 LLVM_READelf="$LLVM_BIN/llvm-readelf"
16 RISC_V_SYSROOT="$RISC_V_SYSROOT:-/usr/riscv64-linux-gnu}"
17 CLANG_TARGET="--target=riscv64-unknown-linux-gnu --sysroot=
    $RISC_V_SYSROOT"
18 FLAGS_ONNX="--macc=gemmini --mtriple=riscv64-unknown-linux-gnu"
19 FLAGS_RV="-O2 -march=rv64gc -mabi=lp64d"
20
21 echo "=== Stage 1: ONNX -> ONNX Dialect ==="
22 $BIN_DIR/onnx-mlir $FLAGS_ONNX --EmitONNXIR \
23   -o $OUT_DIR/${B}_onnxir $MODEL_FILE
24
25 echo "=== Stage 2: ONNX -> Krnl/Gemmini ==="
26 $BIN_DIR/onnx-mlir $FLAGS_ONNX --EmitMLIR \
27   -o $OUT_DIR/${B}_krnl $MODEL_FILE
28
29 echo "=== Stage 3: Krnl -> LLVM Dialect ==="
30 $BIN_DIR/onnx-mlir $FLAGS_ONNX --EmitLLVMIR \
31   -o $OUT_DIR/${B} $MODEL_FILE
32
33 echo "=== Stage 4: LLVM Dialect -> LLVM IR ==="
34 $LLVM_BIN/mlir-translate --mlir-to-llvmir \
35   $OUT_DIR/${B}.onnx.mlir -o $OUT_DIR/${B}.ll
36
37 echo "=== Stage 5: LLVM IR -> RISC-V Object ==="
38 $LLVM_BIN/llc $FLAGS_RV --mtriple=riscv64-unknown-linux-gnu \
39   --mcpu=generic-rv64 --mattr=+m,+a,+c,+f,+d \
40   -filetype=obj -relocation-model=static \
41   -o $OUT_DIR/${B}.o $OUT_DIR/${B}.ll
42
43 echo "=== Stage 6: Build ONNX-MLIR C Runtime ==="
44 mkdir -p $OUT_DIR/rt_obj
45 $RISC_V_CC $CLANG_TARGET -c $FLAGS_RV -I $ROOT_DIR/include \
46   $ROOT_DIR/src/Runtime/OnnxDataType.c -o $OUT_DIR/rt_obj/OnnxDataType.
47   o
48 $RISC_V_CC $CLANG_TARGET -c $FLAGS_RV -I $ROOT_DIR/include \
49   $ROOT_DIR/src/Support/SmallFPConversion.c -o $OUT_DIR/rt_obj/
50   SmallFPConversion.o
51 $RISC_V_CC $CLANG_TARGET -c $FLAGS_RV -I $ROOT_DIR/include \
52   $ROOT_DIR/src/Runtime/OMTensor.c -o $OUT_DIR/rt_obj/OMTensor.o
53 $RISC_V_CC $CLANG_TARGET -c $FLAGS_RV -I $ROOT_DIR/include \
54   $ROOT_DIR/src/Runtime/OMTensorList.c -o $OUT_DIR/rt_obj/OMTensorList.
55   o
56 $RISC_V_CC $CLANG_TARGET -c $FLAGS_RV -I $ROOT_DIR/include \
57   $ROOT_DIR/src/Runtime/OMInstrument.c -o $OUT_DIR/rt_obj/OMInstrument.
58   o
59 $LLVM_AR rcs $OUT_DIR/libcruntime_rv64.a \

```

```

57 $OUT_DIR/rt_obj/OnnxDataType.o \
58 $OUT_DIR/rt_obj/SmallFPConversion.o \
59 $OUT_DIR/rt_obj/OMTensor.o \
60 $OUT_DIR/rt_obj/OMTensorList.o \
61 $OUT_DIR/rt_obj/OMInstrument.o
62
63 echo "=== Stage 7: Build Gemmini Runtime ==="
64 mkdir -p $OUT_DIR/gem_rt_obj
65 GEMMINI_HW_ABI_DIR=$ROOT_DIR/src/Accelerators/Gemmini/Runtime/gemmini-
    hardware-abi
66 GEMMINI_HW_ABI_INCLUDE=$GEMMINI_HW_ABI_DIR/include
67 $RISCV_CXX $CLANG_TARGET -c $FLAGS_RV -I $ROOT_DIR/include -I $ROOT_DIR
    \
68 -I $GEMMINI_HW_ABI_DIR -I $GEMMINI_HW_ABI_INCLUDE \
69 $ROOT_DIR/src/Accelerators/Gemmini/Runtime/OMRuntimeGemmini.cpp \
70 -o $OUT_DIR/gem_rt_obj/OMRuntimeGemmini.o
71 $LLVM_AR rcs $OUT_DIR/libgemmini_f32_rv64.a \
72 $OUT_DIR/gem_rt_obj/OMRuntimeGemmini.o
73
74 echo "=== Stage 8: Compile Runner ==="
75 $RISCV_CC $CLANG_TARGET -c $FLAGS_RV -I $ROOT_DIR/include \
76 $ROOT_DIR/gemmini/examples/tools/float_model_runner.c \
77 -o $OUT_DIR/float_model_runner.o
78
79 echo "=== Stage 9: Static Link ==="
80 $RISCV_CXX $CLANG_TARGET $FLAGS_RV -fuse-ld=lld -static \
81 -o $OUT_DIR/${B}_rv64 \
82 $OUT_DIR/float_model_runner.o \
83 $OUT_DIR/${B}.o \
84 $OUT_DIR/libgemmini_f32_rv64.a \
85 $OUT_DIR/libcruntime_rv64.a \
86 -lm
87
88 echo "=== Verification ==="
89 file $OUT_DIR/${B}_rv64
90 $LLVM_READELF -SW $OUT_DIR/${B}_rv64 | grep -E '\.text|\.rodata|\.data
    |\.bss'
91 $LLVM_NM -S $OUT_DIR/${B}_rv64 | grep -c 'constant.*_resnet18'
92
93 echo "Done: $OUT_DIR/${B}_rv64"

```

Listing 10: Full Pipeline Script

8 Detailed Build Command Explanation

The full script above is useful when you want to run the pipeline, but it does not explain enough by itself. This section expands every command in the script. The goal is to make each stage readable in three ways:

- **What command is executed** – the exact shell command used in the build script.
- **What each argument means** – why the flag is present and what output it affects.
- **What you should inspect** – the file, symbol, or IR pattern that proves the stage worked.

Important Reading Rule

Most `onnx-mlir` emission commands start again from the original ONNX model file. For example, `-EmitONNXIR`, `-EmitMLIR`, and `-EmitLLVMIR` are separate compiler runs. Stage 2 does not literally read the Stage 1 output file as its input; it re-reads `resnet18.onnx` and stops at a deeper point in the compiler pipeline.

8.1 Stage 1: ONNX File to ONNXIR

```
1 $BIN_DIR/onnx-mlir \
2   --maccel=Gemmini \
3   --mtriple=riscv64-unknown-linux-gnu \
4   --EmitONNXIR \
5   -o $OUT_DIR/${MODEL_BASENAME}_onnxir \
6   $MODEL_FILE
```

Listing 11: Generate ONNX Dialect MLIR

Stage 1 is the **frontend import stage**. The compiler reads the binary ONNX protobuf file and converts it into MLIR using the ONNX dialect. At this point, the model still looks like a neural-network graph: convolution is still `onnx.Conv`, ReLU is still `onnx.Relu`, matrix multiplication is still `onnx.Gemm` or `onnx.MatMul`, and the trained weights are represented as constants.

Argument	Meaning	Output Effect
<code>onnx-mlir</code>	Runs the ONNX-MLIR compiler driver. This binary understands the ONNX model format and can emit different intermediate representations depending on the flag.	Starts a compiler run.
<code>-maccel=Gemmini</code>	Enables the Gemmini accelerator target. Even though Stage 1 is still ONNXIR, this keeps the command consistent with later stages.	No major visible change yet.
<code>-mtriple=...</code>	Tells the compiler that the final target platform is 64-bit RISC-V Linux.	Shapes and ABI-sensitive lowering remain target-aware.
<code>-EmitONNXIR</code>	Stops after importing ONNX into the ONNX dialect. No Krnl, LLVM, or RISC-V code is generated yet.	Produces ONNX dialect MLIR.
<code>-o output_prefix</code>	Output prefix used by ONNX-MLIR.	Writes <code>resnet18_onnxir.onnx.mlir</code> .
<code>\$MODEL_FILE</code>	The original model file, for example <code>resnet18.onnx</code> .	Source model is parsed.

Table 10: Stage 1 command arguments.

What You Should See in the Output

Search in `resnet18_onnxir.onnx.mlir` for these patterns:

- `onnx.Conv`: convolution layers were imported.
- `onnx.BatchNormalization`: batch normalization nodes were imported.
- `onnx.Relu`: activation nodes were imported.
- `dense_resource` or `constants`: trained weights were imported into the compiler representation.

If the output file is very small or these operators are missing, the ONNX model was not imported correctly.

8.2 Stage 2: ONNXIR to Gemmini/Krnl MLIR

```

1 $BIN_DIR/onnx-mlir \
2   --maccel=Gemmini \
3   --mtriple=riscv64-unknown-linux-gnu \
4   --EmitMLIR \
5   -o $OUT_DIR/${MODEL_BASENAME}_krnl \
6   $MODEL_FILE

```

Listing 12: Generate Gemmini/Krnl MLIR

Stage 2 is the first stage where the document becomes really important for Gemmini. The compiler lowers the model beyond pure ONNX dialect. Eligible neural-network operators are rewritten toward Gemmini runtime calls, while unsupported or unsuitable operators are lowered to the generic Krnl dialect.

In simple words, this stage answers the question: **which parts of the model will use the accelerator, and which parts will stay as normal generated code?**

Argument	Meaning	Output Effect
<code>onnx-mlir</code>	Runs the same compiler driver, but asks it to lower beyond the ONNX dialect.	Starts a deeper compiler run.
<code>-maccel=Gemmini</code>	Activates the accelerator lowering path. Eligible ONNX operators such as convolution, batch normalization, ReLU, pooling, residual add, GEMM, and softmax are routed toward Gemmini runtime calls.	Gemmini-related calls can appear.
<code>-mtriple=...</code>	Keeps target assumptions aligned with the final RISC-V Linux executable.	Uses the correct target model.
<code>-EmitMLIR</code>	Stops after the main MLIR lowering phase. The output is no longer pure ONNX dialect: it contains Krnl operations, memrefs, globals, and runtime calls.	Produces lowered MLIR.
<code>-o output_prefix</code>	Output prefix.	Writes <code>resnet18_krnl.onnx.mlir</code> .
<code>\$MODEL_FILE</code>	The original ONNX model is read again. This command does not consume <code>resnet18_onnxir.onnx.mlir</code> directly.	Re-parses the model and lowers it.

Table 11: Stage 2 command arguments.

8.2.1 What Changes in the IR

Before Stage 2, a convolution is still represented as a high-level ONNX operation:

```

1 %y = "onnx.Conv"(%x, %w, %b) {
2   kernel_shape = [7, 7],
3   pads = [3, 3, 3, 3],
4   strides = [2, 2]
5 } : (...) -> tensor<1x64x112x112xf32>

```

Listing 13: Before Stage 2: High-Level ONNX Operation

After Stage 2, an eligible convolution is represented as a call to the Gemmini runtime:

```

1 krnl.call @om_gemmini_conv_f32(
2   %output_tensor,
3   %input_tensor,
4   %weight_tensor,
5   %stride,
6   %padding
7 )

```

Listing 14: After Stage 2: Runtime Call Form

This does not mean the RoCC instruction is already visible here. Stage 2 only creates a call to a C/C++ runtime function. The actual RoCC-level hardware instructions are inside the Gemmini runtime implementation, which is compiled later in Stage 7.

8.2.2 What `krnl.call`, `krnl.global`, and `memref` Mean

These names look abstract, but they map to simple programming concepts:

MLIR Term	Simple Meaning	CPU/C Analogy
<code>krnl.call</code>	A compiler-level call to a runtime function.	Like writing <code>conv2d_optimized(out, x, w, stride, pad);</code> .
<code>krnl.global</code>	A named global constant stored in the compiled program.	Like <code>static const float weights[] = {...};</code> .
<code>memref<...></code>	A memory reference with shape information.	Like a pointer plus sizes and strides.
<code>alloc</code> / temporary buffer	Memory created for intermediate tensors.	Like <code>malloc()</code> or a compiler-generated temporary array.

Table 12: Plain meaning of common Krnl/MLIR terms.

```

1 // In MLIR you may see a krnl.global for a weight tensor.
2 // In C, think of it like this:
3 static const float conv1_weight[64][3][7][7] = {
4     /* trained ResNet-18 weights */
5 };

```

Listing 15: CPU Analogy for `krnl.global`

```

1 // In MLIR you may see krnl.call @om_gemmini_conv_f32(...).
2 // In C, think of it like this:
3 om_gemmini_conv_f32(output_tensor, input_tensor, weight_tensor, 2, 3);

```

Listing 16: CPU Analogy for `krnl.call`

Why `krnl.global` Is Important

ResNet-18 has many trained weights. Those weights are not recomputed at runtime. They are compiled into the object file as read-only constants, then linked into the final executable. This is why `resnet18_rv64` is large: most of its size is model weight data in read-only sections such as `.rodata`.

Order	Representative Pass Name	What Changes in the IR
1	ONNX shape inference and canonicalization	Tensor ranks and dimensions are propagated where possible. This makes later accelerator checks possible.
2	ONNXToGemmini	Eligible ONNX operations are mapped to Gemmini runtime-call forms.
3	Gemmini legality checks	The compiler checks operation type, tensor rank, data type, stride, padding, dilation, and grouping constraints.
4	Gemmini tiling or preparation	Large operations are prepared so the runtime can execute them within Gemmini scratchpad and accumulator constraints.
5	ONNXToKrnl	Non-accelerated ONNX operations are lowered to Krnl loops, memrefs, constants, and standard runtime calls.
6	Constant/global lowering	Weight tensors become global constants that can later be emitted into the object file and final executable.

Table 13: Representative transformations in the Gemmini/Krnl lowering stage. Exact internal pass names may differ by ONNX-MLIR/Gemmini branch, but this is the practical lowering logic to understand.

8.2.3 Gemmini Compatibility Decision

Not every ONNX operator is automatically accelerated. The lowering must be legal for the runtime and for the hardware. For example, a convolution is a good Gemmini candidate when it has a layout and parameter set that the runtime knows how to translate into matrix multiplication.

Condition	Plain Explanation	How to Recognize the Path
Rank-4 input tensor	Standard image convolution uses $N \times C \times H \times W$. Example: $1 \times 3 \times 224 \times 224$.	ACCELERATOR Look for <code>om_gemmini_conv_f32</code> .
Float32 element type	The suffix <code>f32</code> means the generated call expects 32-bit floating-point data.	ACCELERATOR The call name contains <code>_f32</code> .
<code>group = 1</code>	Normal convolution. All input channels participate in each output-channel filter. In grouped convolution, channels are split into groups, so the dense GEMM mapping is different.	ACCELERATOR for dense conv; GENERIC/KRNL or another specialized path if grouped conv is not implemented.
<code>dilation = 1</code>	Kernel samples are adjacent. A dilated kernel inserts spaces between samples, changing the <code>im2col</code> indexing.	ACCELERATOR when supported by runtime; GENERIC/KRNL if only generic indexing is generated.
Supported stride and padding	The runtime must compute output shape and data movement. Example: stride 2 and pad 3 in the first ResNet-18 convolution.	Check the generated call arguments and output shape.
Static or inferable shapes	The compiler needs enough shape information to create buffers, calls, and metadata.	Search for generated tensor shape constants and runtime call construction.

Table 14: Typical legality checks before mapping an ONNX operation to a Gemmini runtime call.

Accelerator vs. Generic Path

The reliable evidence is in the artifacts, not in the intuition. Use this rule:

- **ACCELERATOR** means you see `om_gemmini_*` calls in `resnet18_krnl.onnx.mlir`, declarations in `resnet18.ll`, undefined symbols in `resnet18.o`, and resolved definitions in `resnet18_rv64`.
- **GENERIC/KRNL** means the operation is represented as compiler-generated loops, standard runtime calls, or ordinary LLVM operations instead of a Gemmini runtime call.

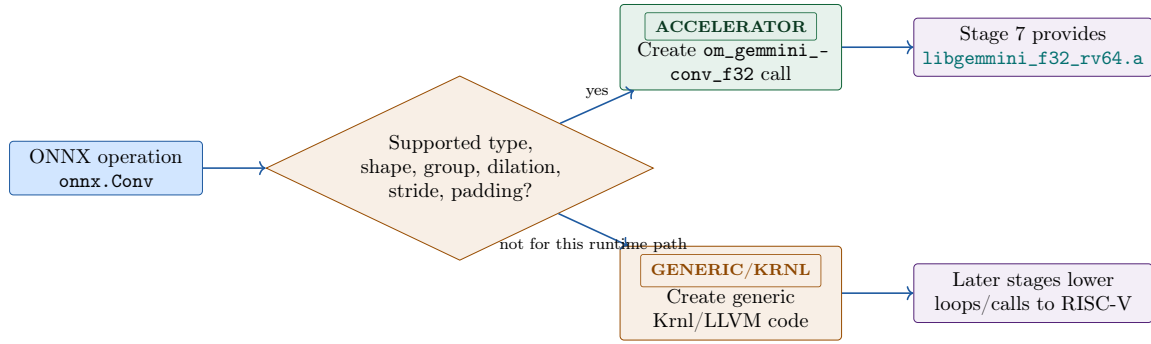


Figure 8: Stage 2 decision flow. The accelerator path is visible as `om_gemmini_*` calls; the generic path is visible as Krnl/LLVM operations instead of Gemmini runtime calls.

8.2.4 Observed Runtime Calls for ResNet-18

For the ResNet-18 float pipeline described here, the important observation is that many high-level model layers are converted to calls whose names begin with `om_gemmini_`. These calls are the bridge between compiler-generated model code and the runtime library compiled in Stage 7.

Function	Calls	Purpose
<code>om_gemmini_conv_f32</code>	20	2-D convolution using Gemmini-oriented convolution preparation, usually converted internally to an im2col plus GEMM-style execution path.
<code>om_gemmini_batchnorm_f32</code>	20	Per-channel normalization using scale, bias, mean, and variance.
<code>om_gemmini_relu_f32</code>	17	Element-wise ReLU activation. Negative values are clipped to zero.
<code>om_gemmini_add_f32</code>	8	Residual addition used by ResNet skip connections.
<code>om_gemmini_maxpool_f32</code>	1	Max pooling after the first convolution block.
<code>om_gemmini_globalavgpool_f32</code>	1	Spatial average pooling before the classifier.
<code>om_gemmini_gemm_f32_bias</code>	1	Final fully connected layer with bias.
<code>om_gemmini_softmax_f32</code>	1	Converts classifier scores into class-probability-like values.
Total	69	Runtime calls representing the accelerator-oriented execution path for ResNet-18.

Table 15: Observed Gemmini runtime calls for the ResNet-18 pipeline.

Do Not Confuse Runtime Calls with Hardware Instructions

A call such as `om_gemmini_conv_f32` is not itself a RISC-V custom instruction. It is a normal function call generated by the compiler. Inside that function, the Gemmini runtime can call lower-level Gemmini helper functions and RoCC instruction macros.

Therefore, the path is:

ONNX Conv → Gemmini runtime call → Gemmini helper → RoCC/custom instructions.

How to Verify Stage 2

Use these commands after Stage 2:

```
1 grep -n "om_gemmini_" $OUT_DIR/${B}_krnl.onnx.mlir | head
2 grep -c "om_gemmini_conv_f32" $OUT_DIR/${B}_krnl.onnx.mlir
3 grep -c "krnl.global" $OUT_DIR/${B}_krnl.onnx.mlir
```

Expected result: you should see Gemmini runtime call names and many global constants representing model weights.

8.3 Stage 3: Gemmini/Krnl MLIR to LLVM Dialect MLIR

```
1 $BIN_DIR/onnx-mlir \
2   --maccel=Gemmini \
3   --mtriple=riscv64-unknown-linux-gnu \
4   --EmitLLVMIR \
5   -o $OUT_DIR/${MODEL_BASENAME} \
6   $MODEL_FILE
```

Listing 17: Generate LLVM Dialect MLIR

Stage 3 lowers the compiler representation close to LLVM. The model is still in an MLIR file, but the operations are now LLVM-dialect operations. This stage is where tensors, memrefs, globals, and runtime calls become closer to normal pointer-based ABI objects.

Argument	Meaning	Output Effect
<code>onnx-mlir</code>	Runs ONNX-MLIR again from the original ONNX model.	Starts a compiler run.
<code>-maccel=Gemmini</code>	Keeps the Gemmini lowering path enabled so the generated LLVM-dialect code still calls the Gemmini runtime symbols.	Runtime calls remain visible.
<code>-mtriple=...</code>	Selects the RISC-V Linux target triple.	Target triple appears in later LLVM IR.
<code>-EmitLLVMIR</code>	In ONNX-MLIR naming, this emits an MLIR file in the LLVM dialect. It is not yet plain <code>.ll</code> text.	Writes <code>resnet18.onnx.mlir</code> .
<code>-o output_prefix</code>	Output prefix without the <code>_krnl</code> suffix.	Produces <code>resnet18.onnx.mlir</code> .
<code>\$MODEL_FILE</code>	Original ONNX input.	The model is parsed and lowered to the requested level.

Table 16: Stage 3 command arguments.

8.3.1 Why This Stage Matters

Krnl and memref abstractions are useful inside the compiler, but C/C++ runtime libraries do not accept compiler-internal MLIR objects. Runtime functions expect ordinary ABI-compatible values such as pointers and integers. Stage 3 therefore prepares the program for the ABI boundary.

Simple Example: Memref to Pointer-Based Runtime Data

A memref is not just a raw pointer. It also carries shape/stride information inside the compiler.

```
1 memref<1x3x224x224xf32>
2 // means: float data with shape [1, 3, 224, 224]
```

Listing 18: Compiler-Side View

At the C runtime boundary, the runtime prefers a normal pointer to a tensor object:

```
1 typedef struct OMTensor OMTensor;
2
3 // Conceptually contains:
4 // - void* data pointer
5 // - int64_t* shape
6 // - int64_t rank
7 // - data type, strides, ownership info
8 OMTensor *input_tensor;
```

Listing 19: C Runtime View

So the phrase "Creates `OMTensor*` wrappers" simply means: the compiler prepares a normal C-compatible tensor object so the runtime function can read the data pointer and dimensions.

Simple Example: Runtime Boundary

Before the boundary, the compiler thinks in tensor/memref form:

```
1 %input : memref<1x3x224x224xf32>
```

After lowering, the runtime call receives ordinary ABI values:

```
1 om_gemmini_conv_f32(out_omtensor, input_omtensor, weight_omtensor,
2 //                2, 3);
   int              pointer      pointer      pointer      int
```

This is similar to converting a rich internal object into a C function call with `void*` pointers and `int64_t` arguments.

Before Stage 3	After Stage 3	Meaning
<code>krnl.call</code>	<code>llvm.call</code>	Compiler-specific runtime call becomes an LLVM-dialect function call.
<code>memref<...xf32></code>	<code>!llvm.ptr</code> plus metadata construction	Tensor memory is represented using pointer-compatible structures.
<code>krnl.global</code>	LLVM-dialect global	Weights are closer to final global constants.
index values	i64 values	MLIR index types become fixed-width ABI integer values.

Table 17: Main representation changes in Stage 3.

```
1 llvm.call @om_gemmini_conv_f32(
2   %out_omtensor,
3   %input_omtensor,
4   %weight_omtensor,
5   %stride_i64,
6   %pad_i64
7 ) : (!llvm.ptr, !llvm.ptr, !llvm.ptr, i64, i64) -> ()
```

Listing 20: Runtime Call After Lowering Toward LLVM Dialect

CPU Programmer Analogy

Stage 2 is like deciding to call an optimized library function. Stage 3 is like converting your internal C++ objects into a plain C ABI: pointers, integer sizes, and external

function symbols. The runtime does not need to know MLIR; it only needs to receive valid `OMTensor*` objects and scalar parameters.

How to Verify Stage 3

```
1 grep -n "llvm.call.*om_gemmini" $OUT_DIR/${B}.onnx.mlir | head
2 grep -n "llvm.mlir.global" $OUT_DIR/${B}.onnx.mlir | head
3 grep -n "run_main_graph" $OUT_DIR/${B}.onnx.mlir | head
```

You should see LLVM-dialect calls to Gemmini runtime functions and globals for model constants.

8.4 Stage 4: LLVM Dialect MLIR to Plain LLVM IR

```
1 $LLVM_BIN/mlir-translate \
2   --mlir-to-llvmir \
3   $OUT_DIR/${MODEL_BASENAME}.onnx.mlir \
4   -o $OUT_DIR/${MODEL_BASENAME}.ll
```

Listing 21: Translate MLIR LLVM Dialect to LLVM IR Text

Stage 4 is a translation step, not a neural-network optimization step. It converts MLIR syntax into standard LLVM IR syntax so that standard LLVM backend tools can read it.

Argument	Meaning	Output Effect
<code>mlir-translate</code>	Runs the MLIR translation utility from the LLVM installation.	Converts one IR text format to another.
<code>-mlir-to-llvmir</code>	Selects translation from MLIR LLVM dialect to standard LLVM IR.	Emits <code>.ll</code> .
input <code>.onnx.mlir</code>	Input file produced by Stage 3.	Reads LLVM-dialect MLIR.
<code>-o output.ll</code>	Output file path.	Writes <code>resnet18.ll</code> .

Table 18: Stage 4 command arguments.

```
1 target triple = "riscv64-unknown-linux-gnu"
2
3 declare void @om_gemmini_conv_f32(ptr, ptr, ptr, i64, i64)
4
5 define ptr @run_main_graph(ptr %arg0) {
6   ; compiler-generated model body
7 }
```

Listing 22: Expected LLVM IR Patterns

Why the Runtime Function Is Declared but Not Defined

The `declare` line means: “the generated model code calls this function, but the function body is not in this file.” This is normal. The implementation will come from `libgemmini_f32_rv64.a` during the final static link.

8.5 Stage 5: LLVM IR to RISC-V Object File

```
1 $LLVM_BIN/llc \  
2 -O2 \  
3 -march=rv64gc \  
4 -mabi=lp64d \  
5 --mtriple=riscv64-unknown-linux-gnu \  
6 --mcpu=generic-rv64 \  
7 --mattr=+m,+a,+c,+f,+d \  
8 -filetype=obj \  
9 -relocation-model=static \  
10 -o $OUT_DIR/${MODEL_BASENAME}.o \  
11 $OUT_DIR/${MODEL_BASENAME}.ll
```

Listing 23: Compile LLVM IR to a RISC-V Object File

Stage 5 is the actual RISC-V code generation step for the compiled model. The LLVM backend selects RISC-V instructions, assigns registers, creates sections such as `.text` and `.rodata`, and produces a relocatable object file.

Argument	Meaning	Output Effect
<code>llc</code>	LLVM static compiler backend.	Converts LLVM IR into target machine code.
<code>-O2</code>	Optimization level for backend code generation.	Better instruction selection and scheduling.
<code>-march=rv64gc</code>	64-bit RISC-V with common extensions.	Matches target ISA expected by the runtime.
<code>-mabi=lp64d</code>	ABI with 64-bit long and pointer values plus double-precision floating-point calling convention.	Must match LLVM/Clang-compiled runtime objects.
<code>-mtriple=...</code>	Target triple.	Generates Linux RISC-V object format.
<code>-mcpu=generic-rv64</code>	Generic RISC-V CPU selection.	Avoids depending on a more specific core.
<code>-mattr=...</code>	Enables integer multiply/divide, atomics, compressed instructions, single-precision FP, and double-precision FP.	Allows backend to emit those instructions.
<code>-filetype=obj</code>	Requests an object file instead of assembly text.	Writes ELF relocatable <code>.o</code> .
<code>-reloc-model=static</code>	Generates code intended for static linking.	Aligns with final <code>-static</code> link.

Table 19: Stage 5 command arguments.

8.5.1 What Is Inside `resnet18.o`?

Object Content	Example	Meaning
Defined model functions	<code>run_main_graph</code>	Code generated from the neural-network graph.
Read-only constants	<code>constant_30_resnet18</code>	Trained weights and constant tensors.
Undefined runtime symbols	<code>U om_gemmini_conv_f32</code>	Calls that must be resolved by Stage 9 linking.
Relocations	Calls and data references with placeholder addresses	The linker fills final addresses later.

Table 20: Main contents of the model object file.

How to Verify Stage 5

```
1 file $OUT_DIR/${B}.o
2 $LLVM_NM -u $OUT_DIR/${B}.o | grep om_gemmini
3 $LLVM_NM -S $OUT_DIR/${B}.o | grep constant_ | head
```

Expected result: the object file should be a RISC-V relocatable ELF object. Gemmini functions should appear as undefined symbols at this point because they are provided by the runtime library later.

8.6 Stage 6: Build the ONNX-MLIR C Runtime Library

LLVM Toolchain Variant

In this version, the C and C++ parts are compiled with LLVM tools: `clang`, `clang++`, `llvm-ar`, `llvm-nm`, `llvm-objdump`, and `llvm-readelf`. The important option is `-target=riscv64-unknown-linux-gnu`. For Linux static linking, Clang still needs a RISC-V sysroot containing `libc`, `libm`, and C++ runtime libraries. That is why the script defines `RISCV_SYSROOT`.

```
1 mkdir -p $OUT_DIR/rt_obj
2
3 $RISCV_CC $CLANG_TARGET -c $FLAGS_RV -I $ROOT_DIR/include \
4   $ROOT_DIR/src/Runtime/OnnxDataType.c \
5   -o $OUT_DIR/rt_obj/OnnxDataType.o
6
7 $RISCV_CC $CLANG_TARGET -c $FLAGS_RV -I $ROOT_DIR/include \
8   $ROOT_DIR/src/Runtime/OMTensor.c \
9   -o $OUT_DIR/rt_obj/OMTensor.o
10
11 $LLVM_AR rcs $OUT_DIR/libcruntime_rv64.a \
12   $OUT_DIR/rt_obj/OnnxDataType.o \
13   $OUT_DIR/rt_obj/SmallFPConversion.o \
14   $OUT_DIR/rt_obj/OMTensor.o \
15   $OUT_DIR/rt_obj/OMTensorList.o \
16   $OUT_DIR/rt_obj/OMInstrument.o
```

Listing 24: Compile and Archive the ONNX-MLIR Runtime

The compiled model does not manage tensors by itself. It expects the ONNX-MLIR runtime to provide tensor containers, tensor-list containers, data-type helpers, and optional instrumentation helpers. Stage 6 builds those pieces for the same RISC-V ABI used by the model object.

Command Part	Meaning	Output Effect
<code>mkdir -p \$OUT_DIR/rt_obj</code>	Creates the folder for runtime object files. The <code>-p</code> option avoids an error if the folder already exists.	Creates or reuses <code>rt_obj/</code> .
<code>\$RISCV_CC</code>	LLVM Clang C compiler.	Provides the compiler executable.
<code>\$CLANG_TARGET</code>	Adds <code>-target=riscv64-unknown-linux-gnu</code> and <code>-sysroot=...</code> .	Tells Clang to emit RISC-V Linux code and find target headers/libraries.
<code>-c</code>	Compile only; do not link yet.	Creates <code>.o</code> files.
<code>\$FLAGS_RV</code>	Common flags such as <code>-O2 -march=rv64gc -mabi=lp64d</code> .	Keeps runtime ABI compatible with the model object.
<code>-I \$ROOT_DIR/include</code>	Adds ONNX-MLIR headers to the include search path.	Lets source files include OMTensor and runtime declarations.
<code>OnnxDataType.c</code> , <code>OMTensor.c</code> , etc.	Runtime source files.	Each source becomes one RISC-V object file.
<code>-o \$OUT_DIR/rt_obj/name.o</code>	Output object path.	Stores compiled objects inside <code>rt_obj/</code> .
<code>\$LLVM_AR rcs</code>	Uses LLVM archiver to create or update a static archive. <code>r</code> inserts files, <code>c</code> creates the archive, <code>s</code> writes an index.	Produces <code>libcruntime_rv64.a</code> .

Table 21: Stage 6 command arguments.

Runtime Object	Provides	Why the Model Needs It
<code>OnnxDataType.o</code>	ONNX data-type IDs, sizes, and names.	The runtime must know whether a tensor is float, int, etc.
<code>SmallFPConversion.o</code>	Float16 and bfloat16 conversion helpers.	Some models or runtime paths need small floating-point conversions.
<code>OMTensor.o</code>	Tensor creation, shape access, data pointer access, ownership handling.	Inputs, outputs, weights, and intermediate tensors are passed as OMTensor objects.
<code>OMTensorList.o</code>	List wrapper for model inputs and outputs.	The compiled entry point receives and returns tensor lists.
<code>OMInstrument.o</code>	Optional instrumentation hooks.	Useful for timing or debug builds.

Table 22: Files included in the ONNX-MLIR C runtime archive.

ABI Consistency

The C runtime must be compiled with the same important target ABI flags as the model object, especially `-march=rv64gc` and `-mabi=lp64d`. If the model object and runtime library use incompatible ABIs, the linker or the final program can fail.

8.7 Stage 7: Build the Gemmini Runtime Library

```

1 mkdir -p $OUT_DIR/gem_rt_obj
2
3 GEMMINI_HW_ABI_DIR=$ROOT_DIR/src/Accelerators/Gemmini/Runtime/gemmini-
  hardware-abi
4 GEMMINI_HW_ABI_INCLUDE=$GEMMINI_HW_ABI_DIR/include
5
6 $RISCV_CXX $CLANG_TARGET -c $FLAGS_RV \
7   -I $ROOT_DIR/include \
8   -I $ROOT_DIR \
9   -I $GEMMINI_HW_ABI_DIR \
10  -I $GEMMINI_HW_ABI_INCLUDE \
11  $ROOT_DIR/src/Accelerators/Gemmini/Runtime/OMRuntimeGemmini.cpp \
12  -o $OUT_DIR/gem_rt_obj/OMRuntimeGemmini.o
13
14 $LLVM_AR rcs $OUT_DIR/libgemmini_f32_rv64.a \
15  $OUT_DIR/gem_rt_obj/OMRuntimeGemmini.o

```

Listing 25: Compile and Archive the Gemmini Runtime

Stage 7 builds the implementation of the `om_gemmini_*` functions that appeared earlier as declarations or undefined symbols. This archive is the bridge from model-level runtime calls to Gemmini-level computation.

Command Part	Meaning	Output Effect
<code>mkdir -p \$OUT_DIR/gem_rt_obj</code>	Creates the output folder for Gemmini runtime object files.	Creates or reuses <code>gem_rt_obj/</code> .
<code>GEMMINI_HW_ABI_DIR=...</code>	Points to the bundled Gemmini hardware ABI directory.	Makes hardware-facing headers available.
<code>\$RISCV_CXX</code>	LLVM Clang++ C++ compiler. The Gemmini runtime is C++ source.	Provides the compiler executable.
<code>\$CLANG_TARGET</code>	Selects RISC-V Linux as the compilation target.	Produces <code>OMRuntimeGemmini.o</code> for the target architecture.
<code>-c</code>	Compile only; do not link.	Keeps runtime code as an object file for archiving.
<code>-I \$ROOT_DIR/include</code>	ONNX-MLIR runtime headers.	Allows access to <code>OMTensor</code> APIs.
<code>-I \$GEMMINI_HW_ABI_DIR/include</code>	Gemmini ABI headers.	Allows the runtime to use Gemmini helper functions/-macros.
<code>Runtime.cpp</code>	Main Gemmini runtime implementation.	Defines <code>om_gemmini_conv_f32</code> , <code>om_gemmini_relu_f32</code> , etc.
<code>\$LLVM_AR rcs</code>	Archives the object into a static library.	Produces <code>libgemmini_f32_rv64.a</code> .

Table 23: Stage 7 command arguments.

Include/Input	Why It Is Needed	Effect
<code>Runtime.cpp</code>	Implements functions such as <code>om_gemmini_conv_f32</code> .	Provides definitions for symbols called by the model object.
<code>\$ROOT_DIR/include</code>	Provides ONNX-MLIR runtime headers such as <code>OMTensor</code> declarations.	Allows the Gemmini runtime to read tensor shapes and data pointers.
<code>gemmini-hardware-abi</code>	Provides Gemmini hardware-facing headers and helper macros.	Allows the runtime to call Gemmini helper functions and RoCC macros.
<code>\$RISCV_CXX + \$CLANG_TARGET</code>	Uses LLVM Clang++ with RISC-V target options.	Produces RISC-V C++ object code.
<code>libgemmini_f32_rv64.a</code>	Static archive containing the Gemmini runtime object.	Linked in Stage 9 to resolve <code>om_gemmini_*</code> .

Table 24: Gemmini runtime build inputs and their roles.

8.7.1 Inside a Gemmini Runtime Function

A runtime function such as `om_gemmini_conv_f32` usually follows this logic:

1. Read input, weight, bias, and output tensor metadata from `OMTensor*` arguments.
2. Validate dimensions and element type.
3. Compute output dimensions from kernel size, stride, and padding.
4. Prepare the data layout expected by Gemmini, often by transforming convolution into matrix multiplication.
5. Call Gemmini helper functions such as tiled matrix multiplication or convolution helpers.
6. Store the computed output back into the output tensor buffer.

Driver Analogy

The compiled model is like an application that calls a driver API. `Runtime.cpp` is the driver. The hardware ABI headers describe the device interface. The RoCC instructions are the low-level commands sent to the device.

8.7.2 Runtime Compile Smoke Test and Warning Pass

This check documents an important practical question: **does `Runtime.cpp` compile for RISC-V with the bundled Gemmini ABI headers?** The answer observed for this project is yes: the runtime compiles cleanly. After enabling stricter warnings, the remaining warnings come from the bundled `gemmini.h`/RoCC header area, not from the local runtime source file.

Why This Check Matters

A runtime file can look correct at the C++ level but still fail when compiled for the target because accelerator headers, ABI flags, or RISC-V target assumptions are missing. A direct compile smoke test catches this before the full static link stage.

```

1 $RISCV_CXX $CLANG_TARGET -std=c++17 -c $FLAGS_RV \
2   -I $ROOT_DIR/include \
3   -I $ROOT_DIR \
4   -I $ROOT_DIR/src/Accelerators/Gemmini/Runtime/gemmini-hardware-abi \
5   -I $ROOT_DIR/src/Accelerators/Gemmini/Runtime/gemmini-hardware-abi/
6   include \
7   -I $ROOT_DIR/src/Accelerators/Gemmini/Runtime/gemmini-hardware-abi/
8   rocc-software/src \
9   $ROOT_DIR/src/Accelerators/Gemmini/Runtime/OMRuntimeGemmini.cpp \
10  -o /tmp/OMRuntimeGemmini.o

```

Listing 26: LLVM/Clang Runtime Smoke Test

```

1 riscv64-linux-gnu-g++ -std=c++17 -c \
2   -I. \
3   -Isrc/Accelerators/Gemmini/Runtime/gemmini-hardware-abi \
4   -Isrc/Accelerators/Gemmini/Runtime/gemmini-hardware-abi/include \
5   -Isrc/Accelerators/Gemmini/Runtime/gemmini-hardware-abi/rocc-software
6   /src \
7   src/Accelerators/Gemmini/Runtime/OMRuntimeGemmini.cpp \
8   -o /tmp/OMRuntimeGemmini.o

```

Listing 27: Equivalent GNU Cross-Compiler Smoke Test

LLVM Is the Main Flow Here

The main documented pipeline uses `clang`, `clang++`, `llvm-ar`, `llvm-nm`, `llvm-objdump`, `llvm-readelf`, and `lld`. The GNU command above is included only as an equivalent cross-compiler sanity check because the same source should compile cleanly with either RISC-V toolchain when the include paths and ABI options are correct.

```

1 riscv64-linux-gnu-g++ -std=c++17 -Wall -Wextra -Wpedantic \
2   -I. \
3   -Isrc/Accelerators/Gemmini/Runtime/gemmini-hardware-abi \
4   -Isrc/Accelerators/Gemmini/Runtime/gemmini-hardware-abi/include \
5   -Isrc/Accelerators/Gemmini/Runtime/gemmini-hardware-abi/rocc-software
6   /src \
7   -c src/Accelerators/Gemmini/Runtime/OMRuntimeGemmini.cpp \
8   -o /tmp/OMRuntimeGemmini.warn.o

```

Listing 28: Warning-Enabled Runtime Compile

Warning Check Result	Meaning	Action
OMRuntimeGemmini.cpp compiles with no blocking error.	The local runtime implementation is syntactically and ABI-compatible enough to produce a RISC-V object file.	Continue to archive/link stages.
Warnings from bundled gemmini.h/RoCC headers.	The warning source is inside the imported hardware ABI headers, not the local runtime file.	Keep separate from local runtime cleanup; avoid editing bundled headers unless intentionally maintaining the ABI package.
Unused output_pad in ConvTranspose.	The parameter is accepted by the API, but the implementation already uses the output tensor shape to represent output padding.	Mark intentional with (void)output_pad; near the beginning of the implementation.
Unused local helper maxAbsF32.	A helper exists but is not used by the current runtime path.	Remove the unused helper to keep local warnings clean.

Table 25: Runtime warning-pass interpretation.

```

1 // Removed unused helper:
2 // static float maxAbsF32(const float *data, int64_t n) { ... }
3
4 void om_gemmini_convtranspose_f32_impl(..., int64_t output_pad, ...) {
5     assert(out && x && w);
6     (void)output_pad; // output padding is already reflected in output
7     shape
8     const int64_t *xShape = omTensorGetShape(x);
9     ...
10 }

```

Listing 29: Small Local Cleanup Applied to the Runtime

How to Read the Result

The important distinction is not “warnings exist” versus “no warnings exist”. The useful question is **where the warnings come from**. In this check, the runtime-local warnings were removed. The remaining warning output is from bundled Gemmini ABI/RoCC headers. Therefore, the local file `Runtime.cpp` is considered clean enough for the pipeline.

Repository-State Note

During this kind of check, use `git status -short` before interpreting `git diff`. If `Runtime.cpp` and `Runtime.hpp` are untracked, a normal tracked diff may show nothing even though the on-disk source has been edited. In the observed project state, `Runtime/CMakeLists.txt` already had staged/uncommitted work, so the runtime cleanup should be accounted for separately from broader CMake changes.

```

1 git status --short \
2   src/Accelerators/Gemmini/Runtime/OMRuntimeGemmini.cpp \
3   src/Accelerators/Gemmini/Runtime/OMRuntimeGemmini.hpp \
4   src/Accelerators/Gemmini/Runtime/CMakeLists.txt
5
6 git diff -- src/Accelerators/Gemmini/Runtime/OMRuntimeGemmini.cpp
7
8 git diff --cached -- \
9   src/Accelerators/Gemmini/Runtime/OMRuntimeGemmini.cpp

```

Listing 30: Useful Status and Diff Checks

8.8 Stage 8: Compile the Runner Program

```

1 $RISCV_CC $CLANG_TARGET -c $FLAGS_RV -I $ROOT_DIR/include \
2   $ROOT_DIR/gemmini/examples/tools/float_model_runner.c \
3   -o $OUT_DIR/float_model_runner.o

```

Listing 31: Compile the Test Runner

The compiled model object provides the model function, but a Linux executable also needs an entry point. The runner provides `main()`, prepares input tensors, calls `run_main_graph()`, and prints or checks the output.

Argument	Meaning	Output Effect
<code>\$RISCV_CC</code>	LLVM Clang C compiler.	Compiles the C runner source.
<code>\$CLANG_TARGET</code>	Selects the RISC-V Linux target and sysroot.	Produces target-compatible <code>float_model_runner.o</code> .
<code>-c</code>	Compile only; do not link yet.	Produces <code>float_model_runner.o</code> .
<code>\$FLAGS_RV</code>	Shared optimization, ISA, and ABI flags.	Keeps runner ABI compatible with model and libraries.
<code>-I include</code>	Includes ONNX-MLIR runtime headers.	Runner can create <code>OMTensor</code> and <code>OMTensorList</code> objects.
<code>float_model_runner.c</code>	Test harness source file.	Provides <code>main()</code> .

Table 26: Stage 8 command arguments.

8.8.1 How the ResNet-18 Runner Connects to the Compiled Model

For `gemmini/tests/15-resnet18-float`, the runner is not a special file stored inside only that test folder. The test uses the generic floating-point runner from `gemmini/examples/tools/float_`

`model_runner.c`. That runner is compiled into `float_model_runner.o` and then linked with the ResNet-18 model object and both runtime libraries.

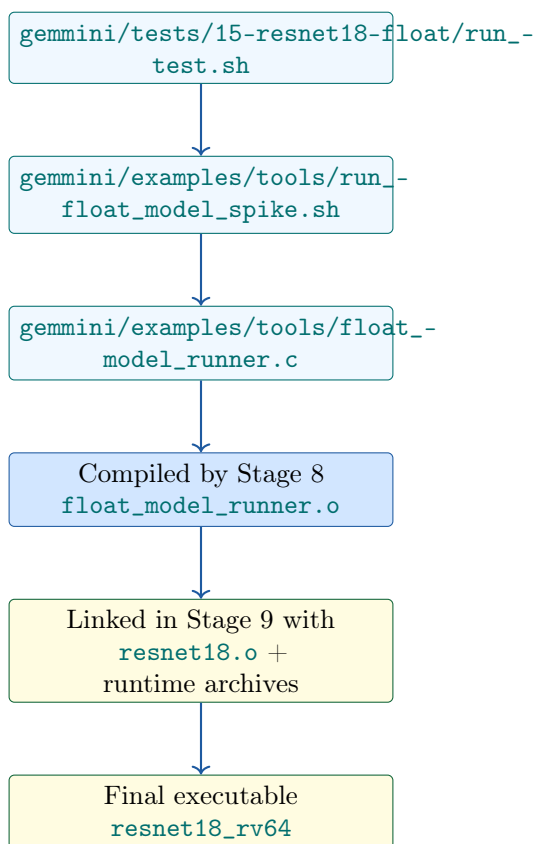


Figure 9: Runner script and source-file path for the ResNet-18 float test.

File	Role	Important Line/Concept
<code>gemmini/tests/15-resnet18-float/run_test.sh</code>	Test-specific script for ResNet-18 float.	Starts the test flow for this model.
<code>gemmini/examples/tools/run_float_model_spike.sh</code>	Generic helper script for compiling, linking, and running float models on Spike.	Uses <code>float_model_runner.c</code> by default.
<code>gemmini/examples/tools/float_model_runner.c</code>	Generic RV64 runner source.	Provides <code>main()</code> and declares <code>run_main_graph</code> .
<code>float_model_runner.o</code>	Compiled runner object inside the test output folder.	Provides the executable entry point.
<code>resnet18.o</code>	Compiled model object.	Provides the generated <code>run_main_graph</code> implementation.
<code>libgemmini_f32_rv64.a</code>	Gemmini runtime archive.	Resolves <code>om_gemmini_*</code> calls.
<code>libcruntime_rv64.a</code>	ONNX-MLIR C runtime archive.	Resolves <code>OMTensor</code> and <code>OMTensorList</code> functions.
<code>resnet18_rv64</code>	Final statically linked RISC-V executable.	Can be run under Spike with Gemmini extension.

Table 27: Runner and link artifacts for test 15-resnet18-float.

```

1 #include <OnnxMlirRuntime.h>
2
3 /* Model entry point provided by the compiled model object: resnet18.o
   */

```



```

4 extern OMTensorList *run_main_graph(OMTensorList *);
5
6 /* Gemmini accelerator compatibility/init symbol used by the runtime.
   */
7 uint64_t OMInitCompatibleAccelGemmini(uint64_t versionNum) {
8     return versionNum == 0 ? 1 : versionNum;
9 }

```

Listing 32: Key Runner Entry Point

Simple Meaning of run_main_graph

run_main_graph is the compiled neural-network function. The runner does not implement ResNet-18 itself. It only builds input tensors, calls run_main_graph(inputList), receives an output tensor list, and prints/checks the result.

```

1 int main(int argc, char **argv) {
2     // 1. Resolve input shape, defaulting to CNN image input
3     // [1,3,224,224].
4     int64_t inputShape[4] = {1, 3, 224, 224};
5
6     // 2. Allocate deterministic input data.
7     float *inputData = malloc(sizeof(float) * 1 * 3 * 224 * 224);
8     for (int64_t i = 0; i < 1 * 3 * 224 * 224; ++i)
9         inputData[i] = deterministic_f32(i);
10
11     // 3. Wrap raw data in OMTensor and then in OMTensorList.
12     OMTensor *input = omTensorCreateWithOwnership(
13         inputData, inputShape, 4, ONNX_TYPE_FLOAT, 1);
14     OMTensorList *inputList = omTensorListCreate(&input, 1);
15
16     // 4. Call compiled model.
17     OMTensorList *outputList = run_main_graph(inputList);
18
19     // 5. Read and verify the output tensor.
20     OMTensor *out = omTensorListGetOmtByIndex(outputList, 0);
21     const float *outData = (const float *)omTensorGetDataPtr(out);
22     print_top5(outData, omTensorGetNumElems(out));
23 }

```

Listing 33: Simplified Runner Logic

What the Runner Prints

The runner checks that the model produced an output tensor, prints the output shape and number of elements, checks for NaN/Inf values, prints the top-5 output scores when possible, and can optionally dump the output tensor as JSON using -dump-output.

```

1 spike --extension=gemmini --isa=rv64imafdc pk ./resnet18_rv64

```

Listing 34: Typical Simulator Invocation

Why the Runner Is Generic

The same `float_model_runner.c` can run different ONNX-MLIR float models because it does not know the internal layers. It only knows the ABI: create `OMTensorList` inputs, call `run_main_graph`, and inspect `OMTensorList` outputs. For CNNs it defaults to `[1,3,224,224]`; for BERT-like models it can create three `INT64` input tensors.

8.9 Stage 9: Static Linking into the Final RISC-V Executable

```
1 $RISCV_CXX $CLANG_TARGET $FLAGS_RV -fuse-ld=lld -static \  
2 -o $OUT_DIR/${MODEL_BASENAME}_rv64 \  
3 $OUT_DIR/float_model_runner.o \  
4 $OUT_DIR/${MODEL_BASENAME}.o \  
5 $OUT_DIR/libgemmini_f32_rv64.a \  
6 $OUT_DIR/libcruntime_rv64.a \  
7 -lm
```

Listing 35: Static Link

Stage 9 combines everything into one statically linked ELF executable. This is the first point where all function calls can be fully resolved: `main()` comes from the runner, `run_main_graph()` comes from the model object, `om_gemmini_*` comes from the Gemmini runtime archive, and `omTensor*` helpers come from the ONNX-MLIR C runtime archive.

Command Part	Meaning	Output Effect
<code>\$RISCV_CXX</code>	Uses the LLVM Clang++ linker driver. C++ is used because the Gemmini runtime was compiled as C++.	Links model, runner, C runtime, C++ runtime dependencies, and Gemmini runtime correctly.
<code>\$CLANG_TARGET</code>	Keeps the linker target and sysroot aligned with compilation.	Finds RISC-V target libraries instead of host x86 libraries.
<code>-fuse-ld=lld</code>	Requests LLVM LLD as the linker.	Keeps the final link in the LLVM toolchain.
<code>\$FLAGS_RV</code>	Same RISC-V ISA and ABI flags used earlier.	Prevents ABI mismatch between objects and libraries.
<code>-static</code>	Link statically instead of relying on shared libraries on the target.	Produces a self-contained executable.
<code>-o resnet18_rv64</code>	Output executable path.	Writes the final ELF file.
<code>float_model_runner.o</code>	Provides <code>main()</code> .	Program has an entry point.
<code>resnet18.o</code>	Provides generated model code and weights.	Adds <code>run_main_graph</code> and model constants.
<code>libgemmini_f32_rv64.a</code>	Provides accelerator runtime function definitions.	Resolves <code>om_gemmini_*</code> .
<code>libcruntime_rv64.a</code>	Provides tensor runtime functions.	Resolves <code>omTensor*</code> and <code>OMTensorList*</code> .
<code>-lm</code>	Links the math library.	Resolves math functions if present.

Table 28: Stage 9 command arguments.

Link Input	Provides	Why Order Matters
<code>float_model_runner.o</code>	<code>main()</code> and test harness logic.	It references model entry points.
<code>resnet18.o</code>	Compiled model functions and weights.	It references Gemmini and ONNX runtime functions.
<code>libgemmini_f32_rv64.a</code>	<code>om_gemmini_*</code> definitions.	Must appear after the object that references those symbols.
<code>libcruntime_rv64.a</code>	<code>omTensor*</code> and tensor-list helper definitions.	Must appear after model/runtime objects that reference tensor helpers.
<code>-lm</code>	Math functions such as <code>expf</code> , <code>sqrtf</code> , or other libm functions.	Resolves math-library calls if used.

Table 29: Final link inputs and symbol resolution.

Why -static Is Used

Static linking copies needed library code into the executable. For a simulator, FPGA SoC image, or minimal RISC-V Linux environment, this is easier than managing shared libraries on the target filesystem. The tradeoff is file size: the executable becomes larger, but it is more self-contained.

How to Verify the Final Executable

```

1 file $OUT_DIR/${B}_rv64
2 $LLVM_READELF -h $OUT_DIR/${B}_rv64 | grep Machine
3 $LLVM_READELF -SW $OUT_DIR/${B}_rv64 | grep -E '\.text|\.rodata|\.
  data|\.bss'
4 $LLVM_NM -S $OUT_DIR/${B}_rv64 | grep om_gemmini_conv_f32
5 $LLVM_NM -u $OUT_DIR/${B}_rv64 | head

```

Expected result: the file should be a statically linked RISC-V ELF executable. The Gemmini runtime functions should now be defined, not undefined. Ideally, `nm -u` should not show unresolved application/runtime symbols.

8.10 End-to-End Artifact Map

Stage	Main Output	Human Interpretation	Main Check
1	<code>resnet18_-onnxir.onnx.mlir</code>	Model graph in ONNX dialect.	Search for <code>onnx.Conv</code> .
2	<code>resnet18_-krnl.onnx.mlir</code>	Accelerator calls plus generic Krnl/LLVM code for non-Gemmini operations.	Search for <code>om_gemmini_</code> .
3	<code>resnet18.onnx.mlir</code>	LLVM-dialect MLIR, close to ABI.	Search for <code>llvm.call</code> .
4	<code>resnet18.ll</code>	Plain LLVM IR.	Search for target triple and <code>declare</code> .
5	<code>resnet18.o</code>	RISC-V model object.	<code>file</code> and <code>nm -u</code> .
6	<code>libcruntime_rv64.a</code>	Tensor runtime archive.	<code>ar t</code> .
7	<code>libgemmini_f32_-rv64.a</code>	Gemmini runtime archive.	<code>nm</code> shows <code>om_gemmini_*</code> .
8	<code>float_model_runner.o</code>	Program entry point object.	<code>nm</code> shows <code>main</code> .
9	<code>resnet18_rv64</code>	Final static RISC-V executable.	<code>file</code> , <code>readelf</code> , and no unresolved symbols.

Table 30: End-to-end artifact map for the ONNX to RISC-V/Gemmini build.

8.11 Most Important Conceptual Flow

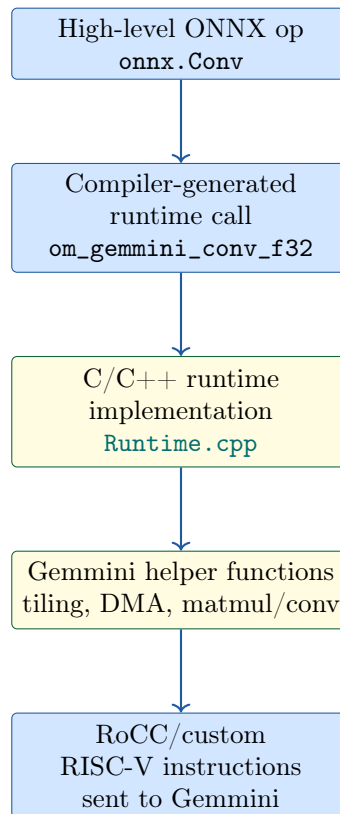


Figure 10: The essential path from a neural-network operation to Gemmini hardware execution.

9 Running the Final Executable on Spike with Gemmini

After the static executable `resnet18_rv64` is built, the final practical validation is to run it on the RISC-V Spike simulator with the Gemmini extension enabled. This stage does not compile anything new. It answers a different question: **can the produced RISC-V executable actually start, call the runner, execute the model, and return a valid output tensor under a Gemmini-enabled simulator?**

Observed Run Location

The commands below were executed from the ONNX-MLIR project root:

```
1 /home/lahcen/toolchains/onnx-mlir-xcompile/onnx-mlir
```

The executable being tested is:

```
1 gemmini/tests/15-resnet18-float/resnet18_rv64
```

9.1 Preparing the Simulator Library Path

```
1 export LD_LIBRARY_PATH=/home/lahcen/riscv-gemmini/lib:${LD_LIBRARY_PATH}
}
```

Listing 36: Expose the Gemmini Spike Libraries

Command Part	Meaning	Why It Matters
<code>export</code>	Sets an environment variable for the current shell.	The following spike command inherits this setting.
<code>LD_LIBRARY_PATH</code>	Runtime search path for shared libraries on Linux.	Helps the host system find Spike/Gemmini shared libraries.
<code>/home/lahcen/riscv-gemmini/lib</code>	Directory containing the Gemmini simulator libraries.	Required when the Gemmini Spike build depends on libraries outside the default system path.
<code>:\${LD_LIBRARY_PATH}</code>	Keeps the old library path after adding the new one.	Avoids deleting existing library search paths.

Table 31: Meaning of the LD_LIBRARY_PATH setup command.

Why This Is a Host-Side Setting

This command is not for the RISC-V program itself. It is for the **host Linux machine** running Spike. If the host cannot load the Gemmini-enabled Spike libraries, the simulation may fail before the RISC-V executable even starts.

9.2 Basic Spike Run Command

```

1 /home/lahcen/riscv-gemmini/bin/spike --extension=gemmini --isa=
   rv64imafdc \
2 /usr/local/riscv/riscv64-unknown-elf/bin/pk \
3 gemmini/tests/15-resnet18-float/resnet18_rv64 \
4 2>&1 | tee gemmini/tests/15-resnet18-float/spike_run.log

```

Listing 37: Run the ResNet-18 RV64 Executable on Spike

Argument	Meaning	Output Effect
<code>/home/lahcen/ riscv-gemmini/bin/spike -extension=gemmini</code>	Gemmini-enabled Spike executable. Loads/enables the Gemmini custom extension.	Starts the RISC-V simulator. Allows RoCC/Gemmini instructions to be recognized instead of treated as illegal instructions.
<code>-isa=rv64imafdc</code>	Selects the simulated RISC-V ISA: RV64 plus integer multiply/divide, atomics, single/-double floating point, and compressed instructions.	Must match the ISA used during compilation/linking, such as <code>rv64gc/rv64imafdc</code> .
<code>/usr/local/riscv/ riscv64-unknown-elf/bin/ pk gemmini/tests/ 15-resnet18-float/ resnet18_rv64</code>	Proxy kernel used by Spike to run a user-level program. Final statically linked ResNet-18 executable.	Provides a small execution environment for the RISC-V ELF. The actual program being executed.
<code>2>&1</code>	Redirects stderr into stdout.	Captures both normal prints and warnings/errors.
<code> tee spike_run.log</code>	Prints output to the terminal and saves it to a log file.	Preserves the execution transcript for documentation/debugging.

Table 32: Spike command arguments and why each one is used.

9.3 Line-Buffered Spike Run Command

Sometimes simulator or program output can appear late because of buffering. The line-buffered version uses `stdbuf` so that output is easier to see live while also saving it to a log file.

```

1 stdbuf -oL -eL /home/lahcen/riscv-gemmini/bin/spike \
2   --extension=gemmini \
3   --isa=rv64imafdc \
4   /usr/local/riscv/riscv64-unknown-elf/bin/pk \
5   gemmini/tests/15-resnet18-float/resnet18_rv64 \
6   2>&1 | tee gemmini/tests/15-resnet18-float/spike_run.log

```

Listing 38: Run Spike with Line-Buffered Output

Option	Meaning	Why It Helps
<code>stdbuf</code>	Changes buffering behavior of the launched program.	Useful when a long-running simulator does not print immediately.
<code>-oL</code>	Makes stdout line-buffered.	Each printed line is flushed more quickly.
<code>-eL</code>	Makes stderr line-buffered.	Error/warning lines also appear quickly.
<code>tee spike_run.log</code>	Saves the same terminal output to a file.	The run can be reviewed later without rerunning the simulator.

Table 33: Meaning of the `stdbuf` wrapper.

9.4 Observed Spike Output

```

1 Gemmini extension configured with:
2   dim = 16
3 [runner] input  shape: [1, 3, 224, 224]
4 [runner] output shape: [1, 1000]  elems=1000
5 [runner] output sum=1.000000  nans=0  infs=0
6 Top-5 outputs:
7   [1] index=125    score=+0.001040
8   [2] index=766    score=+0.001033
9   [3] index=350    score=+0.001029
10  [4] index=985     score=+0.001026
11  [5] index=292     score=+0.001026
12 [runner] PASS -- model produced output

```

Listing 39: Observed Spike Run Log

9.5 How to Interpret the Output

Output Line	Meaning	Validation Result
Gemmini extension configured with: <code>dim = 16</code>	Spike successfully loaded the Gemmini extension and reports a 16-by-16 systolic array dimension.	Strong evidence that the simulator is using the Gemmini-enabled path, not plain Spike.
input shape: <code>[1, 3, 224, 224]</code>	The generic runner created a CNN input tensor with batch 1, 3 channels, and 224-by-224 image size.	Matches the expected ResNet-style input shape.
output shape: <code>[1, 1000]</code>	The compiled model returned one vector of 1000 class scores.	Matches ImageNet-style ResNet-18 output.
output sum=1.000000	The output probabilities sum to 1.	Indicates the final softmax produced a normalized distribution.
nans=0 infs=0	No invalid floating-point values were found.	Confirms the run did not numerically explode.
Top-5 outputs	The runner prints the five largest output scores.	Confirms readable model output was produced.
PASS - model produced output	The runner reached the success path after calling <code>run_main_graph</code> .	End-to-end execution succeeded.

Table 34: Interpretation of the successful Spike run.

What This Run Proves

This run proves that:

- the final executable `resnet18_rv64` is runnable under Spike;
- the Gemmini extension is loaded by the simulator;
- the runner can create the expected input tensor;
- the compiled model entry point `run_main_graph` returns an output tensor;
- the output tensor has the expected ResNet-18 shape `[1,1000]`;
- the output is finite and normalized by softmax.

What This Run Does Not Prove

A successful run with deterministic dummy input proves that the pipeline executes, but it does not prove ImageNet accuracy. The printed top-5 indices are only a smoke-test result for the generated input tensor. To validate real accuracy, run the executable with real preprocessed images and compare predictions against expected

labels.

9.6 Spike Execution Flow Diagram

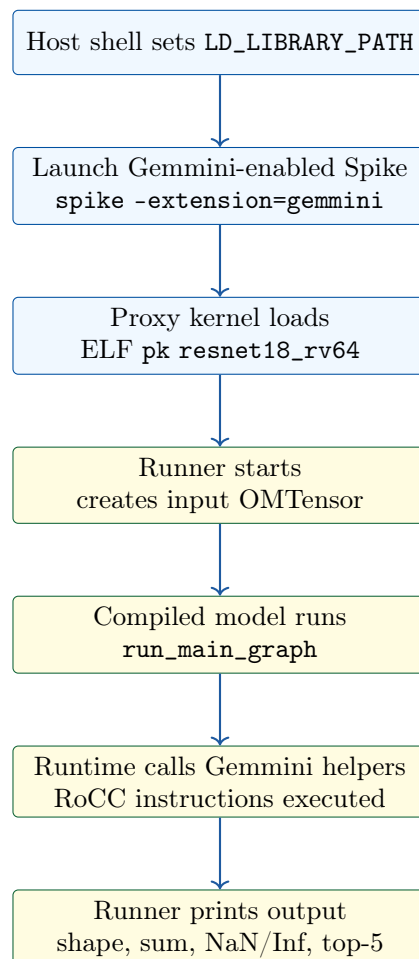


Figure 11: End-to-end execution path when running `resnet18_rv64` on Spike with Gemmini enabled.

10 Six-Level Responsibility Stack: Who Really Talks to Gemmini?

A common source of confusion is to think that every file in the final executable talks directly to the accelerator. That is not true. The application runner, the compiled model, the Gemmini runtime, the hardware ABI, the Spike simulator, and the real FPGA/silicon accelerator have different responsibilities.

Main idea

The runner does **not** know Gemmini. The compiled model does **not** implement Gemmini. The runtime library is the first software layer that directly translates model operations into Gemmini-oriented commands. Spike then emulates those commands in

software, while an FPGA/silicon implementation would execute them in real hardware.

10.1 Colored Overview Graph

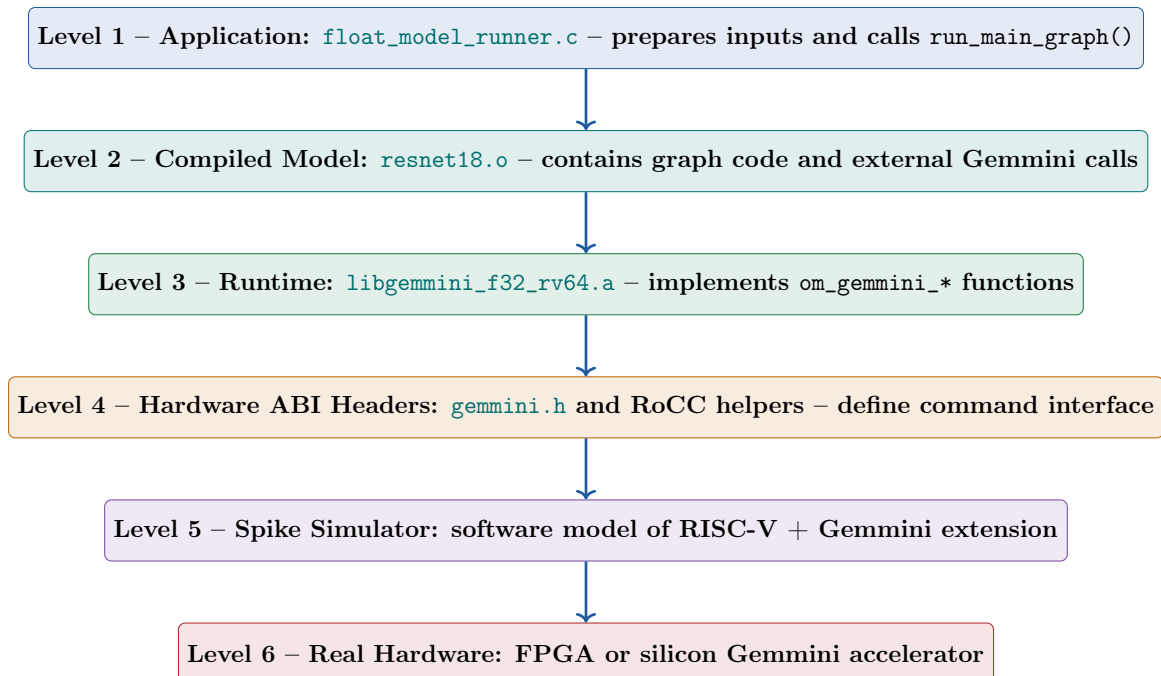


Figure 12: Responsibility stack from the application runner down to the real accelerator. Each layer has a different relationship with Gemmini.

10.2 Detailed Level-by-Level Explanation

Level 1: Application Layer (`float_model_runner.c`)

Relation with the accelerator: no direct accelerator relation.

- It does not configure Gemmini directly.
- It does not issue RoCC/custom accelerator instructions.
- It prepares inputs: image file or deterministic data → normalized tensor → `OMTensorList`.
- It calls only the compiled model entry point: `run_main_graph(inputList)`.
- It checks outputs: shape, number of elements, sum, NaN/Inf count, and top-5 scores.



Level 2: Compiled Model Layer (resnet18.o)

Relation with the accelerator: calls Gemmini runtime functions, but does not implement them.

- It contains the compiled ResNet-18 graph and the exported model entry point.
- It calls functions such as `om_gemmini_conv_f32()`, `om_gemmini_batchnorm_f32()`, `om_gemmini_relu_f32()`, `om_gemmini_add_f32()`, and `om_gemmini_gemm_f32_bias()`.
- These functions are **external symbols** in the model object. The model object knows their names and calling convention, but the function bodies are not inside `resnet18.o`.
- The linker later resolves those external symbols using `libgemmini_f32_rv64.a`.



Level 3: Gemmini Runtime Layer (libgemmini_f32_rv64.a)

Relation with the accelerator: direct software interface to the Gemmini execution path.

- It contains the compiled implementation of `Runtime.cpp`.
- It implements the functions called by the compiled model, for example `om_gemmini_conv_f32()` and `om_gemmini_gemm_f32_bias()`.
- It extracts raw data pointers and shapes from `OMTensor` objects.
- It prepares the operation: shape checks, layout assumptions, padding/stride handling, quantization or packing when needed, and output writing.
- It is the first layer where the model operation becomes a Gemmini-oriented operation rather than a generic neural-network graph operation.



Level 4: Hardware ABI Interface (gemmini-hardware-abi/include/gemmini.h and RoCC helpers)

Relation with the accelerator: declares the low-level command interface.

- It provides the C/C++ declarations, macros, constants, and helper interfaces used by the runtime to talk to Gemmini.
- It describes operations such as moving data into accelerator memory, computing, and moving results out.
- It exposes hardware configuration assumptions such as the array dimension. In the

observed Spike log, Gemmini reports `dim = 16`.

- It is not the neural-network model and not the application. It is the hardware-facing software contract.



Level 5: Spike Simulator with Gemmini Extension

Relation with the accelerator: software emulation of the accelerator.

- Spike executes the RISC-V program and intercepts Gemmini/RoCC operations when `--extension=gemmini` is enabled.
- The trace log proves this level is active because it prints: `Gemmini extension configured with: dim = 16`.
- Spike is useful for validation because it can run the final RV64 executable before FPGA/silicon deployment.
- It is still a simulator. It emulates behavior in software; it is not the physical accelerator.



Level 6: Real Hardware Layer (FPGA or Silicon)

Relation with the accelerator: the real accelerator implementation.

- This is where the systolic array, scratchpad, accumulator, DMA path, and RoCC interface exist as hardware logic.
- On FPGA, this means a synthesized Gemmini design running in programmable logic.
- On silicon, this means the accelerator is physically implemented on the chip.
- The Spike run does not prove FPGA timing or silicon performance. It proves software compatibility and simulator-level functional execution.

10.3 Separated Responsibility Table

Level	Main file or component	Accelerator relation	Best proof or check
1	<code>float_model_runner.c</code>	No direct Gemmini control. It only calls <code>run_main_graph()</code> .	Runner output lines: image input, input shape, output shape, top-5, and PASS.
2	<code>resnet18.o</code>	Calls external <code>om_gemmini_*</code> functions.	<code>llvm-nm -u resnet18.o grep om_gemmini.</code>
3	<code>libgemmini_f32_rv64.a / Runtime.cpp</code>	Implements the <code>om_gemmini_*</code> runtime functions.	<code>llvm-nm libgemmini_f32_rv64.a grep om_gemmini_conv_f32.</code>
4	<code>gemmini.h</code> , RoCC helpers	Declares Gemmini command interface and constants.	Inspect bundled ABI headers under <code>src/Accelerators/Gemmini/Runtime/gemmini-hardware-abi.</code>
5	Gemmini-enabled Spike	Emulates RISC-V + Gemmini in software.	Log line: <code>Gemmini extension configured with: dim = 16.</code>
6	FPGA / silicon implementation	Executes Gemmini operations in real hardware.	Requires FPGA/silicon deployment, not just Spike.

Table 35: Clear separated table showing what each layer does and how to verify it. The borders and alternating colors are intentional so adjacent responsibilities are not confused.

10.4 Do Not Confuse These Layers

Confusion	Correct meaning	Reason
<code>float_model_runner.c</code> uses Gemmini directly.	Incorrect. It only builds inputs, calls <code>run_main_graph()</code> , and prints outputs.	The runner is application-level code. It does not contain convolution/GEMM accelerator logic.
<code>resnet18.o</code> contains the full accelerator implementation.	Incorrect. It contains calls to external Gemmini runtime symbols.	The implementations are linked from <code>libgemmini_f32_rv64.a</code> .
<code>gemmini.h</code> is the simulator.	Incorrect. It is a hardware ABI/header interface used by runtime code.	The simulator behavior is in the Gemmini-enabled Spike build.
Spike is the real accelerator.	Incorrect. Spike is a software simulator.	It validates functionality, but it does not measure real FPGA/silicon timing.
A successful Spike run proves FPGA performance.	Incorrect. It proves the RV64 executable runs functionally under the simulator.	Hardware deployment is a separate validation stage.

Table 36: Common layer confusions and the corrected interpretation.

10.5 Minimal Verification Commands

```

1 # Level 1 -> Level 2: runner expects the model entry point.
2 llvm-nm float_model_runner.o | grep run_main_graph
3
4 # Level 2 -> Level 3: model object calls Gemmini runtime symbols.
5 llvm-nm -u resnet18.o | grep om_gemmini
6
7 # Level 3: runtime archive provides those symbols.
8 llvm-nm libgemmini_f32_rv64.a | grep om_gemmini_conv_f32
9
10 # Level 4: hardware ABI headers expose Gemmini command helpers.
11 grep -R "mvin\|compute\|mvout" \
12     src/Accelerators/Gemmini/Runtime/gemmini-hardware-abi/include \
13     src/Accelerators/Gemmini/Runtime/gemmini-hardware-abi/rocc-software/
14     src
15
16 # Level 5: Spike confirms the Gemmini extension at runtime.
17 grep "Gemmini extension configured" gemmini/tests/15-resnet18-float/
18     spike_run.log

```

Listing 40: Checks that connect each software layer

11 Real Image Execution with Gemmini Trace

The previous Spike run proves that the executable starts and returns an output tensor. The next validation is stronger: it passes a real image file into the same executable and

prints Gemmini runtime trace lines for the convolution and final GEMM path. This makes the run easier to audit because the log now shows both the **runner-side input preprocessing** and the **runtime-side accelerated operation statistics**.

Why this section was added

This section is designed to remove ambiguity. A simple PASS message only tells us that the program returned an output. The trace run tells us more:

- the runner accepted the image path and converted the image into a [1,3,224,224] tensor;
- the Gemmini-enabled Spike simulator started and reported a 16-by-16 Gemmini array;
- the runtime printed convolution and GEMM traces, so the model reached the Gemmini runtime functions;
- every printed tensor statistic reports `nans=0` and `infs=0`, so the observed run did not produce invalid floating-point values.

11.1 Command Used for the Real Image Run

```

1 cd /home/lahcen/toolchains/onnx-mlir-xcompile/onnx-mlir
2
3 GEMMINI_TRACE=1 /home/lahcen/riscv-gemmini/bin/spike \
4   --extension=gemmini \
5   --isa=rv64imafdc \
6   /usr/local/riscv/riscv64-unknown-elf/bin/pk \
7   gemmini/tests/15-resnet18-float/resnet18_rv64 \
8   --image gemmini/tests/15-resnet18-float/data/dog_py.ppm \
9   --in-n 1 --in-c 3 --in-h 224 --in-w 224

```

Listing 41: Run ResNet-18 on a real PPM image with Gemmini trace

Recommended saved-log variant

For documentation and debugging, it is better to save the full terminal output:

```

1 GEMMINI_TRACE=1 /home/lahcen/riscv-gemmini/bin/spike \
2   --extension=gemmini \
3   --isa=rv64imafdc \
4   /usr/local/riscv/riscv64-unknown-elf/bin/pk \
5   gemmini/tests/15-resnet18-float/resnet18_rv64 \
6   --image gemmini/tests/15-resnet18-float/data/dog_py.ppm \
7   --in-n 1 --in-c 3 --in-h 224 --in-w 224 \
8   2>&1 | tee gemmini/tests/15-resnet18-float/spike_dog_trace.log

```

`2>&1` merges standard error into standard output. This matters because trace messages are normally printed through the diagnostic stream. `tee` keeps the output visible on the screen while also writing it to a log file.

11.2 Argument-by-Argument Explanation

Argument	Meaning	Why it is used here
GEMMINI_TRACE=1	Enables the runtime trace mode for this command.	Makes the runtime print shapes and statistics for operations such as convolution and GEMM.
/home/lahcen/ riscv-gemmini/bin/ spike	Gemmini-enabled Spike simulator.	Runs the RV64 executable inside a simulator that understands the Gemmini extension.
--extension=gemmini	Loads the Gemmini Spike extension.	Without this, RoCC/Gemmini instructions are not interpreted by the simulator.
--isa=rv64imafdc	Selects the RISC-V ISA profile.	Matches the RV64 integer, multiply/divide, atomic, float, double, and compressed instruction support expected by the binary.
/usr/local/riscv/ riscv64-unknown-elf/ bin/pk	RISC-V proxy kernel.	Provides the minimal runtime environment used by Spike to launch a user ELF.
resnet18_rv64	Final static RV64 executable.	Contains the compiled ResNet-18 graph, weights, runner, and runtime libraries.
-image dog_py.ppm	Runner argument selecting a real input image.	Replaces deterministic dummy input with a real PPM image.
-in-n 1 -in-c 3 -in-h 224 -in-w 224	Runtime input dimensions.	Matches ResNet-18 image input shape: batch 1, RGB 3 channels, 224-by-224 pixels.

Table 37: Separated command table for the real-image Spike run. Each row has explicit borders to avoid confusing adjacent arguments.

11.3 Execution Flow Design

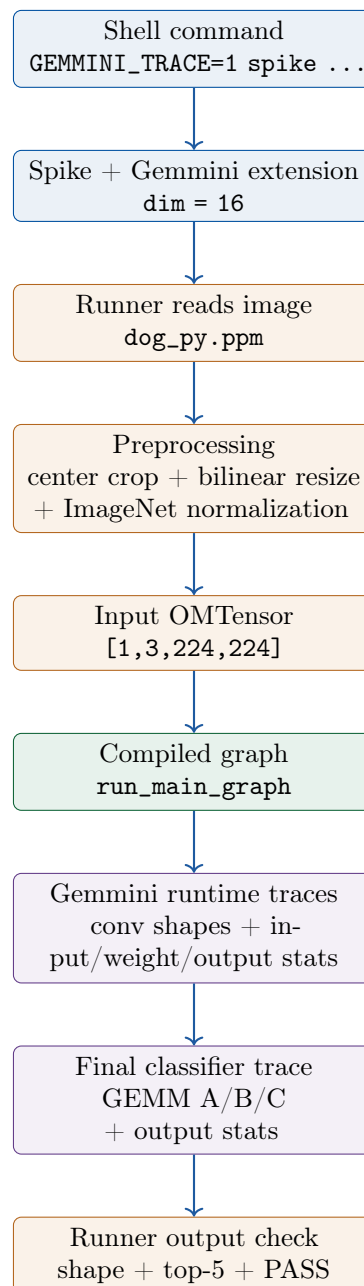


Figure 13: Real-image validation flow. The important difference from the dummy-input run is that the runner now constructs the model input from `dog_py.ppm`, then the Gemmini runtime trace confirms the convolution/GEMM path.

11.4 What the Trace Lines Mean

Reading one convolution trace

A line such as

```
1 [gemmini-trace] conv f32: x=[1,3,224,224] w=[64,3,7,7] y=[1,64,112,112] stride=2 pad=3 bias=no
```

means:

- **x** is the input tensor: batch 1, 3 channels, 224-by-224 image.
- **w** is the weight tensor: 64 output filters, 3 input channels, 7-by-7 kernel.
- **y** is the output tensor: batch 1, 64 output channels, 112-by-112 feature map.
- **stride=2** halves the spatial resolution from 224 to 112.
- **pad=3** adds three pixels of zero-padding around the image before the 7-by-7 filter is applied.
- **bias=no** means this convolution call did not receive a separate bias tensor.

The following statistics line summarizes a buffer:

```
1 [gemmini-trace] conv input: min=-2.100779 max=+2.640000 mean=+0.017939 nonzero=150528/150528 nans=0 infs=0
```

It is not another layer. It is a quick health check for the tensor values before or after an operation.

Printed field	Plain meaning	How to use it for debugging
min, max	Smallest and largest values in the tensor.	Detects exploding values or impossible ranges after preprocessing or computation.
mean	Average value across all elements.	Useful for seeing whether activations are centered, positive-only after ReLU, or drifting.
nonzero=a/b	Number of nonzero elements out of total elements.	After ReLU, many zeros are expected. Before preprocessing, all image pixels should usually become nonzero after normalization.
nans=0	No Not-a-Number values were observed.	Confirms the layer did not produce invalid arithmetic such as undefined results.
infs=0	No infinite values were observed.	Confirms the layer did not overflow to positive or negative infinity.

Table 38: Separated interpretation table for trace statistics. The row borders make each diagnostic field visually independent.

11.5 Shape Ladder Observed in the Trace

The trace follows the expected ResNet-18 downsampling pattern. Spatial resolution decreases while channel count increases. This is exactly what a CNN usually does: early layers keep more spatial detail, and deeper layers keep more semantic channels.

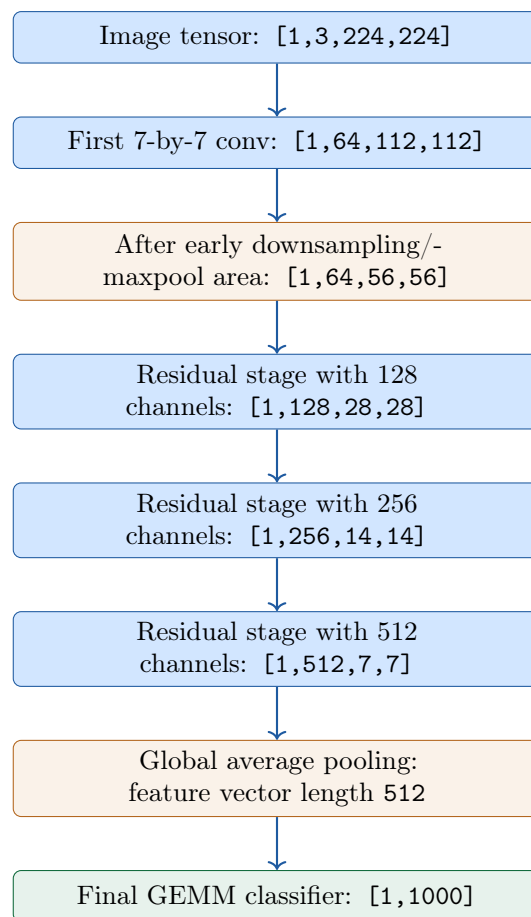


Figure 14: Shape ladder from the observed trace. The model compresses spatial size from 224-by-224 to 7-by-7, then produces 1000 class scores.

11.6 Evidence Table: What Was Proven by the Real-Image Run

Evidence in log	Observed value	Conclusion
RUNNER image input	dog_py.ppm, source [224,224,3], crop 224.	The runner used a real image instead of only deterministic synthetic data.
RUNNER prepro- cessing	center-crop + bilinear resize + ImageNet normalization.	The input was transformed into the format expected by ImageNet-style ResNet models.
RUNNER input tensor	[1, 3, 224, 224].	Runtime input dimensions match the compiled ResNet-18 input.
VALID input stats	nans=0 infs=0, 150528/150528 nonzero.	The preprocessed input tensor is finite and dense.
ACCELERATOR simulator	Gemmini extension configured with: dim = 16.	Spike loaded the Gemmini extension and reported the array dimension.
TRACE conv trace	Multiple [gemmini-trace] conv f32 lines.	The compiled model reached the Gemmini convolution runtime path.
TRACE GEMM trace	gemm A, gemm B, gemm C/bias, gemm output.	The final fully connected classifier path executed.
VALID output check	output shape: [1, 1000], nans=0, infs=0.	The model returned a finite 1000-class output tensor.
VALID final status	PASS - model produced output.	The runner completed successfully.

Table 39: Evidence table with explicit separators. Each row links one log observation to one validation conclusion.

11.7 Important Note About Comparing Scores

Do not over-interpret the top-5 labels

The trace log is mainly a **pipeline validation**, not an accuracy benchmark. The two observed logs show different output summaries: one run prints an output sum of 1.000000, while the traced run prints an output sum of 0.117750. This means the numeric top-5 scores should not be compared unless the exact same executable, runtime configuration, and final activation behavior are confirmed. For this documentation, the important checks are:

- the executable starts under Gemmini-enabled Spike;
- the image is loaded and normalized;
- Gemmini runtime conv/GEMM traces appear;
- the output has the expected shape [1,1000];

- there are no NaNs or infinities;
- the runner reports PASS.

11.8 Compact Extract of the Observed Log

```

1 [runner] image input: gemmini/tests/15-resnet18-float/data/dog_py.ppm source=[224,224,3] crop=224
  preprocess=center-crop+ilinear+imagenet-norm
2 [runner] input shape: [1, 3, 224, 224]
3 [runner] input stats: min=-2.100779 max=+2.640000 mean=+0.017939 nonzero=150528/150528 nans=0 infs=0
4 Gemmini extension configured with:
5   dim = 16
6 [gemmini-trace] conv f32: x=[1,3,224,224] w=[64,3,7,7] y=[1,64,112,112] stride=2 pad=3 bias=no
7 [gemmini-trace] conv input: min=-2.100779 max=+2.640000 mean=+0.017939 nonzero=150528/150528 nans=0 infs=0
8 [gemmini-trace] conv weights: min=-0.882005 max=+1.176508 mean=-0.000933 nonzero=9408/9408 nans=0 infs=0
9 [gemmini-trace] conv output: min=-23.801222 max=+27.967497 mean=-0.013168 nonzero=802816/802816 nans=0
  infs=0
10 ...
11 [gemmini-trace] gemm A: min=+0.000000 max=+4.550326 mean=+0.780484 nonzero=511/512 nans=0 infs=0
12 [gemmini-trace] gemm B: min=-0.360843 max=+0.683930 mean=+0.000000 nonzero=512000/512000 nans=0 infs=0
13 [gemmini-trace] gemm C/bias: min=-0.060659 max=+0.052528 mean=+0.000000 nonzero=1000/1000 nans=0 infs=0
14 [gemmini-trace] gemm output: min=-5.429282 max=+13.784193 mean=+0.000118 nonzero=1000/1000 nans=0 infs=0
15 [runner] output shape: [1, 1000] elems=1000
16 [runner] output sum=0.117750 nans=0 infs=0
17 Top-5 outputs:
18   [1] index=258 score=+13.784193
19   [2] index=257 score=+9.617604
20   [3] index=279 score=+9.542806
21   [4] index=270 score=+9.194843
22   [5] index=248 score=+8.888422
23 [runner] PASS -- model produced output

```

Listing 42: Compact real-image Gemmini trace evidence

12 Line-by-Line Explanation of the Spike Trace Run

This section explains the exact Spike command and the important output lines from the traced ResNet-18 run. The goal is to remove ambiguity about which part is handled by the shell, which part is handled by Spike, which part is handled by the proxy kernel, and which part is handled by the compiled runner.

```

1 cd $ROOT_DIR
2
3 GEMMINI_TRACE=1 stdbuf -oL -eL $SPIKE \
4   --extension=gemmini \
5   --isa=rv64imafdc \
6   -m2048 \
7   $PK \
8   $OUT_DIR/resnet18_rv64 \
9   --image $OUT_DIR/data/dog_224.ppm \
10  --in-n 1 --in-c 3 --in-h 224 --in-w 224 \
11  2>&1 | tee $OUT_DIR/spike_trace.log

```

Listing 43: Full traced Spike command

12.1 Who Receives Each Part of the Command?

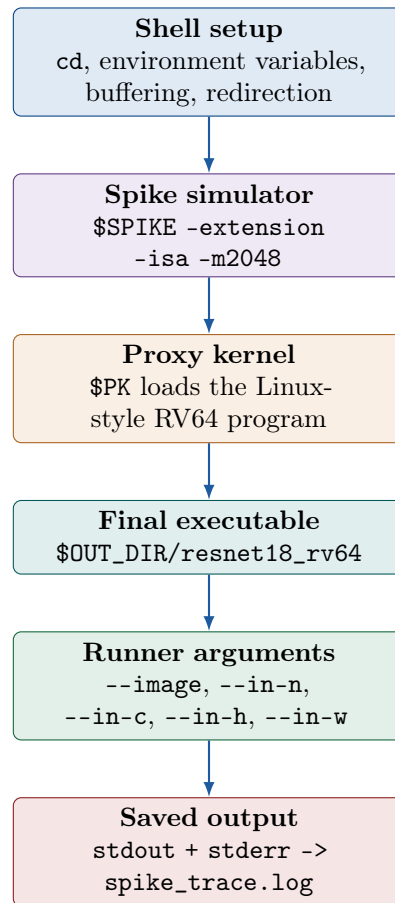


Figure 15: Ownership of the command line. Not every argument is for Spike: arguments after `resnet18_rv64` are received by the runner program.

12.2 Command Lines Explained

Line / argument	Owner	Meaning
<code>cd \$ROOT_DIR</code>	Shell	Moves to the ONNX-MLIR repository root. This makes relative paths such as <code>gemmini/tests/...</code> resolve correctly.
<code>GEMMINI_TRACE=1</code>	Environment variable	Enables trace printing inside the Gemmini runtime. The runtime checks this variable before printing <code>[gemmini-trace]</code> messages.
<code>stdbuf -oL -eL</code>	Shell utility	Forces line-buffered stdout and stderr. Without it, logs may appear late or in large chunks while Spike is running.
<code>\$SPIKE</code>	Spike executable	Runs the RISC-V ISA simulator. In this project it points to the Gemmini-enabled Spike binary.
<code>--extension=gemmini</code>	Spike option	Loads the Gemmini extension into Spike so RoCC/custom accelerator instructions are recognized instead of being treated as illegal instructions.
<code>--isa=rv64imafdc</code>	Spike option	Selects the simulated RISC-V ISA: 64-bit base integer plus integer multiply/divide, atomic, single-precision floating point, double-precision floating point, and compressed instructions.
<code>-m2048</code>	Spike memory option	Gives the simulated machine a large memory region, here 2048 MB. This is useful because the static executable contains model weights and needs enough memory for tensors and temporary buffers.
<code>\$PK</code>	Proxy kernel	Runs the RISC-V proxy kernel. Spike simulates the CPU, while <code>pk</code> loads and starts the user program <code>resnet18_rv64</code> .
<code>\$OUT_DIR/resnet18_rv64</code>	Program executed by <code>pk</code>	The final statically linked RV64 executable. It contains the runner, the compiled ResNet-18 object, model constants, and runtime libraries.
<code>--image ../dog_224.ppm</code>	Runner argument	Tells <code>float_model_runner.c</code> to load a real image instead of generating deterministic dummy input.
<code>--in-n 1</code>	Runner argument	Sets the batch dimension to 1. The input tensor starts with <code>N=1</code> .
<code>--in-c 3</code>	Runner argument	Sets the number of input channels to 3: red, green, and blue.
<code>--in-h 224</code>	Runner argument	Sets the input height to 224 pixels.
<code>--in-w 224</code>	Runner argument	Sets the input width to 224 pixels. Together, the runner creates an input tensor of shape <code>[1,3,224,224]</code> .
<code>2>&1</code>	Shell redirection	Merges stderr into stdout. This is important because runtime trace messages are usually printed to stderr.
<code>tee \$OUT_DIR/spike_trace.log</code>	Shell pipe	Displays the run output on the terminal and writes the same output to <code>spike_trace.log</code> for later debugging.

Table 40: Line-by-line explanation of the traced Spike command. Visible row borders are used to avoid mixing command ownership.

Important separation

Everything before `$OUT_DIR/resnet18_rv64` configures the shell, Spike, or the proxy kernel. Everything after `resnet18_rv64` is passed as `argv` to the runner program. This is why `--image`, `--in-n`, `--in-c`, `--in-h`, and `--in-w` are not Spike options.

12.3 Output Lines Explained

```

1 [runner] image input: ../data/dog_224.ppm  source=[224,224,3]  crop=224  preprocess=center-crop+bilinear+
   imagenet-norm
2 [runner] input  shape: [1, 3, 224, 224]
```

```

3 [runner] input stats: min=-2.117904 max=+2.640000 mean=-0.233865 nonzero=150528/150528 nans=0 infs=0
4 [gemmini-trace] conv f32: x=[1,3,224,224] w=[64,3,7,7] y=[1,64,112,112] stride=2 pad=3 bias=no
5 [gemmini-trace] conv input: min=-2.117904 max=+2.640000 mean=-0.233865 nonzero=150528/150528 nans=0 infs=0
6 [gemmini-trace] conv weights: min=-0.882005 max=+1.176508 mean=-0.000933 nonzero=9408/9408 nans=0 infs=0
7 Gemmini extension configured with:
8   dim = 16
9 [gemmini-trace] conv output: min=-24.287453 max=+27.469229 mean=+0.017834 nonzero=802816/802816 nans=0
   infs=0
10 ...
11 [gemmini-trace] gemm A: min=+0.000000 max=+4.017405 mean=+0.752001 nonzero=509/512 nans=0 infs=0
12 [gemmini-trace] gemm B: min=-0.360843 max=+0.683930 mean=+0.000000 nonzero=512000/512000 nans=0 infs=0
13 [gemmini-trace] gemm C/bias: min=-0.060659 max=+0.052528 mean=+0.000000 nonzero=1000/1000 nans=0 infs=0
14 [gemmini-trace] gemm output: min=-5.362539 max=+12.577187 mean=+0.000113 nonzero=1000/1000 nans=0 infs=0
15 [runner] output shape: [1, 1000] elems=1000
16 [runner] output sum=0.112664 nans=0 infs=0
17 Top-5 outputs:
18   [1] index=258      score=+12.577187
19   [2] index=279      score=+9.349453
20   [3] index=257      score=+8.558382
21   [4] index=270      score=+8.331315
22   [5] index=248      score=+8.139629
23 [runner] PASS -- model produced output

```

Listing 44: Compact output from the dog_224 traced run

Output line	Produced by	How to read it
[runner] image input: ... dog_224.ppm	Runner image loader	The executable received the --image argument, opened the PPM file, recognized the source image as [224,224,3], and prepared it for the network.
preprocess=...imagenet-norm	Runner preprocessing	The image path is not fed directly as bytes. It is center-cropped, resized/interpolated if needed, then normalized using ImageNet-style channel statistics.
[runner] input shape: [1,3,224,224]	OMTensor creation	The runner created one NCHW tensor: batch 1, channels 3, height 224, width 224. This matches the compiled ResNet-18 input shape.
input stats: min=... max=... mean=...	Runner validation	The runner checks the input tensor before inference. nonzero=150528/150528 means all 1*3*224*224 values are non-zero after normalization. nans=0 infs=0 means the tensor is numerically valid.
[gemmini-trace] conv f32: x=... w=... y=...	Gemmini runtime	A convolution runtime function was entered. x is the input tensor, w is the weight tensor, and y is the output tensor. This line proves that the compiled model called the Gemmini runtime path.
x=[1,3,224,224]	Conv shape field	Input tensor in NCHW layout: 1 image, 3 channels, 224 rows, 224 columns.
w=[64,3,7,7]	Conv weight shape	First ResNet convolution has 64 output channels, 3 input channels, and a 7x7 kernel. The number of weights is $64*3*7*7 = 9408$.
y=[1,64,112,112]	Conv output shape	The first convolution produces 64 feature maps at 112x112 resolution. The spatial size is reduced because stride=2.
stride=2 pad=3 bias=no	Conv attributes	The kernel moves two pixels at a time. Three pixels of zero-padding are added around the image. bias=no means this convolution call has no separate bias tensor.
conv input: min=... max=...	Runtime tensor stats	Statistics of the input buffer used by this convolution. This helps detect bad preprocessing, NaNs, infinities, or all-zero tensors.
conv weights: nonzero=9408/9408	Runtime weight stats	Statistics of the convolution weights. Here all 9408 weights are non-zero, so the constants were loaded into the executable and accessed correctly.
Gemmini extension configured: dim=16	Spike Gemmini extension	Spike loaded the Gemmini extension. dim=16 means the simulated Gemmini systolic array dimension is 16 by 16.
conv output: nonzero=802816/802816	Runtime output stats	The first convolution produced all $1*64*112*112 = 802816$ output values. nans=0 infs=0 confirms the layer output is finite.
Repeated conv f32 blocks	ResNet-18 convolution stages	Each later block has the same meaning. The shapes show ResNet-18 moving from 56x56 to 28x28 to 14x14 to 7x7 while increasing channels from 64 to 128 to 256 to 512.

Table 41: Line-by-line explanation of the runner and convolution trace output.

Output line	Produced by	How to read it
[gemmini-trace] gemm A	Final classifier input	A is the 512-element feature vector after global average pooling. nonzero=509/512 means nearly all feature values are active.
[gemmini-trace] gemm B	Fully connected weights	B is the classifier weight matrix. 512000/512000 equals 512 input features times 1000 output classes.
[gemmini-trace] gemm C/bias	Classifier bias	C/bias is the 1000-element bias vector added to the classifier output.
[gemmini-trace] gemm output	Classifier output	This is the final GEMM result before the runner prints the output summary. Since the later sum is not 1, these values should be read as raw scores/logits in this run, not normalized probabilities.
[runner] output shape: [1,1000]	Runner output check	The model returned one vector with 1000 entries, matching ImageNet-style classification output.
output sum=0.112664 nans=0 infs=0	Runner numeric check	The output is finite. Because the sum is not 1, do not interpret this specific traced output as a softmax probability distribution.
Top-5 outputs	Runner summary	The runner sorts the 1000 output scores and prints the five highest class indices.
PASS - model produced output	Runner final status	The executable completed inference and returned a valid output tensor. This validates the end-to-end path: image loader -> OMTensor -> compiled model -> Gemmini runtime -> output.

Table 42: Line-by-line explanation of the final GEMM and output summary lines.

12.4 Visual Reading of the Trace

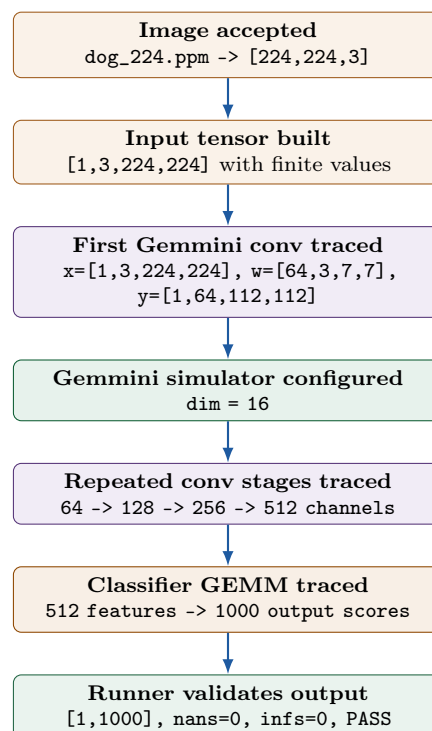


Figure 16: How to read the trace as a sequence of evidence checks.

Final interpretation of this run

This command proves that the final executable can run under Gemmini-enabled Spike, load a real image, create the expected ResNet-18 input tensor, enter the Gemmini runtime for convolution and GEMM operations, and produce a finite `[1,1000]` output tensor. The trace does not by itself prove classification accuracy; it proves that the compiled model and runtime execution path are functioning end-to-end.

13 High-Quality ResNet-18 ONNX Graph and Complete Trace Run

Why this section was added

This section connects three things that are often confused:

- the **visual ONNX graph** exported from the model viewer;
- the **Spike command** that runs the final RISC-V executable;
- the **Gemmini trace log** proving that convolution and GEMM runtime calls are actually reached.

The original ONNX viewer graph is very tall and hard to read in a report. Instead of relying on the screenshot, this section uses clean vector diagrams drawn directly in LaTeX/TikZ so the ResNet-18 execution structure, residual paths, and tensor shapes remain readable in Overleaf and in the exported PDF.

13.1 Readable Reconstruction of the ResNet-18 Execution Graph

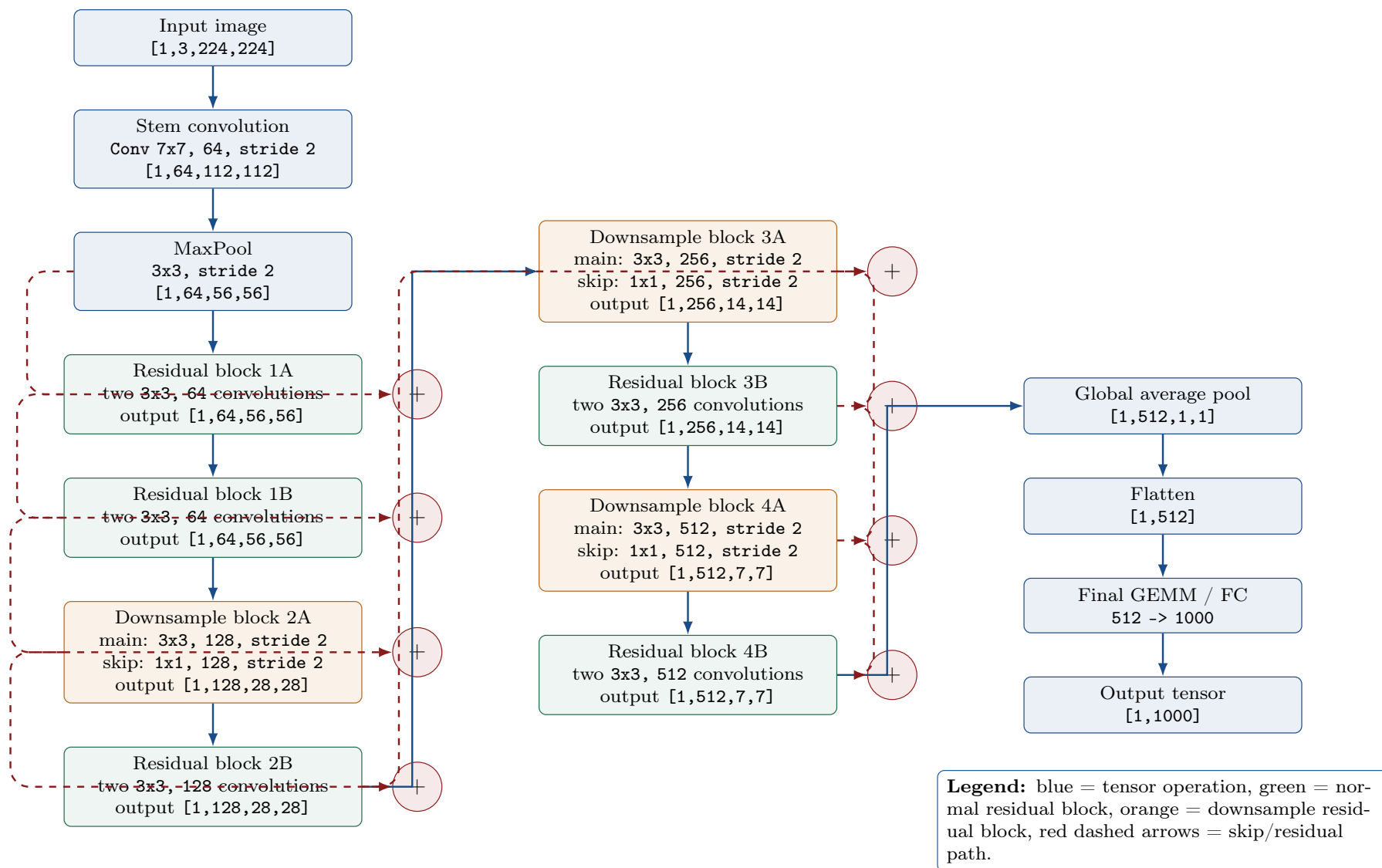


Figure 17: High-quality readable reconstruction of the ResNet-18 ONNX graph. This is a clean vector version designed directly for this document.

13.2 Designed Vector Graph Instead of the Uploaded Screenshot

Why the screenshot was replaced

The original ONNX graph screenshot is very tall and narrow. Even when it is split into panels, the node text remains difficult to read. For documentation, the clearest solution is to redraw the graph as a clean **vector diagram** directly in LaTeX/TikZ. This section therefore replaces the unclear screenshot with designed diagrams that show the same ResNet-18 execution structure: stem, residual stages, downsampling blocks, skip connections, global average pooling, and final GEMM.

How to read the designed graph

Each blue box is a normal tensor operation. Each green box is a normal residual block. Each orange box is a downsampling residual block, meaning the spatial resolution is divided by two and the number of channels increases. Red dashed arrows are skip/residual paths. The red $+$ node is tensor addition between the main branch and the skip branch.

13.2.1 Clean Stage-Level Design

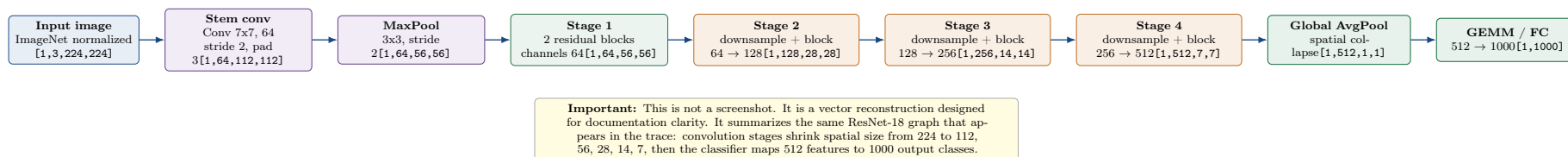


Figure 18: Clean designed ResNet-18 stage-level graph.

13.2.2 Residual Block Design: Normal Block vs Downsample Block

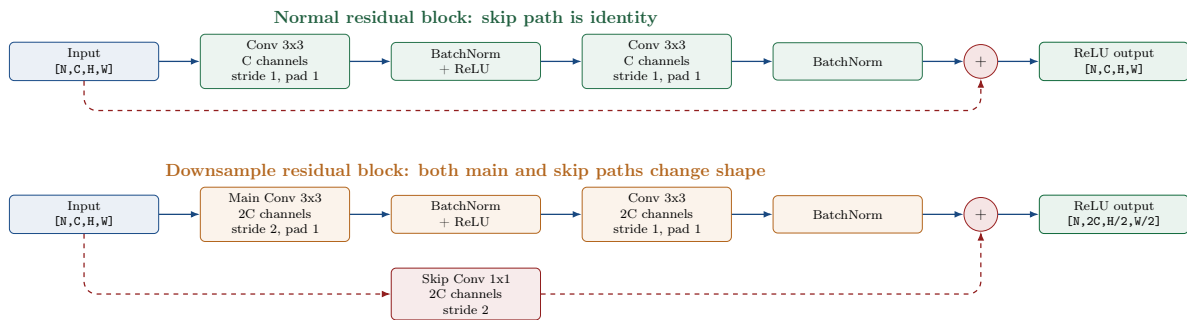


Figure 19: Designed explanation of ResNet residual blocks. The downsample block is used when spatial size decreases and channel count increases.

13.2.3 Detailed Designed Graph by ResNet-18 Stage

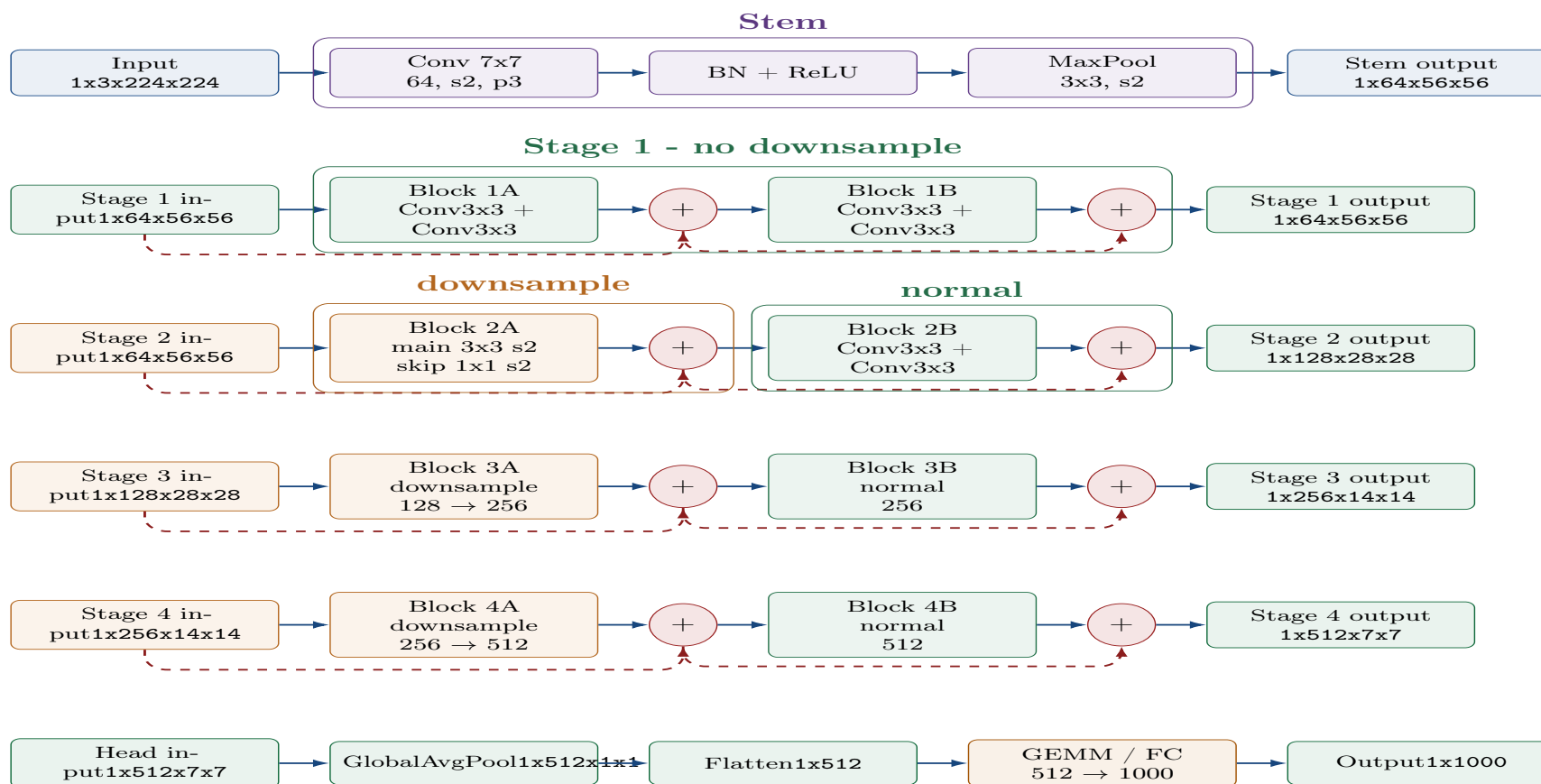


Figure 20: Designed detailed ResNet-18 graph. This replaces the unclear uploaded graph screenshot with readable vector stages and residual paths.

13.2.4 Layer Sequence Table with Clear Separators

Graph part	Main computation	Output shape	Meaning in the trace
Stem	7x7 convolution, batch normalization, ReLU, max pooling	[1,64,56,56] after pooling	The first trace line shows $x=[1,3,224,224]$, $w=[64,3,7,7]$, and $y=[1,64,112,112]$. The following max pool reduces 112 to 56.
Stage 1	Two normal residual blocks at 64 channels	[1,64,56,56]	Spatial size and channel count stay constant. Trace lines repeat $x=[1,64,56,56]$ and $w=[64,64,3,3]$.
Stage 2	One downsample residual block plus one normal block	[1,128,28,28]	The downsample block contains a main convolution and a skip 1x1 convolution. Trace includes $w=[128,64,1,1]$ and $w=[128,64,3,3]$.
Stage 3	One downsample residual block plus one normal block	[1,256,14,14]	The trace shows transition from 128 channels to 256 channels and spatial size from 28 to 14.
Stage 4	One downsample residual block plus one normal block	[1,512,7,7]	The trace shows transition from 256 channels to 512 channels and spatial size from 14 to 7.
Classifier head	Global average pool, flatten, final GEMM	[1,1000]	The trace prints <code>gemm A, gemm B, gemm C/bias,</code> and <code>gemm output</code> ; the runner then prints <code>output shape: [1,1000]</code> .

Table 43: Readable designed graph summary with explicit separators between rows.

Why the designed graph is better than the screenshot

The vector design is resolution independent: it remains sharp in Overleaf, in the compiled PDF, and when zoomed. It also removes unnecessary viewer details and focuses on the information needed for this project: tensor shapes, residual paths, downsampling points, and the exact places where the Gemmini trace lines appear.

13.3 Exact Spike Command Used for the Full Trace

```
1 cd $ROOT_DIR
```

```

2 GEMMINI_TRACE=1 stdbuf -oL -eL $SPIKE \
3   --extension=gemmini \
4   --isa=rv64imafdc \
5   -m2048 \
6   $PK \
7   $OUT_DIR/resnet18_rv64 \
8   --image $OUT_DIR/data/dog_224.ppm \
9   --in-n 1 --in-c 3 --in-h 224 --in-w 224 \
10  2>&1 | tee $OUT_DIR/spike_trace.log
11

```

Listing 45: Exact line-buffered Gemmini trace command

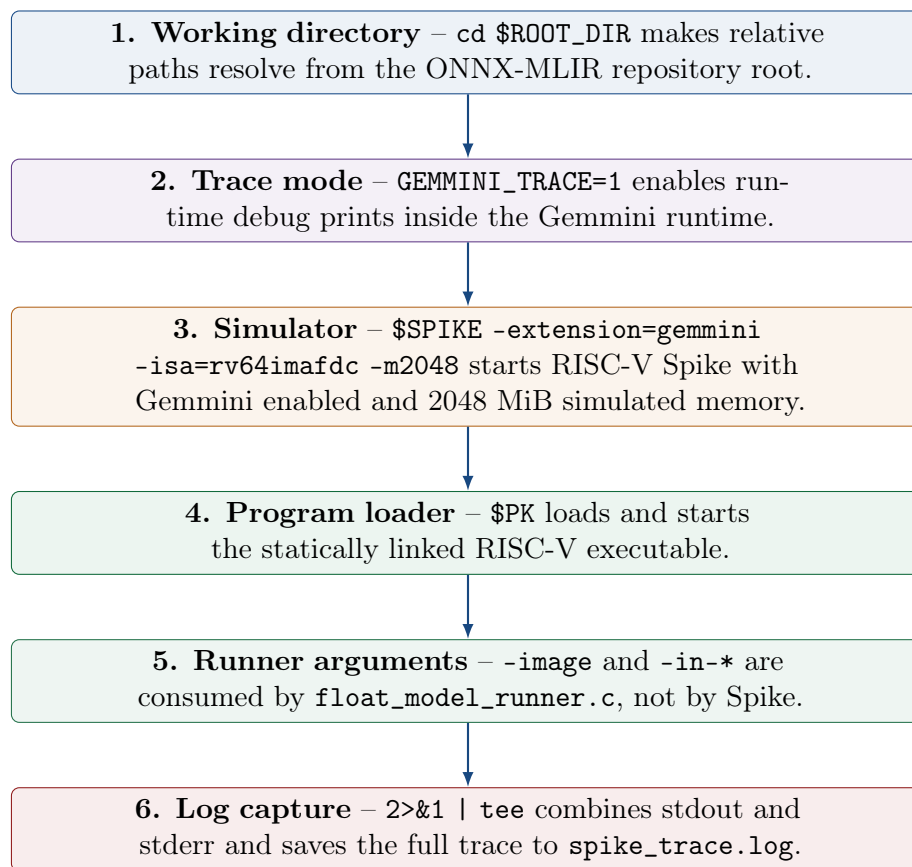


Figure 21: Ownership of every part of the Spike trace command.

13.4 Complete Observed Trace Log

How to read this long log

The repeated `conv f32` blocks are expected for ResNet-18. Each block prints the input tensor shape, weight tensor shape, output tensor shape, stride, padding, and basic sanity statistics. The important validation signs are: no NaN values, no Inf values, Gemmini extension configured with `dim = 16`, GEMM reached near the classifier, and the runner finished with `PASS`.

```

1 [runner] image input: /home/lahcen/toolchains/onnx-mlir-xcompile/onnx-mlir/gemmini/tests
  /15-resnet18-float/data/dog_224.ppm source=[224,224,3] crop=224 preprocess=center
  -crop+bilinear+imagenet-norm
2 [runner] input shape: [1, 3, 224, 224]
3 [runner] input stats: min=-2.117904 max=+2.640000 mean=-0.233865 nonzero=150528/150528
  nans=0 infs=0
4 [gemmini-trace] conv f32: x=[1,3,224,224] w=[64,3,7,7] y=[1,64,112,112] stride=2 pad=3
  bias=no
5 [gemmini-trace] conv input: min=-2.117904 max=+2.640000 mean=-0.233865 nonzero
  =150528/150528 nans=0 infs=0
6 [gemmini-trace] conv weights: min=-0.882005 max=+1.176508 mean=-0.000933 nonzero
  =9408/9408 nans=0 infs=0
7 Gemmini extension configured with:
8   dim = 16
9 [gemmini-trace] conv output: min=-24.287453 max=+27.469229 mean=+0.017834 nonzero
  =802816/802816 nans=0 infs=0
10 [gemmini-trace] conv f32: x=[1,64,56,56] w=[64,64,3,3] y=[1,64,56,56] stride=1 pad=1
  bias=no
11 [gemmini-trace] conv input: min=+0.000000 max=+2.553853 mean=+0.310484 nonzero
  =165448/200704 nans=0 infs=0
12 [gemmini-trace] conv weights: min=-0.683258 max=+0.750469 mean=-0.003034 nonzero
  =36864/36864 nans=0 infs=0
13 [gemmini-trace] conv output: min=-12.074734 max=+4.074152 mean=-0.717864 nonzero
  =200704/200704 nans=0 infs=0
14 [gemmini-trace] conv f32: x=[1,64,56,56] w=[64,64,3,3] y=[1,64,56,56] stride=1 pad=1
  bias=no
15 [gemmini-trace] conv input: min=+0.000000 max=+1.837297 mean=+0.111142 nonzero
  =125203/200704 nans=0 infs=0
16 [gemmini-trace] conv weights: min=-0.472497 max=+0.416569 mean=-0.001082 nonzero
  =36864/36864 nans=0 infs=0
17 [gemmini-trace] conv output: min=-2.825184 max=+1.867851 mean=-0.077735 nonzero
  =200704/200704 nans=0 infs=0
18 [gemmini-trace] conv f32: x=[1,64,56,56] w=[64,64,3,3] y=[1,64,56,56] stride=1 pad=1
  bias=no
19 [gemmini-trace] conv input: min=+0.000000 max=+3.496845 mean=+0.404454 nonzero
  =159230/200704 nans=0 infs=0
20 [gemmini-trace] conv weights: min=-0.367752 max=+0.324016 mean=-0.003403 nonzero
  =36864/36864 nans=0 infs=0
21 [gemmini-trace] conv output: min=-7.378168 max=+3.194499 mean=-0.961425 nonzero
  =200704/200704 nans=0 infs=0
22 [gemmini-trace] conv f32: x=[1,64,56,56] w=[64,64,3,3] y=[1,64,56,56] stride=1 pad=1
  bias=no
23 [gemmini-trace] conv input: min=+0.000000 max=+1.551476 mean=+0.069549 nonzero
  =87955/200704 nans=0 infs=0
24 [gemmini-trace] conv weights: min=-0.322117 max=+0.294465 mean=-0.001007 nonzero
  =36864/36864 nans=0 infs=0
25 [gemmini-trace] conv output: min=-1.331716 max=+0.925030 mean=-0.018186 nonzero
  =200704/200704 nans=0 infs=0
26 [gemmini-trace] conv f32: x=[1,64,56,56] w=[128,64,1,1] y=[1,128,28,28] stride=2 pad=0
  bias=no
27 [gemmini-trace] conv input: min=+0.000000 max=+4.328637 mean=+0.477502 nonzero
  =159703/200704 nans=0 infs=0
28 [gemmini-trace] conv weights: min=-0.708435 max=+0.515147 mean=-0.002705 nonzero
  =8192/8192 nans=0 infs=0
29 [gemmini-trace] conv output: min=-1.961411 max=+2.112203 mean=-0.062188 nonzero
  =100352/100352 nans=0 infs=0
30 [gemmini-trace] conv f32: x=[1,64,56,56] w=[128,64,3,3] y=[1,128,28,28] stride=2 pad=1
  bias=no
31 [gemmini-trace] conv input: min=+0.000000 max=+4.328637 mean=+0.477502 nonzero
  =159703/200704 nans=0 infs=0
32 [gemmini-trace] conv weights: min=-0.344376 max=+0.366623 mean=-0.001721 nonzero
  =73728/73728 nans=0 infs=0
33 [gemmini-trace] conv output: min=-5.550218 max=+4.244257 mean=-0.450897 nonzero
  =100352/100352 nans=0 infs=0
34 [gemmini-trace] conv f32: x=[1,128,28,28] w=[128,128,3,3] y=[1,128,28,28] stride=1 pad=1
  bias=no
35 [gemmini-trace] conv input: min=+0.000000 max=+1.594046 mean=+0.094012 nonzero
  =46476/100352 nans=0 infs=0
36 [gemmini-trace] conv weights: min=-0.368633 max=+0.396001 mean=-0.001145 nonzero
  =147456/147456 nans=0 infs=0
37 [gemmini-trace] conv output: min=-2.230842 max=+1.569665 mean=-0.147251 nonzero
  =100352/100352 nans=0 infs=0
38 [gemmini-trace] conv f32: x=[1,128,28,28] w=[128,128,3,3] y=[1,128,28,28] stride=1 pad=1

```

```

    bias=no
39 [gemmini-trace] conv input: min=+0.000000 max=+3.385663 mean=+0.157456 nonzero
    =53871/100352 nans=0 infs=0
40 [gemmini-trace] conv weights: min=-0.328973 max=+0.449360 mean=-0.001594 nonzero
    =147456/147456 nans=0 infs=0
41 [gemmini-trace] conv output: min=-2.832562 max=+1.986084 mean=-0.217792 nonzero
    =100352/100352 nans=0 infs=0
42 [gemmini-trace] conv f32: x=[1,128,28,28] w=[128,128,3,3] y=[1,128,28,28] stride=1 pad=1
    bias=no
43 [gemmini-trace] conv input: min=+0.000000 max=+2.048266 mean=+0.059467 nonzero
    =33042/100352 nans=0 infs=0
44 [gemmini-trace] conv weights: min=-0.415963 max=+0.235407 mean=-0.001306 nonzero
    =147456/147456 nans=0 infs=0
45 [gemmini-trace] conv output: min=-1.684076 max=+1.463893 mean=-0.084940 nonzero
    =100352/100352 nans=0 infs=0
46 [gemmini-trace] conv f32: x=[1,128,28,28] w=[256,128,1,1] y=[1,256,14,14] stride=2 pad=0
    bias=no
47 [gemmini-trace] conv input: min=+0.000000 max=+3.405058 mean=+0.191331 nonzero
    =56382/100352 nans=0 infs=0
48 [gemmini-trace] conv weights: min=-0.393573 max=+0.509730 mean=-0.001319 nonzero
    =32768/32768 nans=0 infs=0
49 [gemmini-trace] conv output: min=-0.931247 max=+0.694016 mean=-0.027990 nonzero
    =50176/50176 nans=0 infs=0
50 [gemmini-trace] conv f32: x=[1,128,28,28] w=[256,128,3,3] y=[1,256,14,14] stride=2 pad=1
    bias=no
51 [gemmini-trace] conv input: min=+0.000000 max=+3.405058 mean=+0.191331 nonzero
    =56382/100352 nans=0 infs=0
52 [gemmini-trace] conv weights: min=-0.266405 max=+0.333219 mean=-0.001138 nonzero
    =294912/294912 nans=0 infs=0
53 [gemmini-trace] conv output: min=-2.843448 max=+2.347441 mean=-0.265852 nonzero
    =50176/50176 nans=0 infs=0
54 [gemmini-trace] conv f32: x=[1,256,14,14] w=[256,256,3,3] y=[1,256,14,14] stride=1 pad=1
    bias=no
55 [gemmini-trace] conv input: min=+0.000000 max=+1.978851 mean=+0.070506 nonzero
    =17589/50176 nans=0 infs=0
56 [gemmini-trace] conv weights: min=-0.388776 max=+0.416401 mean=-0.001003 nonzero
    =589824/589824 nans=0 infs=0
57 [gemmini-trace] conv output: min=-2.135766 max=+1.634936 mean=-0.164989 nonzero
    =50176/50176 nans=0 infs=0
58 [gemmini-trace] conv f32: x=[1,256,14,14] w=[256,256,3,3] y=[1,256,14,14] stride=1 pad=1
    bias=no
59 [gemmini-trace] conv input: min=+0.000000 max=+2.623990 mean=+0.109721 nonzero
    =21837/50176 nans=0 infs=0
60 [gemmini-trace] conv weights: min=-0.287183 max=+0.276043 mean=-0.001662 nonzero
    =589824/589824 nans=0 infs=0
61 [gemmini-trace] conv output: min=-3.011958 max=+2.673899 mean=-0.356053 nonzero
    =50176/50176 nans=0 infs=0
62 [gemmini-trace] conv f32: x=[1,256,14,14] w=[256,256,3,3] y=[1,256,14,14] stride=1 pad=1
    bias=no
63 [gemmini-trace] conv input: min=+0.000000 max=+2.915019 mean=+0.039143 nonzero
    =11493/50176 nans=0 infs=0
64 [gemmini-trace] conv weights: min=-0.236961 max=+0.239447 mean=-0.001321 nonzero
    =589824/589824 nans=0 infs=0
65 [gemmini-trace] conv output: min=-1.935967 max=+1.219322 mean=-0.144140 nonzero
    =50176/50176 nans=0 infs=0
66 [gemmini-trace] conv f32: x=[1,256,14,14] w=[512,256,1,1] y=[1,512,7,7] stride=2 pad=0
    bias=no
67 [gemmini-trace] conv input: min=+0.000000 max=+3.845144 mean=+0.102257 nonzero
    =18508/50176 nans=0 infs=0
68 [gemmini-trace] conv weights: min=-0.359990 max=+0.833424 mean=-0.000990 nonzero
    =131072/131072 nans=0 infs=0
69 [gemmini-trace] conv output: min=-0.807316 max=+0.887000 mean=-0.043932 nonzero
    =25088/25088 nans=0 infs=0
70 [gemmini-trace] conv f32: x=[1,256,14,14] w=[512,256,3,3] y=[1,512,7,7] stride=2 pad=1
    bias=no
71 [gemmini-trace] conv input: min=+0.000000 max=+3.845144 mean=+0.102257 nonzero
    =18508/50176 nans=0 infs=0
72 [gemmini-trace] conv weights: min=-0.204572 max=+0.263028 mean=-0.001488 nonzero
    =1179648/1179648 nans=0 infs=0
73 [gemmini-trace] conv output: min=-3.350232 max=+2.886567 mean=-0.408650 nonzero
    =25088/25088 nans=0 infs=0
74 [gemmini-trace] conv f32: x=[1,512,7,7] w=[512,512,3,3] y=[1,512,7,7] stride=1 pad=1
    bias=no

```

```

75 [gemmini-trace] conv input: min=+0.000000 max=+2.285467 mean=+0.027602 nonzero
    =4513/25088 nans=0 infs=0
76 [gemmini-trace] conv weights: min=-0.263507 max=+0.334286 mean=-0.001131 nonzero
    =2359296/2359296 nans=0 infs=0
77 [gemmini-trace] conv output: min=-1.425189 max=+1.929627 mean=-0.151702 nonzero
    =25088/25088 nans=0 infs=0
78 [gemmini-trace] conv f32: x=[1,512,7,7] w=[512,512,3,3] y=[1,512,7,7] stride=1 pad=1
    bias=no
79 [gemmini-trace] conv input: min=+0.000000 max=+5.616985 mean=+0.067996 nonzero
    =4923/25088 nans=0 infs=0
80 [gemmini-trace] conv weights: min=-0.188742 max=+0.214452 mean=-0.001986 nonzero
    =2359296/2359296 nans=0 infs=0
81 [gemmini-trace] conv output: min=-3.326899 max=+1.501647 mean=-0.693045 nonzero
    =25088/25088 nans=0 infs=0
82 [gemmini-trace] conv f32: x=[1,512,7,7] w=[512,512,3,3] y=[1,512,7,7] stride=1 pad=1
    bias=no
83 [gemmini-trace] conv input: min=+0.000000 max=+1.202578 mean=+0.018437 nonzero
    =3621/25088 nans=0 infs=0
84 [gemmini-trace] conv weights: min=-0.092203 max=+0.243898 mean=-0.000106 nonzero
    =2359296/2359296 nans=0 infs=0
85 [gemmini-trace] conv output: min=-0.387411 max=+0.507000 mean=-0.021597 nonzero
    =25088/25088 nans=0 infs=0
86 [gemmini-trace] gemm A: min=+0.000000 max=+4.017405 mean=+0.752001 nonzero=509/512 nans
    =0 infs=0
87 [gemmini-trace] gemm B: min=-0.360843 max=+0.683930 mean=+0.000000 nonzero=512000/512000
    nans=0 infs=0
88 [gemmini-trace] gemm C/bias: min=-0.060659 max=+0.052528 mean=+0.000000 nonzero
    =1000/1000 nans=0 infs=0
89 [gemmini-trace] gemm output: min=-5.362539 max=+12.577187 mean=+0.000113 nonzero
    =1000/1000 nans=0 infs=0
90 [runner] output shape: [1, 1000] elems=1000
91 [runner] output sum=0.112664 nans=0 infs=0
92 Top-5 outputs:
93 [1] index=258 score=+12.577187
94 [2] index=279 score=+9.349453
95 [3] index=257 score=+8.558382
96 [4] index=270 score=+8.331315
97 [5] index=248 score=+8.139629
98 [runner] PASS -- model produced output

```

Listing 46: Full observed Spike/Gemmini trace log for dog_224.ppm

Log evidence	Where it appears	Meaning
Image loaded	image input: ... dog_-224.ppm	The runner did not use random data; it read the real PPM image file.
Input tensor correct	input shape: [1, 3, 224, 224]	The runner constructed the NCHW tensor expected by ResNet-18.
First Gemmini convolution	x=[1,3,224,224] w=[64,3,7,7]	The first model convolution reached the Gemmini runtime trace path.
Gemmini extension active	dim = 16	Spike loaded the Gemmini extension and reports a 16-by-16 array.
Classifier reached	gemm A, gemm B, gemm output	The final fully connected/GEMM part of ResNet-18 executed.
No invalid numbers	nans=0 infs=0	The execution produced finite values at the checked points.
End-to-end pass	PASS - model produced output	The runner received an output tensor and completed successfully.

Table 44: Separated validation table for the full trace log.

14 Source Code Reference: GitHub Gemmini Accelerator Folder

The GitHub folder below is the source-code side of the pipeline explained in this document:

Repository path:
<https://github.com/attidev01/onnx-mlir/tree/main/src/Accelerators/Gemmini>

Locally, this corresponds to:

```
1 cd $ROOT_DIR/src/Accelerators/Gemmini
2
3 tree -L 3
```

Listing 47: Local source folder corresponding to the GitHub URL

Do not confuse source code with generated files

The folder `src/Accelerators/Gemmini` contains the **compiler extension and run-time source code**. It does not contain the final ResNet-18 executable. The generated files are produced later in `gemmini/tests/15-resnet18-float`, for example `resnet18.o`, `libgemmini_f32_rv64.a`, and `resnet18_rv64`.

14.1 Designed Directory Map

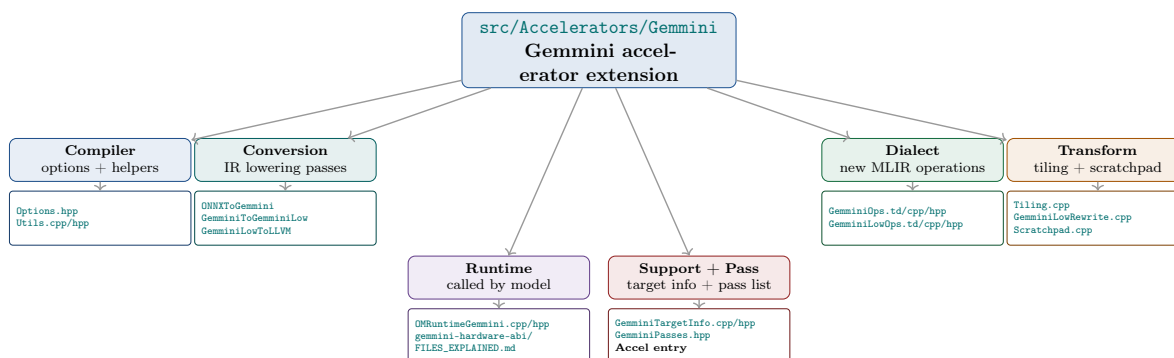


Figure 22: Designed source-code map for the GitHub Gemmini accelerator directory.

14.2 How the Source Folder Connects to the Pipeline

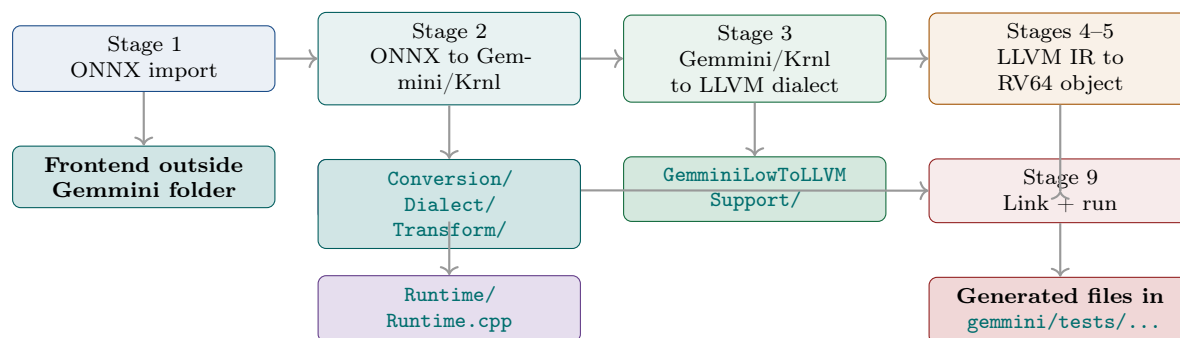


Figure 23: Where each GitHub source directory participates in the ONNX-to-RISC-V pipeline.

14.3 Folder-by-Folder Explanation

The table below is intentionally concise. Long file paths are split into short folder names so that the table remains readable in PDF and Overleaf.

Folder or file	Main responsibility	Pipeline stage	Plain-English meaning
Accel entry	Accelerator registration and pass orchestration	Driver integration	Connects <code>-maccel=Gemmini</code> to the Gemmini-specific compiler pipeline.
Compiler/	Gemmini options and helpers	Stage 2 setup	Stores option handling and shared helper logic used by Gemmini passes.
ONNX to Gemmini	ONNX operator selection	Stage 2	Decides whether ONNX ops such as Conv, Gemm, ReLU, pooling, and Add should go to Gemmini.
Gemmini dialect	High-level Gemmini MLIR ops	Stage 2 IR	Defines the compiler operations used to represent accelerator-friendly work.
Gemmini transform	Tiling and accelerator rewrites	Stage 2 optimization	Splits large operations into pieces suitable for Gemmini local-memory limits.
GemminiLow dialect	Lower-level Gemmini MLIR ops	Stages 2–3	Represents accelerator actions closer to the hardware interface.

Folder or file	Main responsibility	Pipeline stage	Plain-English meaning
GemminiLow transform	Low-level rewrites and scratchpad placement	Stages 2–3	Chooses or rewrites low-level buffer placement, including static scratchpad allocation.
To GemminiLow	High-level to low-level Gemmini lowering	Stage 3 prep	Converts abstract Gemmini ops into more explicit GemminiLow ops.
LowToLLVM	GemminiLow to LLVM-compatible IR	Stage 3	Lowers low-level accelerator operations toward LLVM dialect form.
Runtime	Runtime function implementation	Stage 7 and execution	Implements functions called by the model, for example <code>om_gemmini_conv_f32</code> .
Hardware ABI	Hardware/software ABI headers	Stage 7 and execution	Provides RoCC/Gemmini helper declarations, constants, and hardware-facing macros.
Support/	Target information	Compiler configuration	Centralizes Gemmini target assumptions used by the compiler.
Passes	Pass declarations	Compiler integration	Declares Gemmini passes so they can be registered and inserted into the pass pipeline.

Table 45: Readable source-code responsibility table for `src/Accelerators/Gemmini`.

14.4 File-Level Meaning for the Most Important Components

Instead of one dense path-heavy table, the following separated table uses short file names plus a folder column. This avoids overlap in the PDF.

Area	File	When you edit it	What it controls
Compiler	<code>Options.hpp</code>	Add or change a Gemmini compiler flag	Options understood by the Gemmini path.
Compiler	<code>Utils.cpp/hpp</code>	Share helper logic across passes	Common compiler helper functions.

Area	File	When you edit it	What it controls
Conversion	<code>ONNX2Gem.cpp</code>	Add support for a new ONNX operator	ONNX-to-Gemmini selection and lowering.
Gemmini dialect	<code>Ops.td</code>	Add a high-level Gemmini operation	TableGen definition of Gemmini dialect ops.
GemminiLow dialect	<code>LowOps.td</code>	Add a lower-level accelerator operation	TableGen definition of GemminiLow ops.
Gemmini transform	<code>Tiling.cpp</code>	Change tiling of large ops	How work is split for Gemmini capacity.
GemminiLow transform	<code>Scratchpad.cpp</code>	Change scratchpad placement	Static scratchpad region assignment.
Conversion	<code>ToLow.cpp</code>	Change high-to-low accelerator lowering	Rewrites Gemmini ops into GemminiLow ops.
Conversion	<code>LowToLLVM.cpp</code>	Change low-level LLVM lowering	Converts GemminiLow ops to LLVM-compatible IR.
Runtime	<code>Runtime.cpp</code>	Change actual runtime behavior	Implements conv, GEMM, ReLU, pooling, tracing.
Runtime	<code>Runtime.hpp</code>	Change runtime ABI declarations	Declares exported runtime functions.
Runtime ABI	<code>gemmini.h</code>	Inspect hardware helper API	Gemmini helper and RoCC-facing declarations.

Table 46: Practical file-level guide: what to edit when changing Gemmini behavior.

14.5 How to Know Which Source File Was Responsible

What you observe	Likely responsible area	Reason
No <code>om_gemmini_*</code> calls appear in <code>resnet18_krn1.onnx.mlir</code>	ONNX to Gemmini or compiler options	The ONNX op was not selected for Gemmini, or <code>-maccel=Gemmini</code> was not active.
<code>krnl.global</code> weights exist but no Gemmini calls appear	Stage 2 eligibility rules	Weights were imported, but accelerator conversion did not claim the operation.

What you observe	Likely responsible area	Reason
Gemmini calls exist in MLIR but link fails with undefined symbols	<code>Runtime/</code> or the link command	The generated model calls a runtime function that the runtime archive does not provide or was not linked.
Spike starts but reports no Gemmini extension	Spike command or simulator setup	The binary may be valid, but Spike was not launched with <code>-extension=gemmini</code> .
Trace prints <code>[gemmini-trace] conv f32</code>	<code>Runtime.cpp</code>	The runtime function was reached and trace mode is active.
Runtime output has <code>nans</code> or <code>infs</code>	Runtime math, tensor shape, or input preprocessing	Invalid values usually indicate a mismatch in assumptions, scaling, shapes, or runtime implementation.

Table 47: Debugging map from visible symptoms to source-code responsibility.

14.6 Minimal Commands to Inspect the GitHub-Mapped Source Locally

```

1 # Enter the local folder corresponding to the GitHub URL.
2 cd $ROOT_DIR/src/Accelerators/Gemmini
3
4 # See the directory organization.
5 tree -L 3
6
7 # Find where runtime functions are implemented.
8 grep -R "om_gemmini_conv_f32" -n Runtime Conversion Dialect Transform
   Compiler Support Pass
9
10 # Find where Gemmini passes are declared or registered.
11 grep -R "GemminiPass" -n .
12
13 # Find where ONNX operators are redirected toward Gemmini.
14 grep -R "ONNXToGemmini\|onnx.Conv\|Conv" -n Conversion Dialect
   Transform

```

Listing 48: Useful local inspection commands

Reading the source code together with generated files

The cleanest way to debug the full stack is to compare source files and generated files side by side:

- Source conversion logic: `src/Accelerators/Gemmini/Conversion/ONNXToGemmini/ONNXToGemmini.cpp`
- Generated MLIR after Stage 2: `gemmini/tests/15-resnet18-float/`

```
resnet18_krnl.onnx.mlir
```

- Source runtime implementation: `src/Accelerators/Gemmini/Runtime/OMRuntimeGemmini.cpp`
- Final linked executable: `gemmini/tests/15-resnet18-float/resnet18_rv64`
- Runtime evidence: `gemmini/tests/15-resnet18-float/spike_trace.log`

15 Quick Reference: What Each Command Does

Command	CPU Programmer's Understanding	What It Actually Does
<code>onnx-mlir -EmitONNXIR</code>	Compiler frontend - read the model.	Parses ONNX binary to MLIR ONNX dialect.
<code>onnx-mlir -EmitMLIR</code>	Compiler optimization - route operations.	Replaces eligible operations with Gemmini runtime calls and lowers remaining operations to Krnl.
<code>onnx-mlir -EmitLLVMIR</code>	Compiler backend - prepare low-level form.	Lowers MLIR into LLVM dialect.
<code>mlir-translate -mlir-to-llvmir</code>	Compiler output - write LLVM text.	Exports LLVM dialect MLIR to standard LLVM IR.
<code>llc -filetype=obj</code>	Machine-code generator.	Generates a RISC-V relocatable object file.
<code>clang -c -target=RV64</code>	Compile C source for RISC-V.	Compiles ONNX-MLIR runtime C files to RV64 objects using the full target <code>riscv64-unknown-linux-gnu</code> .
<code>clang++ -c -target=RV64</code>	Compile C++ source for RISC-V.	Compiles the Gemmini runtime implementation to an RV64 object.
<code>llvm-ar rcs</code>	Static archive builder.	Creates <code>libcruntime_rv64.a</code> and <code>libgemmini_f32_rv64.a</code> .
<code>clang++ -fuse-ld=lld -static</code>	LLVM linker driver.	Combines the runner object, compiled model object, runtime archives, and system libraries into <code>resnet18_rv64</code> .
<code>spike --extension=gemmini pk resnet18_rv64</code>	Simulator execution.	Runs the final RISC-V executable with the Gemmini extension enabled.

Table 48: Separated command reference table. Borders are used here deliberately so command rows are visually distinct.

16 Summary: What We've Built

- **Input:** A ResNet-18 ONNX model (46 MB, neural network description)

- **Process:** Nine compilation and linking stages
- **Output:** `resnet18_rv64` (50 MB static ELF executable)
- **Contents:** Model code (1.55 MB), Weights (46.9 MB), Runtime libraries (0.2 MB), System libraries
- **What it does:** Runs ResNet-18 inference using the Gemmini accelerator
- **How:** Compiler-generated code calls Gemmini runtime functions; runtime issues RoCC instructions to the accelerator

Final Thought

This whole pipeline is like:

1. You write a high-level algorithm (the neural network)
2. The compiler translates it to assembly
3. The runtime library provides the hardware-specific details
4. The linker puts it all together
5. The result is a program that runs on the target hardware

The only difference is that "target hardware" includes both a RISC-V CPU and a Gemmini accelerator. The compiler handles the CPU code generation; the runtime library handles the accelerator code generation. The pipeline is the bridge between the model and the hardware.

17 Complete Technical Audit: From ONNX Model to RISC-V Executable

Purpose of this audit section

This section is a detailed explanation of how ONNX-MLIR transforms a neural-network model into a runnable RISC-V executable. It tracks one concrete operation, **Conv**, across every representation: ONNX, ONNX dialect MLIR, Gemmini/Krnl MLIR, LLVM dialect, LLVM IR, RISC-V object code, runtime archive, final executable, and Spike/Gemmini execution.

17.1 The Main Idea in One Sentence

End-to-end rule

ONNX-MLIR does not execute the model directly. It reads the ONNX graph, rewrites supported neural-network operators into lower-level compiler operations and runtime calls, generates RISC-V object code, and links that object code with runtime libraries so the final executable can call Gemmini-aware functions at inference time.

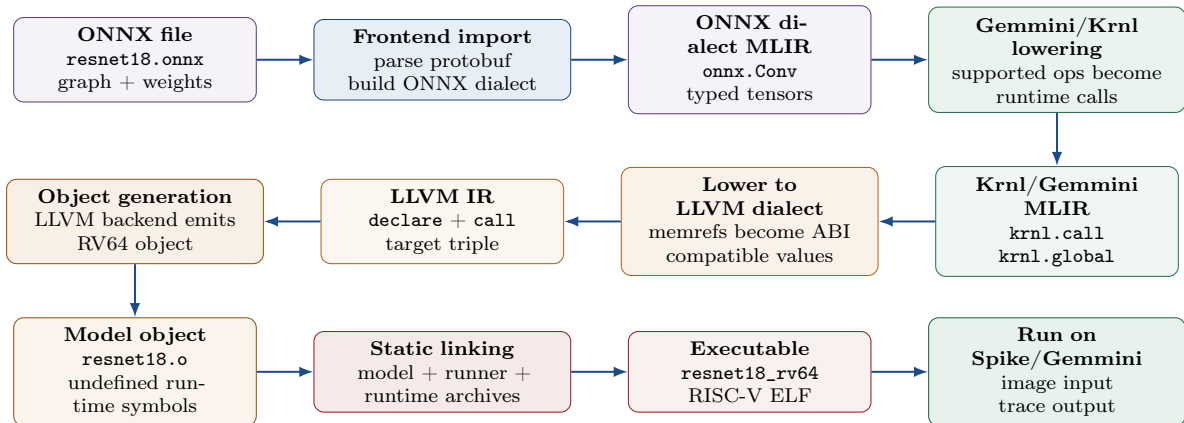


Figure 24: Complete ONNX-MLIR to RISC-V executable transformation chain.

17.2 Stage-by-Stage Transformation Table

Stage	Input	Output	Responsible source area	What changes and why it is useful
1. Import	<code>resnet18.onnx</code>	<code>resnet18-onnxir.onnx.mlir</code>	ONNX frontend and ONNX dialect infrastructure	The binary ONNX graph becomes text MLIR. Operators are still high-level, for example <code>onnx.Conv</code> . This is useful because the compiler can now inspect shapes, attributes, operands, and constants.
2. Accelerator decision	ONNX dialect MLIR	<code>resnet18-krnl.onnx.mlir</code>	<code>ONNXToGemmini.cpp</code>	Supported ONNX operators are matched by legality predicates. For a supported convolution, the high-level <code>onnx.Conv</code> is replaced with a runtime call such as <code>krnl.call @om-gemmini_conv_f32</code> .
3. Krnl/ABI lowering	Krnl/Gemmini MLIR	<code>resnet18.onnx.mlir</code>	Krnl-to-LLVM lowering plus Gemmini lowering utilities	Tensor-like compiler values are lowered toward pointer-based runtime values. This prepares the model to call C/C++ runtime functions.
4. LLVM IR export	LLVM dialect MLIR	<code>resnet18.ll</code>	LLVM dialect translator	MLIR syntax becomes standard LLVM IR. Calls appear as <code>declare</code> and <code>call</code> instructions. This is useful because LLVM backends know how to generate machine code from LLVM IR.
5. Object generation	<code>resnet18.ll</code>	<code>resnet18.o</code>	LLVM backend, usually <code>llc</code> or <code>clang</code>	LLVM IR becomes a RISC-V relocatable object. Runtime functions are still unresolved external symbols, so the object can be linked later.
6. ONNX runtime library	C runtime sources	<code>libcruntime_rv64.a</code>	<code>src/Runtime/</code> + runtime headers	Builds tensor containers such as <code>OMTensor</code> and <code>OMTensorList</code> . This gives the model a stable ABI for input and output tensors.
7. Gemmini runtime library	<code>OMRuntimeGemmini.cpp</code>	<code>libgemmini_f32_rv64.a</code>	<code>Runtime/</code> + hardware ABI	Builds the implementations of <code>om_gemmini_*</code> functions. This is where high-level runtime calls are translated into <code>im2col</code> , quantized <code>matmul</code> , <code>RoCC</code> /Gemmini helper calls, or scalar loops when an operation is not hardware-mapped.
8. Runner	<code>float_model_runner.c</code>	<code>float_model_runner.o</code>	<code>gemmini/examples/tools/</code>	Provides <code>main()</code> . It loads/generates input, creates <code>OMTensorList</code> , calls <code>run_main_graph()</code> , and prints output checks.
9. Link	objects + archives	<code>resnet18_rv64</code>	LLVM LLD or GNU linker	Resolves all symbols. The call from <code>resnet18.o</code> to <code>om_gemmini_conv_f32</code> is connected to the implementation in <code>libgemmini_f32_rv64.a</code> .
10. Execute	<code>resnet18_rv64</code>	trace + output tensor	Spike + Gemmini extension or real FPGA/silicon	The executable runs. <code>GEMMINI_TRACE=1</code> prints evidence that convolution and GEMM functions are reached and that tensors remain finite.

Table 49: Full pipeline from ONNX input to executable output with responsible source areas.

17.3 Tracked Example: The First ResNet-18 Convolution

The first convolution

The first ResNet-18 convolution takes an input image tensor $[1, 3, 224, 224]$, uses 64 filters of shape $[3, 7, 7]$, applies stride 2 and padding 3, and produces $[1, 64, 112, 112]$. This is the best operation to track because it appears clearly in the ONNX graph, in the Gemmini lowering, and in the runtime trace.

```

1 %y = "onnx.Conv"(%x, %w, %b) {
2   dilations = [1, 1],
3   group = 1,
4   kernel_shape = [7, 7],
5   pads = [3, 3, 3, 3],
6   strides = [2, 2]
7 } : (tensor<1x3x224x224xf32>,
8     tensor<64x3x7x7xf32>,
9     tensor<64xf32>) -> tensor<1x64x112x112xf32>

```

Listing 49: ONNX dialect form of a convolution, basic shape

Syntax element	Meaning	Why it matters
%y	SSA result value	MLIR uses SSA form: every intermediate result has a name and is assigned once. This makes compiler rewrites safer and easier to analyze.
onnx.Conv	Operation name in ONNX dialect	This says the operation still has ONNX semantics. It has not yet become loops, runtime calls, or machine code.
(%x, %w, %b)	Operands	Input feature map, weight tensor, and optional bias. These become arguments to later runtime calls.
dilations=[1,1]	Kernel spacing	A dilation of 1 means the kernel samples adjacent pixels. The Gemmini float convolution path accepts this simple dense-kernel case because it maps cleanly to im2col.
group=1	Normal convolution, not grouped convolution	One group means every output channel can use all input channels. Grouped convolution would split channels into independent groups and needs extra indexing logic.
kernel_shape=[7,7]	7-by-7 filter window	The first layer uses a large spatial kernel. Runtime uses this to compute $K = C * KH * KW = 3 * 7 * 7 = 147$.
pads=[3,3,3,3]	top, left, bottom, right padding	Symmetric padding simplifies the runtime formula. Output height is $(224 + 2 * 3 - 7) / 2 + 1 = 112$.
strides=[2,2]	Movement in H and W	Equal stride in both dimensions allows the runtime to pass one scalar stride argument. It reduces spatial size from 224 to 112.
tensor<1x3x224x224xf32>	Input type	Rank-4 NCHW float32 tensor: batch, channels, height, width. The rank and type are checked before the operator can be lowered to Gemmini.
-> tensor<1x64x112x112xf32>	Output type	The compiler knows the expected output shape and can allocate the output tensor before calling the runtime.

Table 50: Syntax explanation for the ONNX dialect convolution operation.

17.4 Why the Gemmini Converter Checks Group, Dilation, Pads, and Strides

Check	Responsible code concept	Reason and benefit
Rank must be 4	<code>xTy.getRank() != 4 /</code> <code>wTy.getRank() != 4 /</code> <code>yTy.getRank() != 4</code>	The runtime implementation assumes NCHW input and OIHW weights. Rank-4 is the layout contract between compiler and runtime.
Element type must be f32	<code>getElementType() != f32</code>	The runtime function <code>om_gemmini_conv_f32</code> expects float32 OMTensors. Other types must use a different lowering or runtime entry point.
<code>group = 1</code>	<code>op.getGroup() != 1</code>	The runtime implements dense convolution where every output channel sees all input channels. This keeps the im2col matrix simple and avoids grouped-channel indexing.
Dilation must be 1	loop over <code>op.getDilations()</code>	Dilation larger than 1 inserts gaps inside the kernel sampling pattern. The current im2col path is written for dense adjacent kernel positions.
H/W stride must match	<code>sh != sw</code>	A single scalar <code>stride</code> is passed to the runtime. Equal H/W stride keeps the runtime ABI smaller and easier to verify.
Pads must be symmetric	<code>pt == pl == pb == pr</code>	A single scalar <code>pad</code> is passed to the runtime. Symmetric padding avoids passing four pad values and simplifies output-shape computation.
Weight shape must be static	<code>!wTy.hasStaticShape()</code>	The runtime needs known kernel and channel counts to build matrices for Gemmini. Static weights also become constants in the executable.
Output shape must agree	ONNX formula	The compiler verifies that $OH = (H + 2 * pad - KH) / stride + 1$ and $OW = (W + 2 * pad - KW) / stride + 1$. This prevents runtime shape mismatch.

Table 51: Why the ONNX-to-Gemmini converter restricts which convolution forms are accepted.

Important interpretation

A rejected operator does not mean the whole model fails. It means that this specific operator is not rewritten into the selected Gemmini runtime call. To know which path was used, inspect the generated MLIR and the final runtime trace, not only the original ONNX graph.

17.5 How to Know That Gemmini Was Actually Used

Verification level	Command or evidence	Meaning
Compiler IR evidence	<code>grep -n "om_gemmini_conv_f32" resnet18_krn1.onnx.mlir</code>	The compiler rewrote at least one ONNX convolution into a Gemmini runtime call.
Object-file evidence	<code>llvm-nm -u resnet18.o grep om_gemmini_conv_f32</code>	The generated model object contains an unresolved call to the runtime function.
Runtime archive evidence	<code>llvm-nm libgemmini_f32_rv64.a grep om_gemmini_conv_f32</code>	The Gemmini runtime archive provides the implementation that will satisfy the model object's external symbol.
Final executable evidence	<code>llvm-nm resnet18_rv64 grep om_gemmini_conv_f32</code>	The final executable contains the resolved runtime function after static linking.
Runtime trace evidence	<code>GEMMINI_TRACE=1 ... spike ... prints [gemmini-trace] conv f32</code>	The program actually entered the Gemmini runtime function while running.
Simulator evidence	Gemmini extension configured with: <code>dim = 16</code>	Spike loaded the Gemmini extension. The simulated accelerator has a 16-by-16 array dimension.

Table 52: Concrete checks that separate compile-time lowering, link-time resolution, and runtime execution.

17.6 Conv Function Tracking Across Files

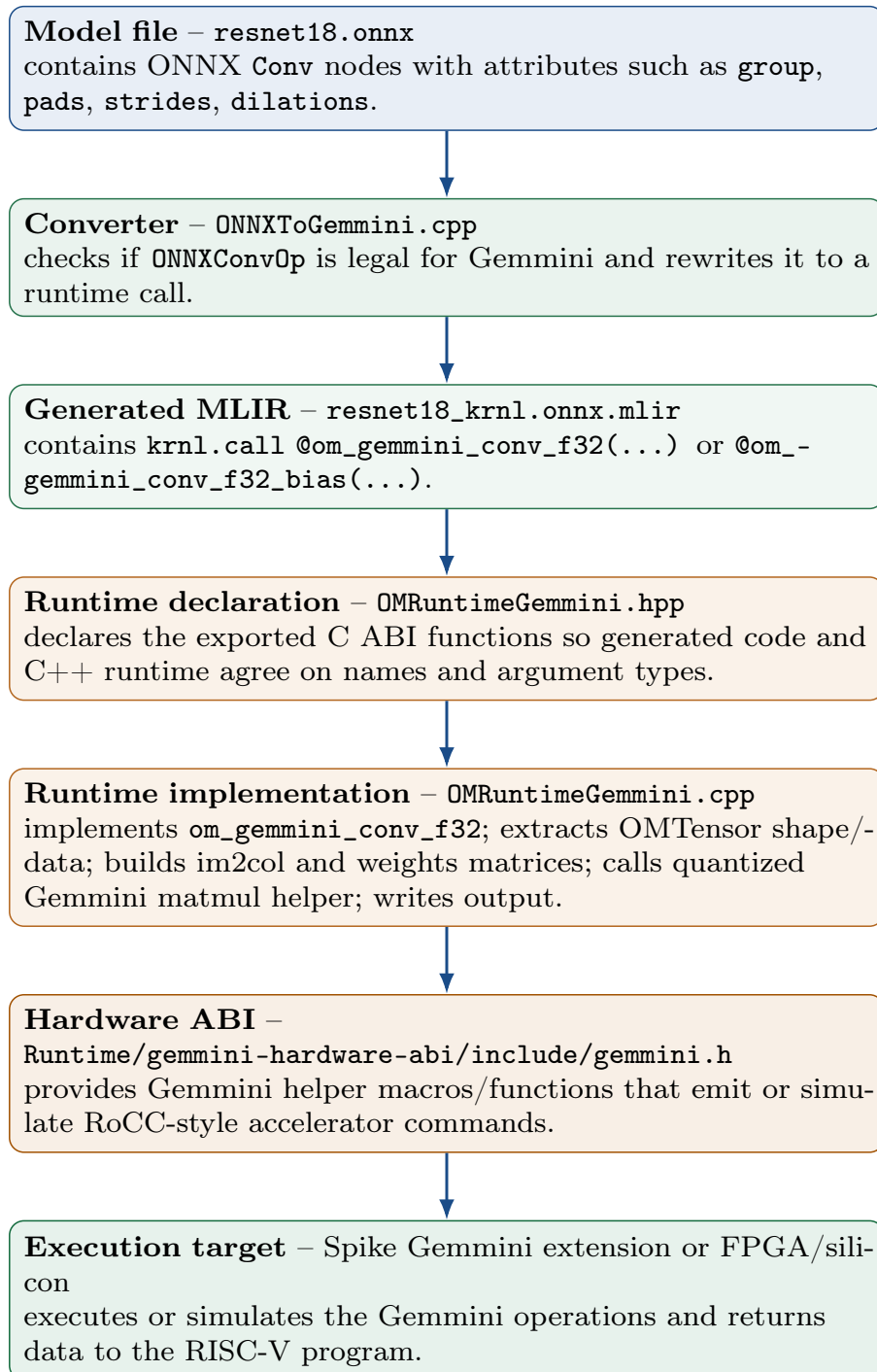


Figure 25: Tracking Conv from ONNX source to Gemmini runtime and hardware interface.

File or folder	What to look for	Why it is responsible
GemminiAccelerator.*	accelerator registration and hooks	Connects the Gemmini accelerator integration to the ONNX-MLIR driver infrastructure.
Compiler options	option definitions	Controls flags such as enabling Gemmini acceleration. These options decide whether Gemmini lowering is active.
ONNXToGemmini.cpp	isSupportedGemminiConvF32, rewrite patterns	This is the main source file that decides whether ONNX Conv, Gemm, MatMul, and other ops become Gemmini runtime calls.
Dialect/Gemmini/*	operation definitions	Defines compiler IR operations for the Gemmini dialect. .td files specify operation syntax; .cpp/.hpp provide C++ classes and verification.
Dialect/GemminiLow/*	lower-level Gemmini operations	Represents lower-level accelerator operations closer to the hardware interface.
GemminiTiling.cpp	tiling transforms	Splits large operations into sizes compatible with Gemmini scratchpad and systolic array constraints.
ScratchpadAllocation.cpp	scratchpad allocation	Assigns explicit on-chip memory regions when lowering to low-level accelerator operations.
GemminiToGemminiLow/*	dialect-to-low lowering	Converts abstract Gemmini operations into lower-level operations.
GemminiLowToLLVM/*	lowering to LLVM	Converts low-level accelerator operations into LLVM-compatible code and intrinsic/custom instruction forms.
OMRuntimeGemmini.hpp	exported function declarations	Contains the public function interface that generated model code expects to call.
OMRuntimeGemmini.cpp	exported function implementations	Implements om_gemmini_conv_f32, om_gemmini_gemm_f32_bias, pooling, activation, and utility operations.
gemmini-hardware-abi/	gemmini.h, RoCC helpers	Provides the software-visible accelerator interface used by the runtime.
GemminiTargetInfo.*	hardware constants	Defines target information such as Gemmini dimensions used by the compiler side.

Table 53: Source-code ownership map for the Gemmini accelerator integration.

17.7 The Runtime Conv Implementation in Plain English

```

1 void om_gemmini_conv_f32(OMTensor *out,
2                           const OMTensor *x,
3                           const OMTensor *w,
4                           int64_t stride,
5                           int64_t pad) {
6     // 1. Read tensor shapes and raw data pointers from OMTensor.
7     // 2. Convert convolution into matrix multiplication using im2col.
8     // 3. Rearrange weights into a matrix layout suitable for GEMM.
9     // 4. Quantize float32 matrices into int8 staging buffers.
10    // 5. Dispatch the tiled int8 matmul path used by Gemmini.
11    // 6. Dequantize/correct the result and write the float32 output
12    //    tensor.
13 }

```

Listing 50: Simplified logic of om_gemmini_conv_f32

Runtime step	Code concept	Explanation
Read metadata	omTensorGetShape, omTensorGetDataPtr	The runtime receives opaque tensor objects. It must extract dimensions and raw memory pointers before doing arithmetic.
Compute dimensions	N,C,H,W,OC,KH,KW,OH,OW	The runtime reconstructs convolution geometry. For the first conv: N=1, C=3, H=W=224, OC=64, KH=KW=7, OH=OW=112.
Build im2col	im2col[row*K+k]	Each output pixel position becomes one row of a matrix. Each row contains the flattened receptive field that the convolution kernel sees.
Pack weights	weights[k*OC+oc]	ONNX stores conv weights as OIHW. The runtime rearranges them into a GEMM-friendly matrix.
Call Gemmini matmul helper	om_gemmini_quantized_- matmul_f32_ws(...)	Float32 data is staged through an int8 Gemmini path. This uses Gemmini efficiently while preserving a float32 output interface to the model.
Apply correction and bias	dequant + (exact - dequant)	The runtime computes a correction term so the final stored value matches full float32 behavior more closely. Optional bias is added per output channel.
Trace output	traceF32BufferStats("conv output", ...)	When tracing is enabled, the runtime prints min, max, mean, nonzero count, NaN count, and Inf count for debugging.

Table 54: Plain-English explanation of the Gemmini float32 convolution runtime implementation.

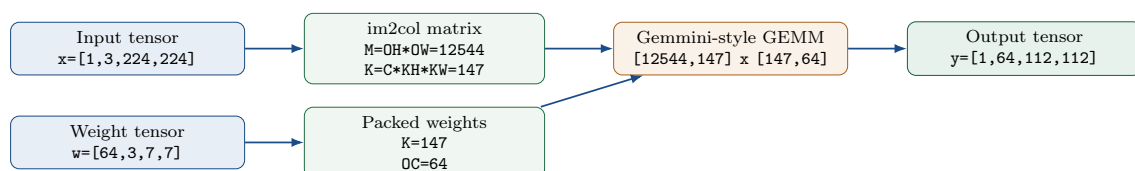


Figure 26: How a convolution becomes matrix multiplication inside the runtime.

17.8 Syntax Dictionary: Tensor, MemRef, OMTensor, Pointer, Symbol

Term	Where you see it	Meaning and benefit
<code>tensor<...></code>	ONNX dialect MLIR	A value with shape and element type, but not yet explicit memory allocation. Useful for high-level compiler analysis.
<code>memref<...></code>	Krnl/lowered MLIR	A memory reference: allocated storage with shape, element type, and layout information. Useful when the compiler starts generating memory operations.
<code>krnl.global</code>	Krnl MLIR	A global constant, often model weights. It later becomes read-only data in the executable, similar to a <code>global const float[]</code> array.
<code>krnl.call</code>	Krnl MLIR	A compiler-level call wrapper. In this project it is used to call runtime functions such as <code>om_gemmini_conv_f32</code> .
<code>OMTensor*</code>	C/C++ runtime ABI	A pointer to a runtime tensor object containing data pointer, shape, rank, strides, data type, and ownership information. It lets compiled code pass tensors to C/C++ runtime functions safely.
<code>ptr</code>	LLVM IR	A generic LLVM pointer type. By this stage the compiler is close to machine-code generation, so rich tensor objects are represented as addresses.
<code>declare</code>	LLVM IR	Says a function exists but is defined elsewhere. For example, <code>declare void @om_gemmini_conv_f32(...)</code> means the linker must find the implementation later.
U in <code>llvm-nm</code>	Object file symbol table	Undefined symbol. The model object calls the function, but the implementation is not inside that object file. The runtime archive will provide it.
T in <code>llvm-nm</code>	Executable or runtime object	Text/code symbol. The function implementation exists in the code section.

Table 55: Compiler and binary syntax dictionary for tracking model operations.

17.9 What the Headers and C++ Files Do

Header vs implementation

A `.hpp` file normally declares the interface: function names, argument types, classes, and declarations. A `.cpp` file normally provides the implementation: the executable logic that runs. The compiled model needs the interface to agree with the runtime, while the linker needs the implementation to resolve symbols.

File type	Example	Role
.hpp declaration	OMRuntimeGemmini.hpp	Declares runtime functions such as <code>om_gemmini_conv_f32</code> . It protects the ABI: generated model code and runtime object code must use the same function names and argument types.
.cpp implementation	OMRuntimeGemmini.cpp	Contains executable logic: tracing, tensor metadata extraction, <code>im2col</code> , quantization, Gemmini <code>matmul</code> call, pooling, activation, and output writes.
.td TableGen file	GemminiOps.td	Describes MLIR operations declaratively. MLIR uses TableGen to generate boilerplate C++ classes and parsers/printers.
CMakeLists.txt	Runtime and dialect folders	Tells the build system which sources belong to each library and which include directories/compile definitions are required.

Table 56: How source file types cooperate in ONNX-MLIR and the Gemmini runtime.

17.10 Why ONNX-MLIR Uses This Multi-Step Design

Design choice	Benefit
Keep ONNX dialect first	Preserves neural-network semantics while the compiler checks shapes and attributes. This makes transformations safer than immediately generating low-level loops.
Use accelerator-specific lowering	Lets Gemmini-supported operators take a faster path while unsupported operators can still be lowered through generic compiler paths.
Use <code>kernels.call</code> runtime calls	Keeps generated model code small and moves complicated accelerator logic into testable C/C++ runtime functions.
Use <code>OMTensor</code> ABI	Provides a stable boundary between compiler-generated code and C/C++ runtime libraries. The runtime can inspect shape, data type, rank, strides, and ownership.
Use LLVM IR/backend	Reuses mature LLVM code generation for RISC-V instead of building a custom machine-code generator.
Use static linking	Produces one self-contained ELF executable for Spike, FPGA Linux, or other RISC-V environments without requiring dynamic library setup at runtime.
Use runtime trace	Gives practical evidence that the final binary entered the expected runtime functions and produced finite tensors.

Table 57: Benefits of the staged ONNX-MLIR to Gemmini compilation design.

17.11 Minimal Debugging Checklist

```

1 # 1. Confirm ONNX Conv exists before accelerator lowering.
2 grep -n '"onnx.Conv"' $OUT_DIR/resnet18_onnxir.onnx.mlir | head
3
4 # 2. Confirm the Gemmini runtime call was emitted.
5 grep -n 'om_gemmini_conv_f32' $OUT_DIR/resnet18_krn1.onnx.mlir | head
6
7 # 3. Confirm the model object references the runtime function.
8 llvm-nm -u $OUT_DIR/resnet18.o | grep om_gemmini_conv_f32
9
10 # 4. Confirm the Gemmini archive provides the implementation.
11 llvm-nm $OUT_DIR/libgemmini_f32_rv64.a | grep om_gemmini_conv_f32
12
13 # 5. Confirm the final executable contains the resolved function.
14 llvm-nm $OUT_DIR/resnet18_rv64 | grep om_gemmini_conv_f32
15
16 # 6. Confirm runtime execution reaches the Gemmini trace.
17 GEMMINI_TRACE=1 stdbuf -oL -eL $SPIKE --extension=gemmini --isa=
    rv64imafdc \
18 -m2048 $PK $OUT_DIR/resnet18_rv64 \
19 --image $OUT_DIR/data/dog_224.ppm --in-n 1 --in-c 3 --in-h 224 --in-w
    224 \
20 2>&1 | tee $OUT_DIR/spike_trace.log
21
22 grep -n '\[gemmini-trace\] conv f32' $OUT_DIR/spike_trace.log | head

```

Listing 51: Commands for following one Conv from IR to runtime

How to interpret the checks

If step 1 succeeds but step 2 fails, the ONNX Conv existed but was not selected by the Gemmini lowering. Check the predicate conditions in `ONNXToGemmini.cpp`. If step 2 succeeds but step 4 fails, the generated model expects a runtime function that is not present in the runtime archive. Check `OMRuntimeGemmini.hpp`, `OMRuntimeGemmini.cpp`, and the runtime build. If steps 2 to 5 succeed but step 6 does not print trace lines, the binary may not be executing the traced model path, the environment variable may be disabled, or the run may fail before the convolution.

18 Deep Runtime Folder Audit: Every File, Function, and Variable Family

Goal of this audit

This section explains the `src/Accelerators/Gemmini/Runtime/` folder as if you are reading it before modifying the backend. The objective is to answer four questions:

1. Which file is responsible for each runtime responsibility?
2. Which runtime functions are callable from compiler-generated code?
3. Which helper functions are internal implementation details?
4. What do the most important variables mean inside convolution, GEMM, quantization, pooling, resize, pad, split, and transpose?

The important mental model is: **ONNX-MLIR emits calls; the Runtime folder implements those calls.** The compiled model object contains calls such as `om_gemmini_conv_f32`; the runtime archive `libgemmini_f32_rv64.a` provides the actual implementation.

18.1 Runtime Folder Responsibility Graph

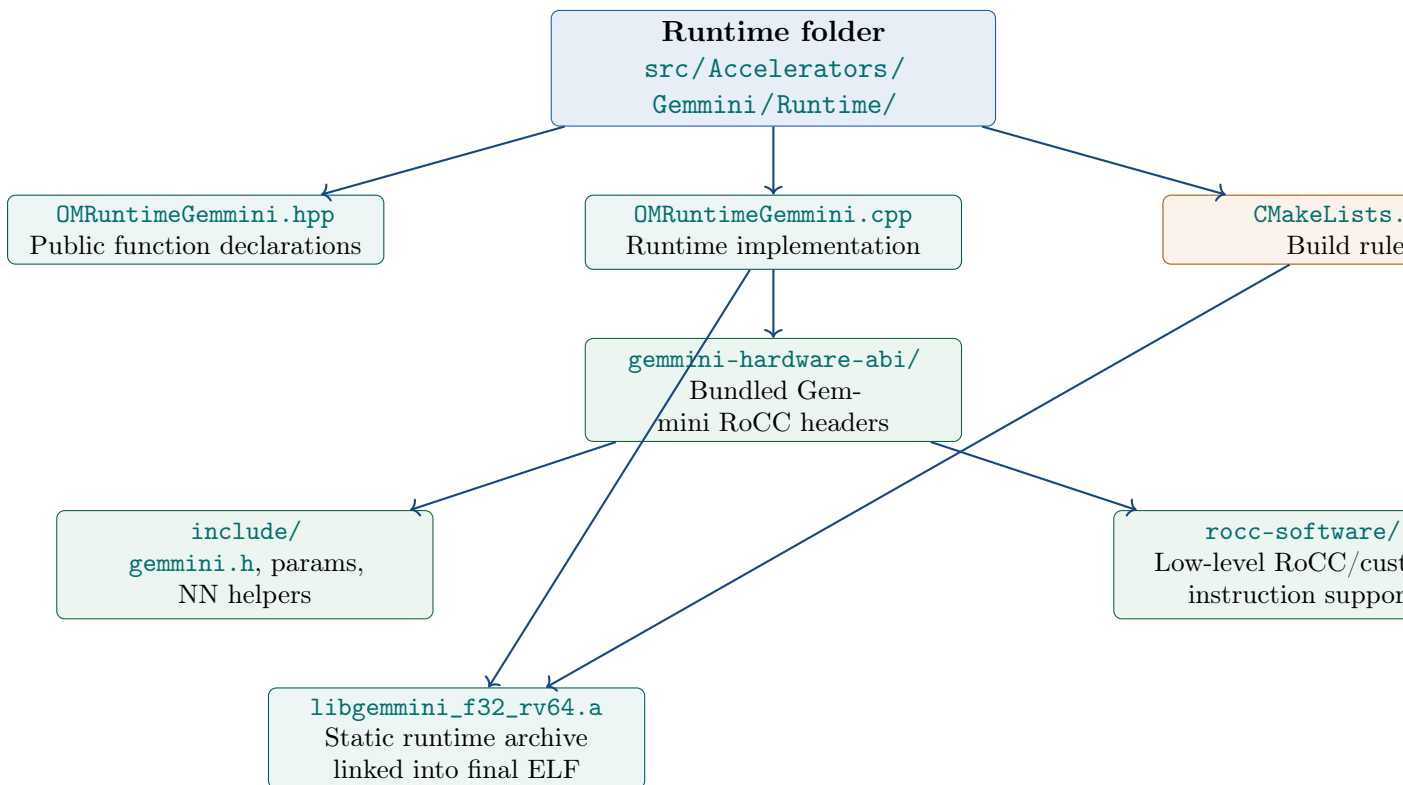


Figure 27: Runtime folder ownership map. The header declares the ABI, the C++ file implements it, the ABI subfolder exposes Gemmini hardware commands, and the build file packages everything into a static archive.

18.2 Every Runtime Folder and File

Path	Role	What it means in the ONNX-to-executable pipeline
<code>Runtime/</code>	Runtime bridge folder	This folder is the boundary between compiler-generated model code and Gemmini execution. The compiler emits calls. This folder provides the callable C/C++ functions.
<code>Runtime / OMRuntimeGemmini.hpp</code>	Public runtime ABI header	Declares the <code>om_gemmini_*</code> functions with C linkage so generated LLVM/RISC-V object code can link to stable symbol names without C++ name mangling.
<code>Runtime / OMRuntimeGemmini.cpp</code>	Main implementation	Implements convolution, GEMM, MatMul, pooling, activation, normalization, padding, resizing, splitting, transposition, tracing, quantization, and CPU scalar fallbacks.
<code>Runtime/CMakeLists.txt</code>	Build integration	Registers the runtime source in the ONNX-MLIR/Gemmini build so it can be compiled into a runtime library. In manual builds, the same idea is done with <code>clang++ -c</code> and <code>llvm-ar</code> .
<code>Runtime/FILES_EXPLAINED.md</code>	Local documentation	Developer notes that summarize the purpose of the Runtime folder. Use it as a first orientation file, then read the header and implementation.
<code>Runtime / gemmini-hardware-abi/</code>	Bundled hardware ABI	Contains the Gemmini-side C headers required by the runtime. This is not ONNX-MLIR logic; it is the low-level interface used to issue accelerator operations.
<code>Runtime / gemmini-hardware-abi/include/</code>	Hardware-facing include directory	Usually contains headers such as <code>gemmini.h</code> , <code>gemmini_params.h</code> , <code>gemmini_nn.h</code> , <code>gemmini_counter.h</code> , and custom-instruction helpers. These define calls like <code>tilled_matmul_auto</code> , <code>tilled_conv_auto</code> , and <code>gemmini_fence</code> .
<code>Runtime / gemmini-hardware-abi/rocc-software/</code>	RoCC support code	Provides support headers/macros for Rocket Custom Coprocessor style instructions. In Spike, these calls are emulated; on FPGA/silicon, they become real accelerator commands.

Do not confuse compiler folder and runtime folder

Compiler lowering files such as `ONNXToGemmini.cpp` decide whether an ONNX operation becomes a Gemmini call. **Runtime files** such as `OMRuntimeGemmini.cpp` implement what happens after that call is made. In other words: compiler code chooses `om_gemmini_`

`conv_f32`; runtime code executes `om_gemmini_conv_f32`.

18.3 Runtime Call Chain: From Compiled Model to Hardware Helper

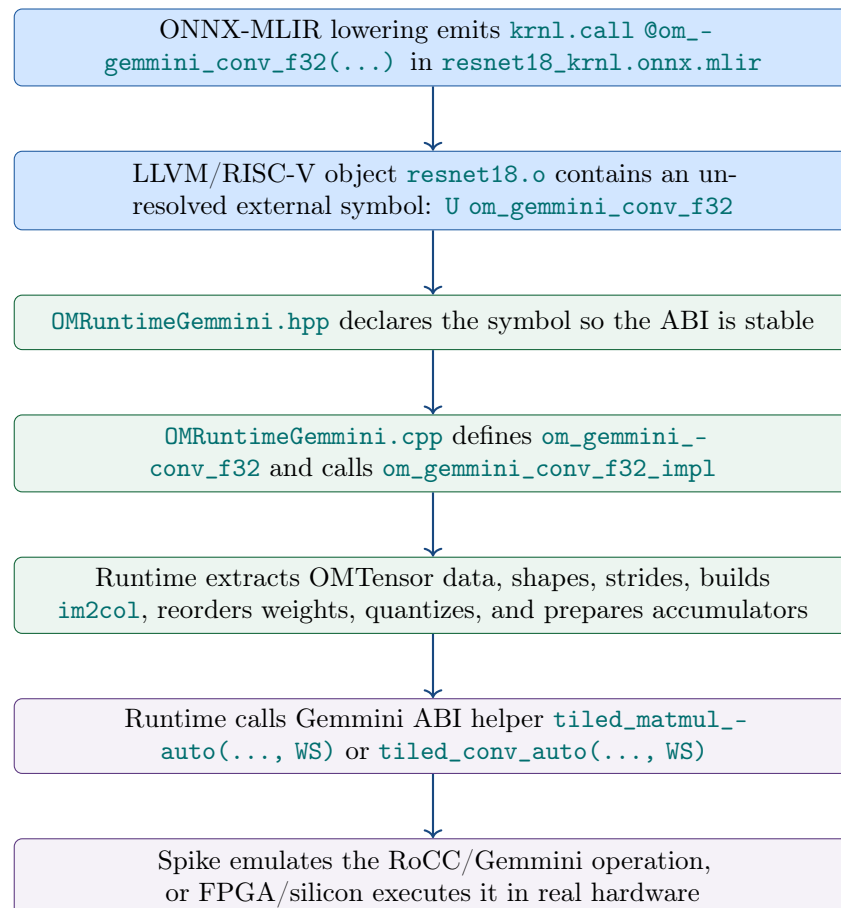


Figure 28: How one compiled Conv call crosses from ONNX-MLIR generated code into the Runtime folder and then into Gemini.

18.4 File-by-File Deep Explanation

18.4.1 `OMRuntimeGemmini.hpp`: public symbol contract

What the header does

The header is the contract between generated model code and the runtime library. It normally contains function declarations such as `om_gemmini_conv_f32`, `om_gemmini_gemm_f32_bias`, and other `om_gemmini_*` entry points. The generated object file does not need to know how those functions work; it only needs their exact names and argument types.

```

1 #ifdef __cplusplus
2 extern "C" {
3 #endif
4
5 void om_gemmini_conv_f32(

```

```

6   OMTensor *out, const OMTensor *x, const OMTensor *w,
7   int64_t stride, int64_t pad);
8
9  #ifdef __cplusplus
10 }
11 #endif

```

Listing 52: Conceptual header pattern

Syntax	Meaning
<code>extern "C"</code>	Prevents C++ name mangling. Without it, the linker may look for a decorated C++ symbol instead of the plain symbol emitted by compiler-generated code.
<code>OMTensor *out</code>	Pointer to a runtime tensor object where the result is already allocated. The runtime fills its data buffer.
<code>const OMTensor *x</code>	Read-only input tensor. The runtime can inspect its shape, rank, strides, and data pointer but should not modify it.
<code>int64_t stride, pad</code>	Scalar attributes from the original ONNX Conv operation. They are passed as plain integers because they are compile-time/runtime parameters, not tensors.

18.4.2 OMRuntimeGemmini.cpp: implementation and execution logic

What this implementation file contains

`OMRuntimeGemmini.cpp` is the main runtime shim. It includes ONNX-MLIR tensor accessors, the runtime header, small floating-point conversion helpers, and `gemmini.h`. It also contains a RISC-V compile guard so it is not accidentally compiled for the host machine. The source declares many `om_gemmini_*` entry points for compiled models, including integer/quantized functions, float convolution/GEMM/MatMul functions, pooling, activations, and scalar data-movement operations.

```

1  #include "include/onnx-mlir/Runtime/OMTensor.h"
2  #include "src/Accelerators/Gemmini/Runtime/OMRuntimeGemmini.hpp"
3  #include "src/Support/SmallFPConversion.h"
4
5  #if !defined(__riscv)
6  #error "OMRuntimeGemmini.cpp must only be compiled for RISC-V targets"
7  #endif
8
9  #include "gemmini.h"

```

Listing 53: Important includes and target guard

Include / guard	Why it is used	Benefit
<code>OMTensor.h</code>	Gives accessors such as <code>omTensorGetShape</code> , <code>omTensorGetDataPtr</code> , <code>omTensorGetRank</code> , <code>omTensorGetStrides</code> .	The runtime can interpret opaque tensors passed by compiler-generated code.

Include / guard	Why it is used	Benefit
<code>OMRuntimeGemmini.hpp</code>	Keeps declarations and definitions consistent.	Catches signature mismatches at compile time.
<code>SmallFPConversion.h</code>	Converts fp16/bfloat16 values when f16 run-time paths need f32 arithmetic.	Allows fp16 model inputs to be handled through the same compute/quantization strategy.
<code>#if !defined(__-riscv)</code>	Stops accidental x86/host compilation.	Prevents false success on a machine that cannot issue Gemmini/RoCC code.
<code>gemmini.h</code>	Exposes Gemmini helper functions and data types.	Connects runtime-level operations to accelerator-level commands.

18.4.3 `CMakeLists.txt`: build integration

What the build file does

`CMakeLists.txt` tells the project build system how the Runtime folder participates in the final ONNX-MLIR/Gemmini build. In your manual test pipeline, the same job is visible in the explicit commands that compile `OMRuntimeGemmini.cpp` into `OMRuntimeGemmini.o` and archive it into `libgemmini_f32_rv64.a`.

```

1 clang++ --target=riscv64-unknown-linux-gnu -std=c++17 -c \
2   -I $ROOT_DIR/include \
3   -I $ROOT_DIR \
4   -I $ROOT_DIR/src/Accelerators/Gemmini/Runtime/gemmini-hardware-abi \
5   -I $ROOT_DIR/src/Accelerators/Gemmini/Runtime/gemmini-hardware-abi/
6   include \
7   $ROOT_DIR/src/Accelerators/Gemmini/Runtime/OMRuntimeGemmini.cpp \
8   -o $OUT_DIR/gem_rt_obj/OMRuntimeGemmini.o
9
10 llvm-ar rcs $OUT_DIR/libgemmini_f32_rv64.a \
    $OUT_DIR/gem_rt_obj/OMRuntimeGemmini.o

```

Listing 54: Manual equivalent of the runtime build

18.4.4 `gemmini-hardware-abi/`: hardware ABI layer

ABI item	Meaning	Why the runtime needs it
<code>gemmini.h</code>	Main user-facing Gemmini helper API.	Provides calls such as <code>tiled_matmul_auto</code> , <code>tiled_conv_auto</code> , and <code>gemmini_fence</code> .
<code>gemmini_params.h</code>	Gemmini hardware configuration constants.	Lets runtime code agree with the accelerator configuration such as array dimension and data types.
<code>gemmini_nn.h</code>	Neural-network helper definitions.	Provides higher-level NN operation helpers and parameter conventions used by convolution/matmul paths.

ABI item	Meaning	Why the runtime needs it
<code>gemmini_counter.h</code>	Performance counter helpers.	Allows optional instrumentation of accelerator events.
<code>xcustom.h</code> / RoCC helpers	Low-level custom-instruction encoding helpers.	Bridges C/C++ helper calls to RoCC custom instruction emission.

18.5 Function Inventory: Every Function in `OMRuntimeGemmini.cpp`

How to read the tables

Public runtime entry points are functions whose names start with `om_gemmini_` and are intended to be called from generated model code or from wrappers. **Static helpers** are private functions used only inside `OMRuntimeGemmini.cpp`. Public functions are the ABI; static helpers are implementation details.

Function	Visibility	Main variables	Purpose
<code>gemminiTraceEnabled</code>	static helper	<code>env</code>	Reads <code>GEMMINI_TRACE</code> . Returns true unless the variable exists and starts with 0.
<code>traceF32BufferStats</code>	static helper	<code>label, data, n, minVal, maxVal, sum, nonzero, nans, infs</code>	Prints numerical statistics for a float buffer to prove tensors are finite and populated.
<code>traceConvF32Shape</code>	static helper	<code>N,C,H,W,OC,KH,KW,OH</code>	Prints the compact convolution trace line showing input, weight, output shape, stride, padding, and bias.
<code>isNoneScalarOrVectorI8</code>	static helper	<code>tensor, rank</code>	Validates optional int8 zero-point tensors. Accepts null, scalar, or rank-1 vector.
<code>loadOptionalScalarI8</code>	static helper	<code>tensor</code>	Reads an optional scalar int8 zero-point. Null means zero.
<code>loadScalarF32</code>	static helper	<code>tensor</code>	Reads a required scalar float32 value such as an ONNX quantization scale.
<code>getZeroPointSpan</code>	static helper	<code>tensor, expectedVectorLen, numElems</code>	Determines whether zero-point is scalar or vector and validates its length.
<code>getZeroPointValue</code>	static helper	<code>tensor, index, span</code>	Fetches scalar or per-row/per-column zero-point value.

Function	Visibility	Main variables	Purpose
<code>decodeF32Bits</code>	static helper	<code>bits</code> , <code>u</code> , <code>f</code>	Reconstructs a float from IEEE-754 bits passed through an integer argument.
<code>om_gemmini_tiled_matmul_i8i8acc32_ws</code>	static helper	<code>dimI</code> , <code>dimJ</code> , <code>dimK</code> , <code>A</code> , <code>B</code> , <code>D</code> , <code>C</code> , <code>strides</code>	Calls <code>tiled_matmul_auto</code> using Gemmini weight-stationary mode and then fences.
<code>applyZeroPointCorrection</code>	static helper	<code>output</code> , <code>lhs</code> , <code>rhs</code> , <code>rowSumsA</code> , <code>colSumsB</code> , <code>aZp</code> , <code>bZp</code>	Applies ONNX MatMulInteger zero-point correction after Gemmini int8 matmul.
<code>om_gemmini_matmulinteger_i8i8acc32</code>	public entry	<code>output</code> , <code>lhs</code> , <code>rhs</code>	Implements int8 x int8 to int32 MatMulInteger without zero-points.
<code>om_gemmini_matmulinteger_i8i8acc32_zp</code>	public entry	<code>aZeroPoint</code> , <code>bZeroPoint</code> , <code>shapes</code> , <code>strides</code>	Implements MatMulInteger with zero-points and correction.
<code>packOIHWtoHWOI</code>	static helper	<code>weights</code> , <code>outChannels</code> , <code>inChannels</code> , <code>kernelDim</code> , <code>packedWeights</code>	Reorders convolution weights from ONNX OIHW layout to Gemmini-friendly HWOI-style order.
<code>copyNHWCtoNCHW</code>	static helper	<code>src</code> , <code>dst</code> , <code>outChannels</code> , <code>outRowDim</code> , <code>outColDim</code>	Converts Gemmini-style NHWC temporary output back to ONNX NCHW output layout.
<code>buildAdjustedConvBias</code>	static helper	<code>weights</code> , <code>bias</code> , <code>xZeroPoint</code> , <code>adjustedBias</code> , <code>weightSum</code>	Builds int32 bias corrected for input zero-point in quantized convolution.
<code>om_gemmini_qlinearconv_i8_impl</code>	static helper	<code>input/output tensors</code> , <code>scales</code> , <code>zero-points</code> , <code>packedWeights</code> , <code>tempOutput</code> , <code>adjustedBias</code>	Shared implementation for quantized QLinearConv with or without bias. Calls <code>tiled_conv_auto</code> .
<code>om_gemmini_qlinearconv_i8</code>	public entry	<code>quantized conv tensors</code> , <code>scales</code> , <code>zero-points</code> , <code>stride</code> , <code>padding</code>	Public no-bias QLinearConv wrapper.
<code>om_gemmini_qlinearconv_i8_bias</code>	public entry	same as no-bias plus <code>bias</code>	Public QLinearConv wrapper with bias.
<code>quantizeSymmetricI8</code>	static helper	<code>value</code> , <code>invScale</code>	Converts a float value to signed int8 using symmetric scaling and saturation.

Function	Visibility	Main variables	Purpose
<code>dotProductF32</code>	static helper	<code>a, b, strideA, strideB, k</code>	Computes exact float dot product used for residual correction.
<code>dequantizedAccF32</code>	static helper	<code>acc, scale</code>	Converts int32 accumulator result back to float.
<code>flattenedLeadingOffset</code>	static helper	<code>shape, rank, index</code>	Computes batch/leading-dimension offset for rank-N MatMul.
<code>om_gemmini_-quantized_matmul_-f32_ws</code>	static helper	<code>M,N,K,A,B,C, Aq, Bq, acc, scale</code>	Quantizes float matrices to int8, calls Gemmini matmul, stores int32 accumulators, and exposes output scale.
<code>om_gemmini_matmul_-f32_hw</code>	public entry	<code>output, lhs, rhs</code>	Float rank-2 MatMul path: quantize, use Gemmini, dequantize, then correct to float semantics.
<code>om_gemmini_matmul_-f32_nd_hw</code>	public entry	batched shapes, leading offsets	Rank-N x rank-2 batched MatMul using repeated matrix slices.
<code>maxAbsF16</code>	static helper	<code>data, n</code>	Finds max absolute f16 value after conversion to f32 for scaling.
<code>om_gemmini_matmul_-f16_hw</code>	public entry	f16 input tensors, f32 staging	F16 MatMul path using f32 conversion/quantization for Gemmini execution.
<code>om_gemmini_conv_f32_-impl</code>	static helper	<code>N,C,H,W,OC,KH,KW,OE, xData, wData, outData, im2col, weights, acc</code>	Shared float Conv implementation: im2col, weight reorder, Gemmini matmul, residual correction, optional bias add.
<code>om_gemmini_conv_f32</code>	public entry	<code>out, x, w, stride, pad</code>	Public no-bias Conv wrapper called by generated model code.
<code>om_gemmini_conv_f32_-bias</code>	public entry	same as no-bias plus <code>b</code>	Public Conv wrapper with per-output-channel bias.
<code>om_gemmini_-convtranspose_f32_-impl</code>	static helper	<code>output_pad, shapes, loops over input and kernel</code>	Shared ConvTranspose implementation. Uses scalar reconstruction semantics; marks output padding as encoded in output shape.
<code>om_gemmini_-convtranspose_f32</code>	public entry	<code>out, x, w, stride, pad, output_pad</code>	Public no-bias ConvTranspose wrapper.

Function	Visibility	Main variables	Purpose
<code>om_gemmini_convtranspose_f32_bias</code>	public entry	same as no-bias plus <code>b</code>	Public ConvTranspose wrapper with bias.
<code>om_gemmini_relu_f32</code>	public entry	<code>out</code> , <code>x</code> , <code>n</code>	Element-wise ReLU: writes $\max(x, 0)$ to output.
<code>om_gemmini_batchnorm_f32</code>	public entry	<code>x</code> , <code>scale</code> , <code>bias</code> , <code>mean</code> , <code>var</code> , <code>epsilon</code>	Per-channel BatchNorm. Uses ONNX formula with epsilon for numerical stability.
<code>om_gemmini_add_f32</code>	public entry	<code>out</code> , <code>a</code> , <code>b</code> , <code>n</code>	Element-wise residual addition for ResNet skip connections.
<code>om_gemmini_globalavgpool_f32</code>	public entry	<code>N</code> , <code>C</code> , <code>H</code> , <code>W</code> , sum over <code>H/W</code>	Global average pool: collapses spatial dimensions into one value per channel.
<code>om_gemmini_maxpool_f32</code>	public entry	<code>kernel</code> , <code>stride</code> , <code>pad</code> , spatial loops	Max pooling over local windows.
<code>om_gemmini_avgpool_f32</code>	public entry	<code>kernel</code> , <code>stride</code> , <code>pad</code> , count/sum	Average pooling over local windows.
<code>om_gemmini_softmax_f32</code>	public entry	<code>axis</code> , <code>maxVal</code> , <code>sum</code> , <code>exp</code>	Numerically stable softmax along an axis.
<code>om_gemmini_gemm_f32_impl</code>	static helper	<code>A</code> , <code>B</code> , <code>C/bias</code> , <code>M</code> , <code>N</code> , <code>K</code> , <code>alpha</code> , <code>beta</code> , <code>acc</code>	Shared GEMM implementation for final fully connected layer; uses Gemmini matmul and bias/scaling.
<code>om_gemmini_gemm_f32</code>	public entry	<code>out</code> , <code>A</code> , <code>B</code> , <code>alpha</code> , <code>beta</code>	Public GEMM wrapper without explicit bias tensor.
<code>om_gemmini_gemm_f32_bias</code>	public entry	same as GEMM plus <code>C</code>	Public GEMM wrapper with bias/input C tensor.
<code>resize_input_coord</code>	static helper	<code>outIndex</code> , <code>inSize</code> , <code>outSize</code> , coordinate mode	Computes continuous input coordinate for resize.
<code>resize_nearest_coord</code>	static helper	coordinate, nearest mode	Chooses nearest integer input coordinate.
<code>resize_clamp</code>	static helper	<code>idx</code> , <code>limit</code>	Clamps resize indices to valid input bounds.
<code>pad_clamp_index</code>	static helper	<code>idx</code> , <code>limit</code>	Implements edge-padding index clamp.
<code>pad_reflect_index</code>	static helper	<code>idx</code> , <code>limit</code>	Implements reflect-padding index mapping.

Function	Visibility	Main variables	Purpose
<code>om_gemmini_resize_-nearest_f32</code>	public entry	input/output shapes, scale factors	CPU scalar nearest-neighbor resize.
<code>om_gemmini_resize_-linear_f32</code>	public entry	input/output shapes, interpolation weights	CPU scalar bilinear resize.
<code>om_gemmini_pad_-constant_f32</code>	public entry	pads, constant value	CPU scalar constant padding.
<code>om_gemmini_pad_-reflect_f32</code>	public entry	pads, reflected indices	CPU scalar reflect padding.
<code>om_gemmini_pad_-edge_f32</code>	public entry	pads, clamped indices	CPU scalar edge padding.
<code>om_gemmini_slice_f32</code>	public entry	starts, ends, axes, steps	CPU scalar slice extraction.
<code>om_gemmini_concat_-f32</code>	public entry	<code>a</code> , <code>b</code> , <code>axis</code>	CPU scalar two-input concat, commonly for NCHW tensors.
<code>om_gemmini_-transpose_f32_-scalar_impl</code>	static helper	<code>perm</code> , input/output strides	General scalar transpose implementation.
<code>om_gemmini_-transpose_perm_has_-hw_route</code>	static helper	<code>perm[4]</code>	Checks whether a rank-4 permutation is eligible for a future/hardware transposer path.
<code>om_gemmini_-transposer_hw_-available</code>	static helper	compile-time macro	Returns true only if a standalone transposer interface is exposed.
<code>om_gemmini_-transpose_f32_hw</code>	public entry	<code>out</code> , <code>x</code> , <code>perm0..<code>perm3</code></code>	Public transpose wrapper; prefers HW route if available, otherwise scalar fallback.
<code>om_gemmini_split_-copy_slice_f32</code>	static helper	<code>axis</code> , slice offsets	Copies one slice for Split output.
<code>om_gemmini_split_2_-f32</code>	public entry	two outputs	CPU scalar Split producing two tensors.
<code>om_gemmini_split_3_-f32</code>	public entry	three outputs	CPU scalar Split producing three tensors.
<code>om_gemmini_split_4_-f32</code>	public entry	four outputs	CPU scalar Split producing four tensors.
<code>om_gemmini_sigmoid_-f32</code>	public entry	<code>out</code> , <code>x</code> , <code>n</code>	CPU scalar element-wise sigmoid.
<code>om_gemmini_mul_f32</code>	public entry	<code>out</code> , <code>a</code> , <code>b</code> , <code>n</code>	CPU scalar element-wise multiplication.

18.6 Variable Dictionary by Runtime Area

18.6.1 Common OMTensor variables

Variable / accessor	Type or shape idea	Meaning
<code>OMTensor *out</code>	output tensor pointer	Destination tensor allocated by generated code or runtime wrappers. The runtime writes result data here.
<code>const OMTensor *x</code>	input tensor pointer	Main input tensor. In Conv, this is the feature map; in activation, it is the element-wise input.
<code>const OMTensor *w</code>	weight tensor pointer	Weight tensor. In Conv, ONNX uses OIHW: output channels, input channels, kernel height, kernel width.
<code>const OMTensor *b</code>	optional bias tensor	Bias vector, usually one value per output channel.
<code>omTensorGetDataPtr(t)</code>	raw pointer	Returns the data buffer. Runtime casts it to <code>float *</code> , <code>int8_t *</code> , <code>int32_t *</code> , etc.
<code>omTensorGetShape(t)</code>	<code>int64_t *</code> shape array	Gives dimension sizes. For NCHW Conv input, shape is <code>[N,C,H,W]</code> .
<code>omTensorGetStrides(t)</code>	<code>int64_t *</code> stride array	Number of elements to skip when increasing each dimension. Used to handle row-major indexing.
<code>omTensorGetRank(t)</code>	scalar int64	Number of dimensions. Conv expects rank 4; matrix multiply often expects rank 2 or batched rank-N.
<code>omTensorGetNumElems(t)</code>	scalar int64	Total element count. Used for allocation sizes, trace stats, and loop bounds.

18.6.2 Convolution variables

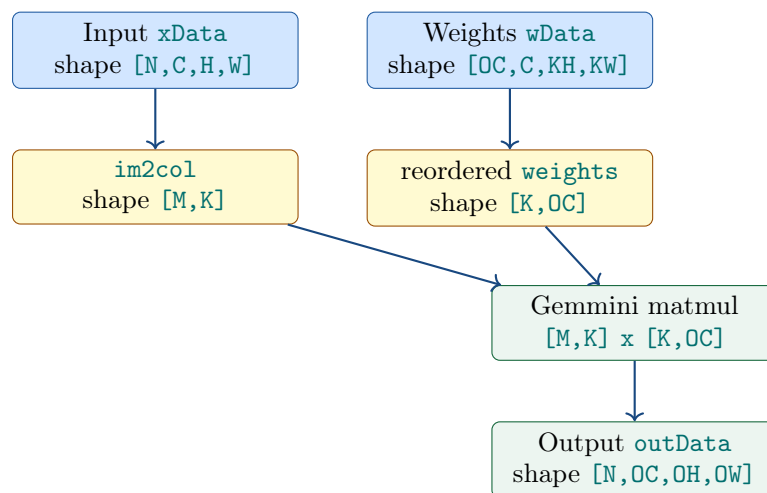


Figure 29: Meaning of the main convolution buffers in `om_gemmini_conv_f32_impl`.

Name	Example in first ResNet-18 Conv	Meaning
N	1	Batch size. Number of images processed at once.
C	3	Input channels. RGB image has 3 channels.
H, W	224, 224	Input height and width.
OC	64	Output channels / number of filters.
KH, KW	7, 7	Kernel height and width.
OH, OW	112, 112	Output height and width. Computed from input size, kernel, stride, and padding.
stride	2	How far the kernel moves each step. Larger stride reduces spatial size.
pad	3	Zero padding around the input spatial dimensions.
M	OH * OW	Number of im2col rows per image. Each row is one output pixel position.
K	C * KH * KW	Dot-product length for each output value.
xData	float *	Raw input feature-map buffer.
wData	float *	Raw convolution weight buffer in ONNX OIHW order.
outData	float *	Raw output feature-map buffer in NCHW order.
im2col	float [M*K]	Temporary matrix made from sliding input patches. Converts Conv into matrix multiplication.
weights	float [K*OC]	Temporary reordered weight matrix compatible with matrix multiplication.

Name	Example in first ResNet-18 Conv	Meaning
<code>acc</code>	<code>int32_t[M*OC]</code>	Gemmini accumulator output from int8 systolic matmul.
<code>outputScale</code>	float	Scale factor connecting int8 quantized math back to float values.
<code>dequant</code>	float	Float estimate reconstructed from Gemmini int32 accumulator.
<code>exact</code>	float	Exact float dot product used to preserve ONNX float semantics after exercising Gemmini.
<code>value</code>	float	Final output value after dequantization/residual correction and optional bias addition.

18.6.3 GEMM and MatMul variables

Name	Where used	Meaning
<code>M</code>	MatMul/GEMM/Conv-im2col	Number of output rows.
<code>N</code>	MatMul/GEMM	Number of output columns. Do not confuse with Conv batch size; variable names are local to each function.
<code>K</code>	MatMul/GEMM/Conv-im2col	Shared reduction dimension. Matrix product is $[M,K] \times [K,N]$.
<code>A, B, C</code>	GEMM	A and B are multiplicands. C is optional bias/initial matrix depending on GEMM attributes.
<code>dimI, dimJ, dimK</code>	int8 tiled matmul helper	Same idea as M, N, K, using names common in Gemmini helpers.
<code>strideA, strideB, strideC, strideD</code>	tiled matmul helper	Row stride values telling Gemmini how far to jump between rows.
<code>Aq, Bq</code>	float-to-int8 matmul helper	Quantized int8 temporary matrices created from float input matrices.
<code>acc</code>	float-to-int8 matmul helper	int32 accumulator matrix returned by Gemmini.
<code>alpha, beta</code>	GEMM	ONNX/GEMM scaling attributes: output is usually $\alpha * A * B + \beta * C$.

18.6.4 Quantization variables

Variable	Meaning
<code>xScale, wScale, yScale</code>	Scalar float tensors used by ONNX quantized operators to map real values to integer ranges and back.

Variable	Meaning
<code>xZeroPoint</code> , <code>wZeroPoint</code> , <code>yZeroPoint</code>	Integer offsets used by affine quantization. Some Gemmini paths restrict these to zero or scalar forms.
<code>qScale</code>	Combined quantization scale for convolution, commonly <code>xScale * wScale / yScale</code> .
<code>aZeroPoint</code> , <code>bZeroPoint</code>	Optional zero-points for MatMulInteger left and right operands.
<code>rowSumsA</code> , <code>colSumsB</code>	Temporary sums used to correct accumulator results when zero-points are not zero.
<code>adjustedBias</code>	Bias corrected to include input zero-point effects before running quantized convolution.

18.7 How to Trace a Runtime Function Back to Source

```

1 # 1. List the Runtime folder.
2 tree -L 3 $ROOT_DIR/src/Accelerators/Gemmini/Runtime
3
4 # 2. Find every public runtime symbol implemented in OMRuntimeGemmini.
   cpp.
5 grep -n '^void om_gemmini_' \
6     $ROOT_DIR/src/Accelerators/Gemmini/Runtime/OMRuntimeGemmini.cpp
7
8 # 3. Find private helper functions.
9 grep -n '^static ' \
10    $ROOT_DIR/src/Accelerators/Gemmini/Runtime/OMRuntimeGemmini.cpp
11
12 # 4. Find where convolution is implemented.
13 grep -n 'om_gemmini_conv_f32' \
14    $ROOT_DIR/src/Accelerators/Gemmini/Runtime/OMRuntimeGemmini.cpp
15
16 # 5. Find where Gemmini hardware helpers are called.
17 grep -n 'tiled_matmul_auto\|tiled_conv_auto\|gemmini_fence' \
18    $ROOT_DIR/src/Accelerators/Gemmini/Runtime/OMRuntimeGemmini.cpp
19
20 # 6. Confirm the final archive contains the runtime symbol.
21 llvm-nm $OUT_DIR/libgemmini_f32_rv64.a | grep om_gemmini_conv_f32

```

Listing 55: Commands for locating Runtime folder ownership

Reading the result

If `grep ^void om_gemmini_` shows a function, it is a public runtime entry point. If `grep ^static` shows it, it is private to `OMRuntimeGemmini.cpp`. If `llvm-nm libgemmini_f32_rv64.a` shows the symbol, the linker can resolve calls from `resnet18.o`. If the Spike log prints `[gemmini-trace] conv f32`, execution reached the runtime Conv function and the trace code ran.

18.8 One Function Fully Explained: `om_gemmini_conv_f32`

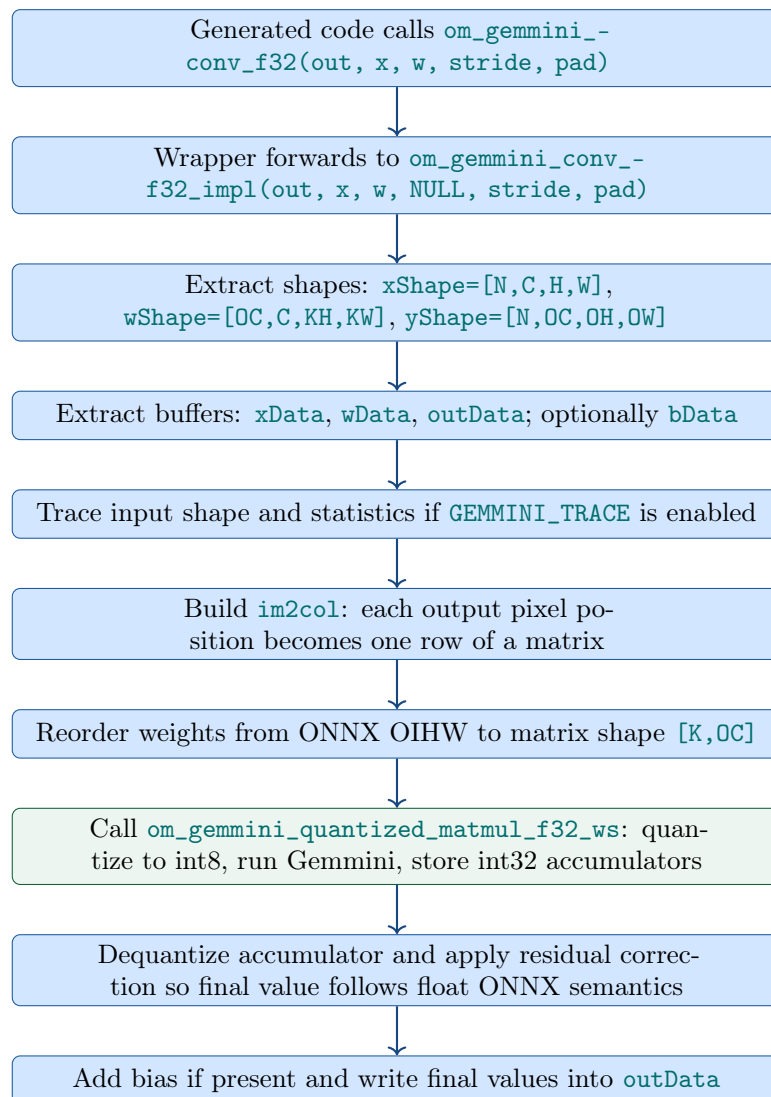


Figure 30: Detailed execution sequence of the float convolution runtime path.

```

1 void om_gemmini_conv_f32(
2     OMTensor *out, const OMTensor *x, const OMTensor *w,
3     int64_t stride, int64_t pad) {
4     om_gemmini_conv_f32_impl(out, x, w, /*b=*/NULL, stride, pad);
5 }

```

Listing 56: Conceptual structure of the Conv wrapper

```

1 // 1. Read tensor metadata.
2 const int64_t *xShape = omTensorGetShape(x); // [N,C,H,W]
3 const int64_t *wShape = omTensorGetShape(w); // [OC,C,KH,KW]
4 const int64_t *yShape = omTensorGetShape(out); // [N,OC,OH,OW]
5 float *xData = (float *)omTensorGetDataPtr(x);
6 float *wData = (float *)omTensorGetDataPtr(w);
7 float *outData = (float *)omTensorGetDataPtr(out);
8
9 // 2. Convert Conv to GEMM.

```

```

10 // im2col shape: [OH*OW, C*KH*KW]
11 // weights shape: [C*KH*KW, OC]
12
13 // 3. Run Gemmini-backed quantized matmul and correct back to f32.
14 om_gemmini_quantized_matmul_f32_ws(M, OC, K, im2col, K, weights, OC,
15                                     acc, &outputScale);

```

Listing 57: Conceptual structure of Conv implementation

18.9 Hardware-backed vs CPU-scalar Runtime Functions

Function family	Execution style	Reason
<code>matmulinteger</code> , <code>qlinearconv</code>	Gemmini hardware path	They already use int8 data, matching the Gemmini systolic path.
<code>matmul_f32</code> , <code>gemm_f32</code> , <code>conv_f32</code>	Gemmini exercised through quantized matmul plus correction	The local Gemmini profile is int8/i32. The runtime quantizes float inputs, runs the accelerator, then corrects output to preserve float semantics.
<code>relu</code> , <code>add</code> , <code>batchnorm</code> , <code>pool</code> , <code>softmax</code>	CPU scalar loops in this runtime	These are simple element-wise or reduction loops. They are still runtime calls, but not necessarily RoCC matrix operations.
<code>resize</code> , <code>pad</code> , <code>slice</code> , <code>concat</code> , <code>split</code> , <code>sigmoid</code> , <code>mul</code>	CPU scalar loops	They are data movement or element-wise operations; the runtime implements correctness first.
<code>transpose_f32_hw</code>	HW-preferred with scalar fallback	The function checks whether a standalone hardware transposer route is available. If not, it uses scalar transpose.

18.10 Why This Runtime Design Is Useful

Design choice	Benefit
Keep the compiled model small	The model object only calls <code>om_gemmini_*</code> . It does not duplicate all convolution/GEMM implementation logic in every model.
Use OMTensor pointers	One ABI can support many tensor shapes and operators. The runtime reads shapes and data pointers at runtime.
Put Gemmini-specific logic in one folder	Hardware interaction is centralized in <code>Runtime/</code> ; compiler passes only need to emit the correct calls.
Keep scalar fallback/correctness loops	Operators not worth accelerating or not exposed by the Gemmini ABI can still produce correct results.
Add <code>GEMMINI_TRACE</code>	You can prove at execution time which runtime functions were reached, what shapes were used, and whether values stayed finite.